

高并发下，如何设计秒杀系统？这是一个高频面试题。

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、shein 希音、shopee、百度、网易的面试资格，遇到很多很重要的面试题：

每秒上万次下单请求，秒杀如何处理？

高并发下，如何设计秒杀系统？

如何解决秒杀超卖？

前几天 小伙伴面试 shopee，这个问题 没有回答好，导致面试挂了。小伙伴面试完了之后，来求助尼恩：如何才能回答得很漂亮，才能 让面试官刮目相看、口水直流。

所以，尼恩给大家做一下系统化、体系化的梳理，使得大家内力猛增，可以充分展示一下大家雄厚的“技术肌肉”，**让面试官爱到“不能自己、口水直流”**，然后实现“offer直提”。

当然，这道面试题，以及参考答案，也会收入咱们的 [《尼恩Java面试宝典》](#) V145版本PDF集群，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

最新《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF，请关注本公众号【技术自由圈】获取，后台回复：领电子书

虽说秒杀只是一个促销活动，但对架构的要点非常多。下面给大家总结一下设计秒杀架构的16个架构要点，组成一个《高并发秒杀圣经》。

秒杀的16大架构

by
架构师
尼恩

1 秒杀系统的业务架构

2 秒杀系统的流量架构

3 秒杀的异步架构

4 电商系统的分层架构

5 秒杀的动静分离架构

6 秒杀的缓存架构

7 库存扣减场景下的数据一致性架构

8 秒杀的mq异步架构

9 秒杀高可用架构

10 秒杀的限流架构

11 秒杀的熔断架构

12 秒杀的降级架构

13 存储层面的高可用架构

14 运维部署层面的架构

15 秒杀的多IDC机房双活高可用架构

16 秒杀的应急预案架构

高并发 10Wqps秒杀架构，可以说 16大架构杀招 / 16大架构绝招 大集结，

掌握 16大架构杀招 / 16大架构绝招 ， 帮你秒变架构师。

一步一步来。

接下来，咱们一个一个 进行 架构杀招 展开和梳理

1 秒杀系统的业务架构

1.1 秒杀系统的业务特点

秒杀，通常指的是在电商平台上进行的限时抢购活动，通常以极低的价格限量出售某些商品，吸引大量用户在短时间内进行购买。

天猫、京东、拼多多等，都会定期举行秒杀活动，特别是在双十一、618等大型购物节期间。

这些活动不仅能够大幅提升平台的销量和流量，还能增强用户黏性和平台影响力。

秒杀活动具有以下几个特点：

1. 时间紧迫：

秒杀活动通常设置在特定的时间段内进行，时间限制通常非常短，从几分钟到几小时不等。用户必须在规定时间内完成购买，增加了活动的紧迫感和参与热情。

2. 数量有限：

秒杀商品的数量通常非常有限，远少于潜在购买者的数量。这种稀缺性促使用户迅速采取行动，生怕错过机会。

3. 价格极低：

秒杀商品的价格通常比市场价低得多，甚至可能低于成本价。超低的价格吸引了大量用户参与，形成抢购热潮。

4. 竞争激烈：

由于时间紧迫和数量有限，用户之间的竞争非常激烈。很多人会提前准备，甚至使用抢购软件，尽一切可能在最短时间内完成购买。

5. 营销效果强：

秒杀活动具有很强的营销效果。超低价格和紧迫感吸引了大量用户访问和参与，提升了品牌知名度和用户活跃度。即使用户没有成功购买，也可能被吸引到平台上的其他商品和活动中。

6. 社交传播：

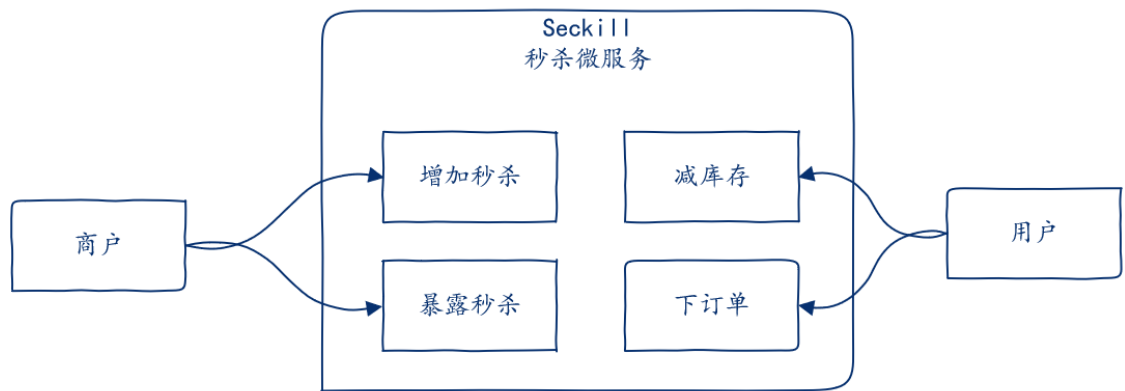
秒杀活动容易引发用户之间的讨论和分享。用户可能会通过社交媒体分享秒杀信息，进一步扩大活动的影响力和参与度。

1.2 秒杀暴露与秒杀按钮

从系统角度来说，秒杀系统的业务流程如图1所示，分成两大维度：

(1) 商户维度的业务流程；

(2) 用户维度的业务流程。



<https://blog.csdn.net/crazymakercircle>

图1 秒杀系统的业务流程

一、商户维度的业务流程，主要涉及两个操作：

(1) 增加秒杀

通过后台的管理界面，增加特定商品、特定数量、特定时段的秒杀。

(2) 暴露秒杀

将符合条件的秒杀，暴露给用户，以便互联网用户能参与商品的秒杀。这个操作可以是商户手动完成，更合理的方式是系统自动维护。

暴露秒杀，就是将符合条件的秒杀暴露给用户，以便互联网用户能参与商品的秒杀。

这个操作可以由商户手动完成，在生产场景下，更合理的方式是系统自动维护。

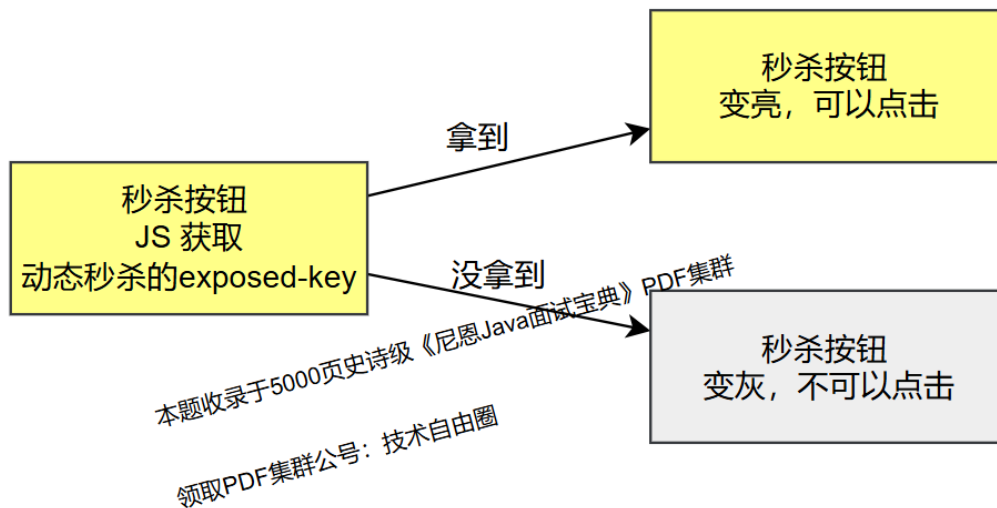
大部分用户怕错过 秒杀时间点，一般会提前进入活动页面。

此时看到的 秒杀按钮 是置灰，不可点击的，秒杀还没有暴露。

但此时很多用户已经迫不及待了，通过不停刷新页面，争取在第一时间看到秒杀按钮的点亮。

从前面得知，该活动页面是静态的。那么我们在静态页面中如何控制秒杀按钮，只在秒杀时间点时才点亮呢？

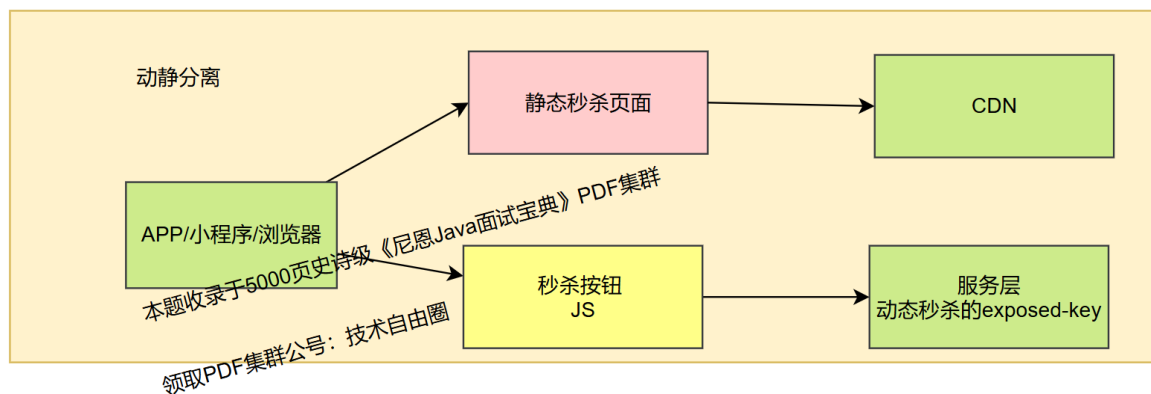
只有到了秒杀时间点那一时刻，秒杀已经暴露，秒杀暴露主要是生成一个 具备实效性的 exposed-key，通过动态秒杀JS异步获取 exposed-key，秒杀按钮才会自动点亮，变成可点击的。



另外，在客户端这一层的用户交互上需要具备一定的控制用户行为和禁止重复秒杀的能力。

比如，当用户提交秒杀请求之后，可以将秒杀按钮置灰，禁止重复提交。

- 1、秒杀开始前，秒杀按钮灰掉为“未开始”，不可点击。
- 2、URL在活动开始前不可露出或者生效，否则容易被使用工具绕过浏览器提前下单。
导致活动还未开始，已经开始下单这个大黑洞。最好做法是在活动开始前，通过JS文件露出下单的URL。
- 3、在秒杀进行中，秒杀按钮才可以点击下单。
- 4、秒杀产品的介绍，详情，参数等等，全部静态化，将秒杀商品页面缓存在CDN上（如果没有CDN，也可以放在Nginx中做动静分离）
- 5、用户点击“下单”后，按钮置灰，禁止用户重复提交请求，限制用户在60秒之内只能提交一次请求。（防止DDOS攻击，后面有讲）
- 6、然后就发送请求了，请求统一发送到Nginx中

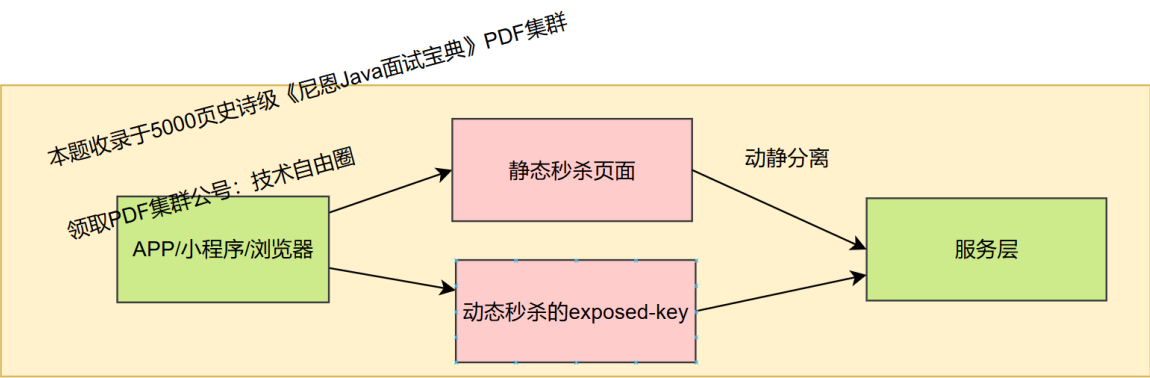


此外，前端还可以加一个定时器，控制比如：60秒之内，只允许发起一次请求。

如果用户点击了一次秒杀按钮，则在60秒之内置灰，不允许再次点击，等到过了时间限制，又允许重新点击该按钮。

秒杀暴露的动作是定时完成的，只有到了秒杀时间点，才开始暴露。

并且，也只有到了秒杀时间点，用户主动点了秒杀按钮才允许访问服务端。

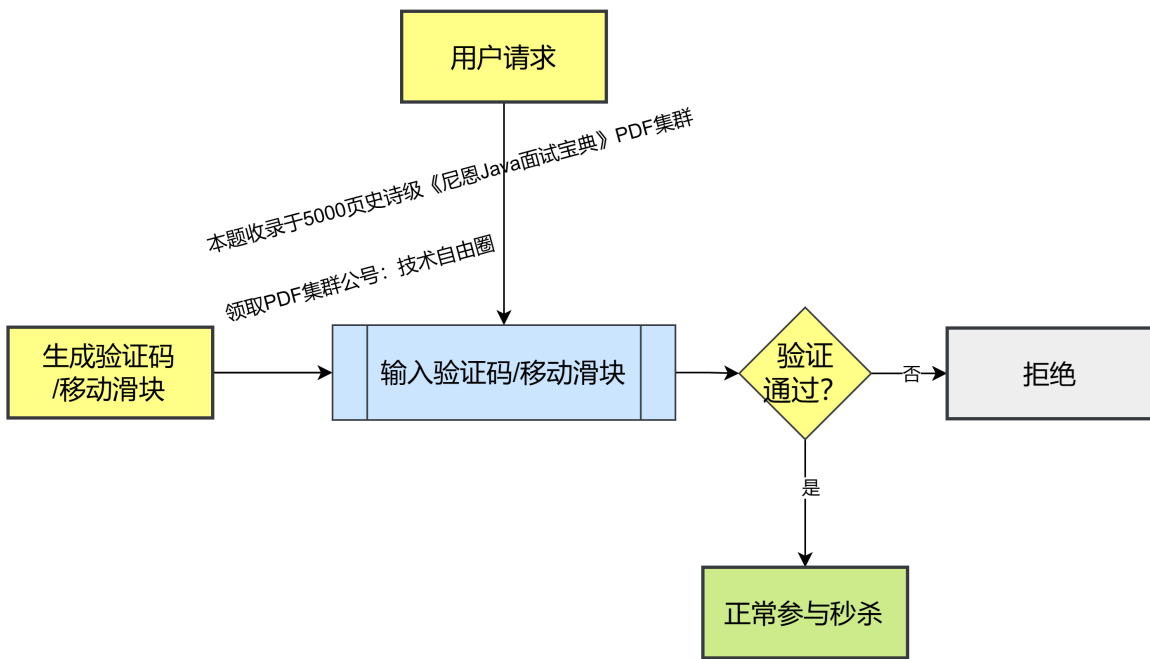


这样能过滤大部分无效请求。

1.3 秒杀的验证码设计

在秒杀活动中添加验证码是为了防止恶意刷单和机器人攻击，确保活动的公平性和安全性。

加验证码的方式，同样能限制用户的访问频次，同时和限流不同，加验证码不会存在误杀的情况。



通常情况下，用户在请求之前，需要先输入验证码。

用户发起请求之后，服务端会去校验该验证码是否正确。

只有正确才允许进行下一步操作，否则直接返回，并且提示验证码错误。

此外，验证码一般是一次性的，同一个验证码只允许使用一次，不允许重复使用。

普通验证码，由于生成的数字或者图案比较简单，可能会被破解。

普通验证码 优点是生成速度比较快，缺点是有安全隐患。

还有一个验证码叫做：移动滑块，它生成速度比较慢，但比较安全，是目前各大互联网公司的首选。

2 秒杀系统的流量架构

2.1 秒杀系统的流量特点

秒杀系统的3个核心特点：

秒杀的特点一：限时、限量、限价

- **限时**：秒杀活动如同“昙花一现”，在规定的时间内进行。举例来说，活动仅限于某天上午10点到10点半，过时不候。
- **限量**：商品数量如“凤毛麟角”，秒杀活动中商品的数量有限，譬如只有10万件，售完为止。
- **限价**：价格低廉，犹如“白菜价”。商品价格远远低于原来的价格，例如1元购等业务场景，吸引众多用户争先恐后。
- 这些限制条件可以“单打独斗”，也可以“联袂登场”，相辅相成，增加活动的吸引力和紧迫感。

秒杀的特点二：活动预热

- **提前配置**：活动需要未雨绸缪，提前配置好各项内容，做到“未雨绸缪”。
- **信息展示**：活动尚未开始时，用户可以查看相关信息，做到“心中有数”。
- **大力宣传**：在秒杀活动开始前，广而告之，宣传造势，做到“声势浩大”。

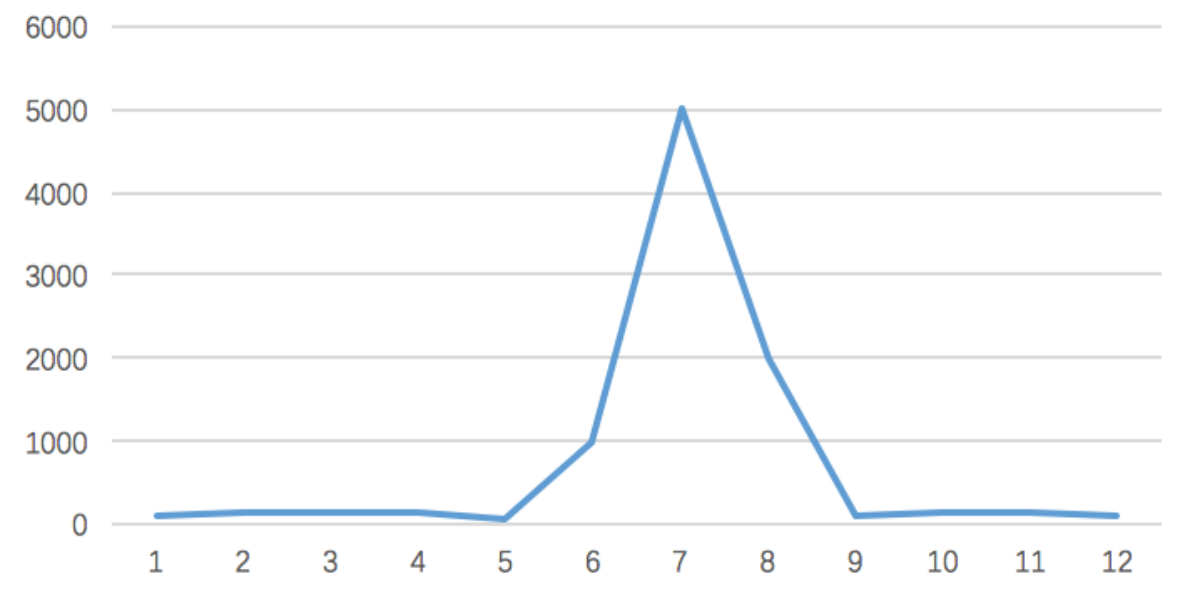
秒杀的特点三：持续时间短

- **大量用户**：购买人数如“过江之鲫”，商品会迅速售罄。
- **高并发访问**：系统流量如“井喷”般激增，并发访问量极高。大多数秒杀场景下，商品在“转瞬之间”即被抢购一空，宛如“争分夺秒”的竞技场。

由于秒杀系统的3个核心特点，导致了 秒杀系统的流量特点：

- 瞬时凸峰 (流量突刺)
- 漏斗模型

秒杀系统的并发量存在瞬时凸峰的特点，也叫做流量突刺现象。 可以使用下图来表示：



比如，小米秒杀系统的流量峰值

1. 秒杀前访问量平稳：在秒杀活动开始前，小米秒杀系统的访问量相对平稳，没有显著变化。
2. 秒杀时刻的并发量激增：秒杀活动通常在上午10点开始，活动开始时，系统的并发访问量会瞬时激增，导致流量出现突刺现象。

秒杀的流量特点，与12306网站的春运访问量特点类似。

来看12306的特点：

1. 平时访问量平缓：在平常日子里，12306网站的访问量相对稳定，没有显著的波动。
2. 春运期间访问量激增：每年春运时节，12306网站的访问量会出现瞬时突增的现象。大量用户在短时间内涌入网站，导致访问量急剧上升。

2.2 高并发 吞吐量 规划和评估

吞吐量评估

吞吐量评估是指我们需要评估好吞吐量，

我们这个系统，是为了应对一个什么体量的业务，这个业务请求量的平均值、高峰的峰值大概都在一个什么级别。

如果是新系统，那么就需要根据产品和运营同学对业务有一个大体的预估，然后开发同学根据产品给的数据再进行详细的评估。如果是老系统，那么就可以根据历史数据来评估。

评估的时候，要从一个整体角度来看全局的量级，然后再细化到每个子业务模块要承载的量级。

吞吐量规划，

是指我们系统在设计的时候，就要能够初步规划好我们的系统大致能够抗多少的量级，比如是十万还是百万级别的请求量，或者更多。

不同的量级对应的系统架构的设计会完全不一样，尤其到了千万、亿级别的量级的时候，架构的设计会有很多的考量。

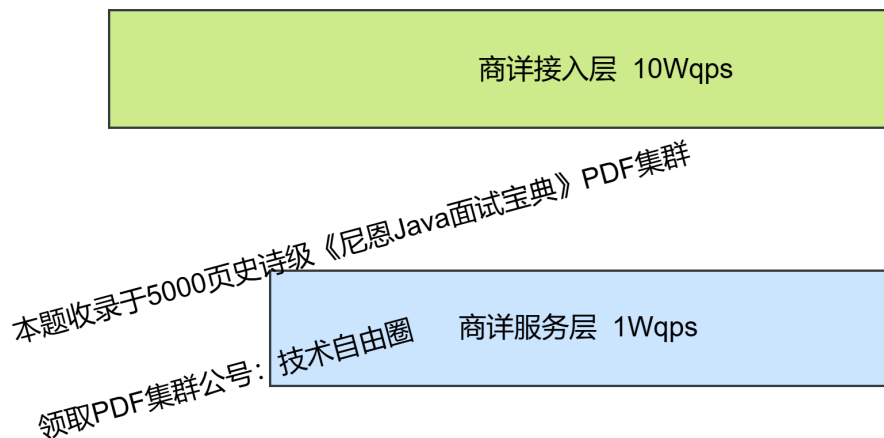
同时，吞吐量规划还涉及到，我们系统上下游的各个模块、依赖的存储、依赖的三方服务，分别需要多少资源，需要有一个相对可以量化的数据出来。

容量规划阶段，更多是要依靠自身和团队的经验，比如要了解我们的 log 的性能、redis 的性能、rpc 接口的性能、服务化框架的性能等等，然后根据各种组件的性能来综合评估自己设计的系统的整体性能情况。

2.3 QPS 预估（漏斗型）

QPS 预估（漏斗型），指的是一个真实的请求过来后，从接入层开始，分别经过了我们整个系统的哪些层级、哪些模块，然后每一个层级的 QPS 的量级分别有多少，从请求链路上来看，层级越往下，那么下游层级的量级应该会逐步减少的，因为每经过一个层级，都有可能被各种条件过滤掉的一部分请求。

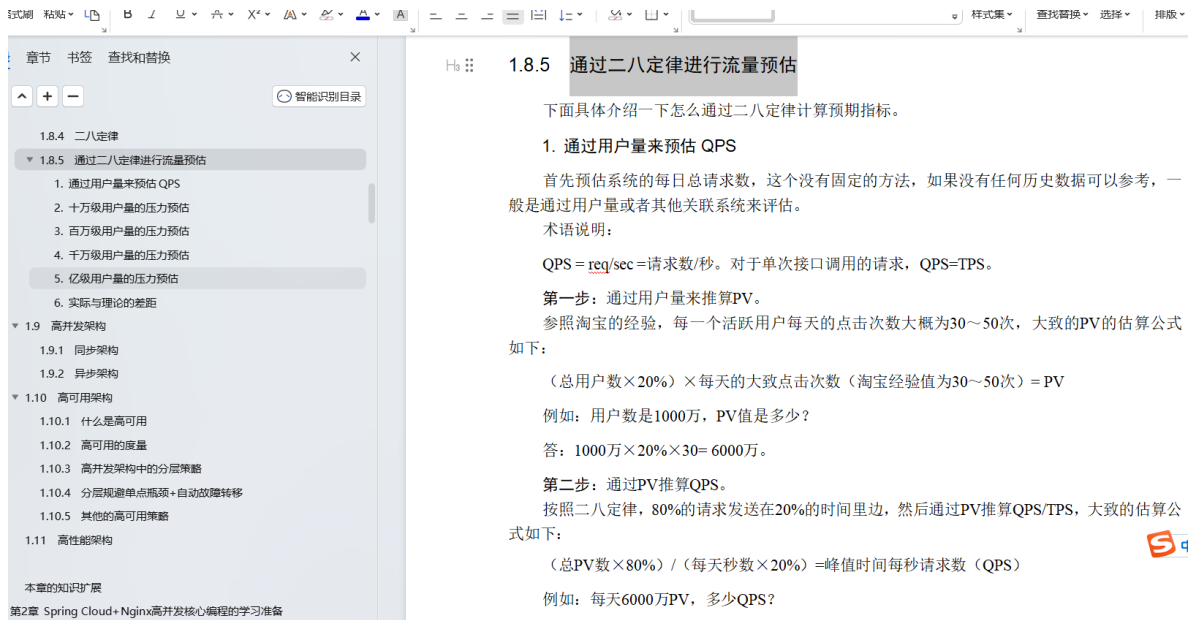
比如说进入商品详情 这个例子。



QPS 预估（漏斗型）就是需要我们预估好每一个层级的量级，包括但不限于从服务、接口、分布式缓存等各个层面来预估，最后构成我们完整的 QPS 漏斗模型。

2.4 通过二八定律进行流量预估

具体请参见 尼恩Java 高并发核心编程 卷3：



2.5 性能压测评估

容量评估和容量规划之后，我们还需要做一件事情，就是性能压测评估，最好是能够做到全链路压测。

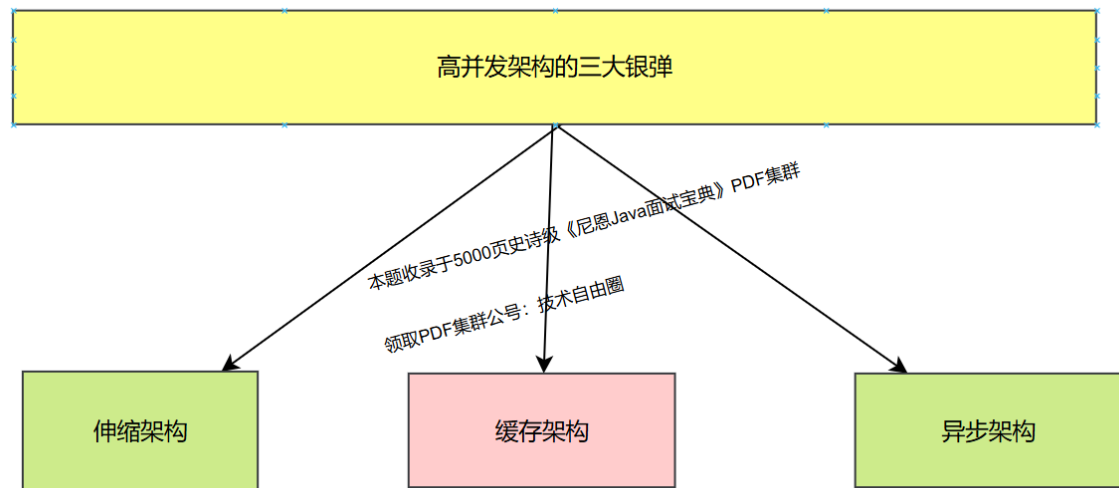
性能压测的目的是为了确保你的容量规划是准确的，比如我设计的这个系统，我规划的是能够抗千万级别的请求，那么实际上，真的能够抗住吗？

这个在上线之前，首先要根据经验来判断，然后是一定要经过性能压测得出准确结论的。

性能压测要关注的指标很多，但是重点要关注是两个指标，一个是 QPS、一个是响应耗时，要确保压测的结果符合预期。

压测的步骤可以先分模块单独压测，最后如果情况允许，那么最好执行全链路压测。

3 秒杀的异步架构

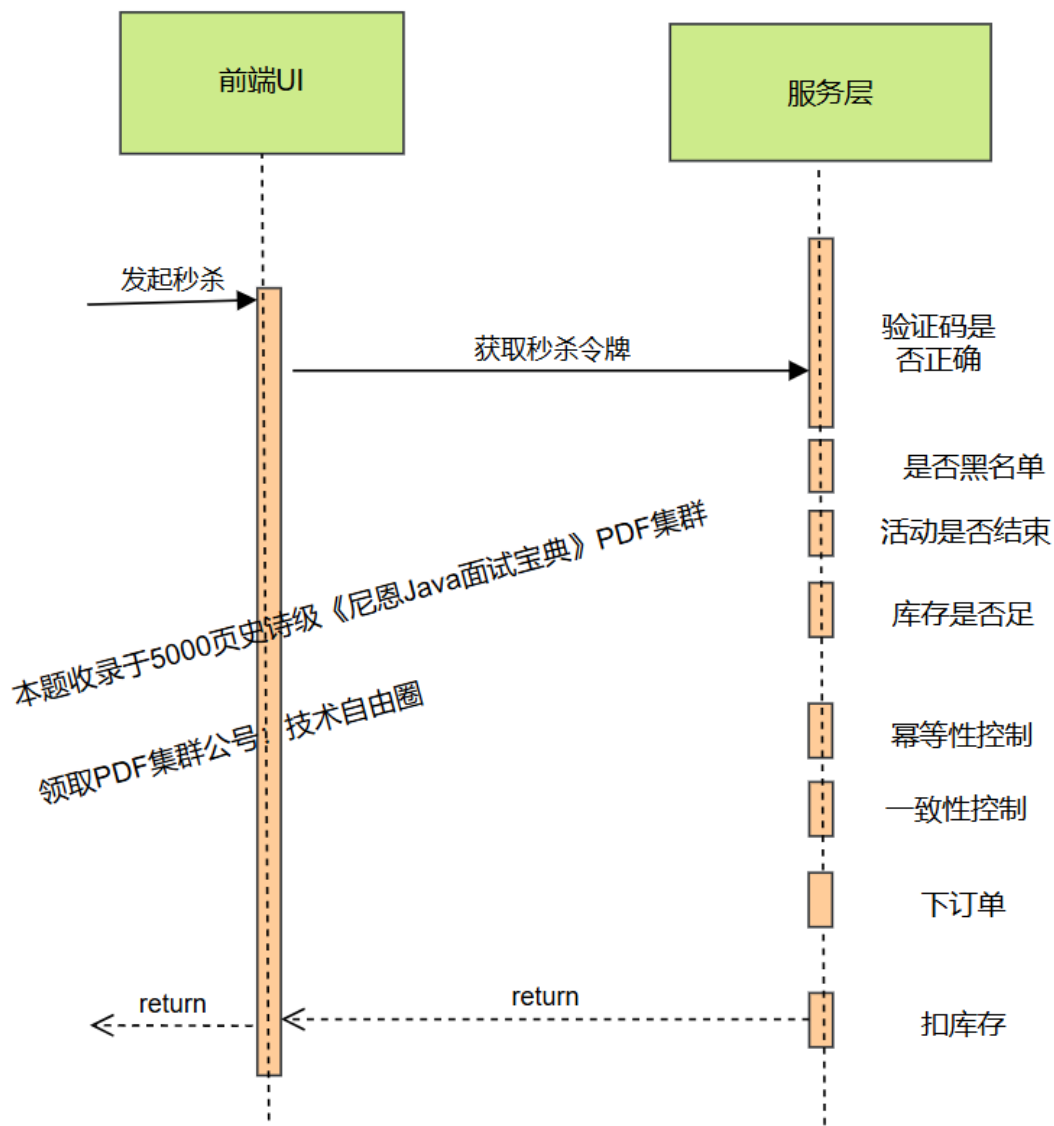


10Wqps或者100Wqps场景，采用的同步处理请求的方案，是万万不能的，一旦并发量真的上来了，他们所谓的秒杀系统的性能会急剧下降。

一定要采用异步架构

3.1 同步模式的下单流程

我们先来看一下秒杀系统在同步下单时的时序图：



在同步下单流程中，用户发起秒杀请求后，商城服务需要依次执行以下步骤来处理秒杀请求的业务：

1. 识别验证码是否正确

商城服务需“火眼金睛”，判断用户提交的验证码是否正确，确保只有“名副其实”的用户才能继续操作。

2. 判断活动是否已经结束

验证当前秒杀活动是否已经“曲终人散”，防止用户在活动结束后依然发起请求。

3. 验证访问请求是否处于黑名单

在电商领域中，恶意竞争犹如“鬼蜮伎俩”。其他商家可能会通过不正当手段占用系统资源。

此时，商城服务需要使用风控系统等实现黑名单机制，如“庖丁解牛”般识别并拦截恶意请求。为了简单，也可以使用拦截器统计访问频次，形成“黑名单”。

4. 验证真实库存是否足够

系统需“运筹帷幄”，验证商品的真实库存是否充足，确保能够支持本次秒杀活动的需求。

5. 扣减缓存中的库存

在秒杀业务中，商品库存等信息通常存放在缓存中。此时，需要“未雨绸缪”，验证并扣减秒杀活动商品的库存，确保“货真价实”。

6. 计算秒杀的价格

由于秒杀活动中的商品价格与真实价格存在差异，需要“精打细算”，计算商品的秒杀价格，确保用户享受到“物超所值”的优惠。

7. 下订单

将用户提交的订单信息“名正言顺”地保存到数据库中，确保每一笔交易都有据可查，如同“案牍劳形”般细致入微。

8. 扣减真实库存

订单入库后，需要在商品的真实库存中扣除本次成功下单的商品数量，确保库存数据“货真价实”，不出现“巧妇难为无米之炊”的尴尬局面。

注意：在实际的秒杀场景中，如果系统涉及的业务更加复杂，还会涉及更多的业务操作。这里只是“管中窥豹”，列举了一些常见的业务操作。

同步模式，当用户发起秒杀请求时，由于系统每个业务流程都是串行执行的，整体上系统的性能不会太高，当并发量太高时，我们会为用户弹出下面的排队页面，来提示用户进行等待。

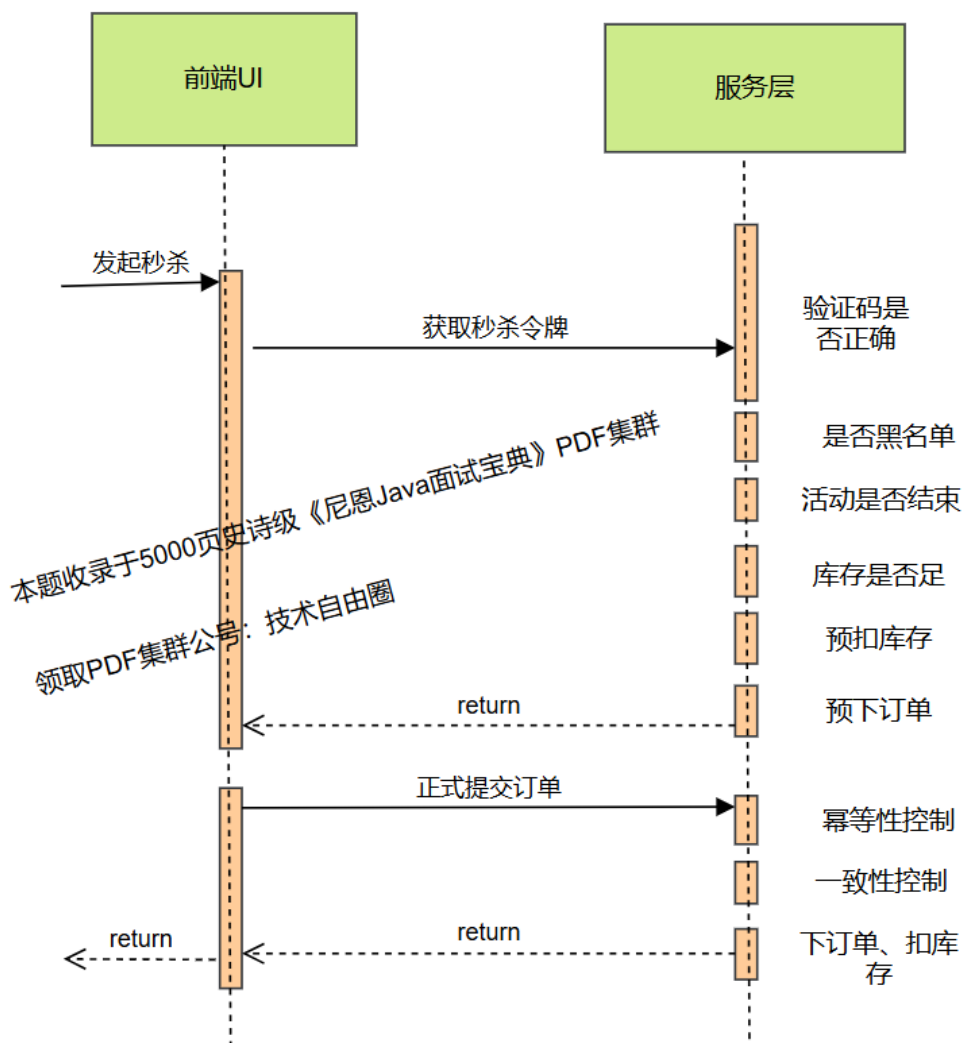
如果12306、淘宝、天猫、京东、小米等大型商城的秒杀系统是这么玩的话，那么，他们的系统迟早会被玩死，他们的系统工程师不被开除才怪！

所以，在秒杀系统中，这种同步处理下单的业务流程的方案是不可取的。

3.2、两阶段异步下单流程

既然同步下单流程的秒杀系统称不上真正的秒杀系统，那我们就要采用异步的下单流程了。

异步的下单流程不会限制系统的高并发流量。



在两阶段异步 下单流程中，用户发起秒杀请求后，商城服务分为两个阶段处理请求的业务。

第一个阶段 预下单阶段：

1. 识别验证码是否正确

商城服务需“火眼金睛”，判断用户提交的验证码是否正确，确保只有“名副其实”的用户才能继续操作。

2. 判断活动是否已经结束

验证当前秒杀活动是否已经“曲终人散”，防止用户在活动结束后依然发起请求。

3. 验证访问请求是否处于黑名单

在电商领域中，恶意竞争犹如“鬼蜮伎俩”。其他商家可能会通过不正当手段占用系统资源。

此时，商城服务需要使用风控系统等实现黑名单机制，如“庖丁解牛”般识别并拦截恶意请求。为了简单，也可以使用拦截器统计访问频次，形成“黑名单”。

4. 验证真实库存是否足够

系统需“运筹帷幄”，验证商品的真实库存是否充足，确保能够支持本次秒杀活动的需求。

5. 扣减缓存中的库存

在秒杀业务中，商品库存等信息通常存放在缓存中。此时，需要“未雨绸缪”，验证并扣减秒杀活动商品的库存，确保“货真价实”。

第二个阶段，正式下单阶段：

1. 计算秒杀的价格

由于秒杀活动中的商品价格与真实价格存在差异，需要“精打细算”，计算商品的秒杀价格，确保用户享受到“物超所值”的优惠。

2. 下订单

将用户提交的订单信息“名正言顺”地保存到数据库中，确保每一笔交易都有据可查，如同“案牍劳形”般细致入微。

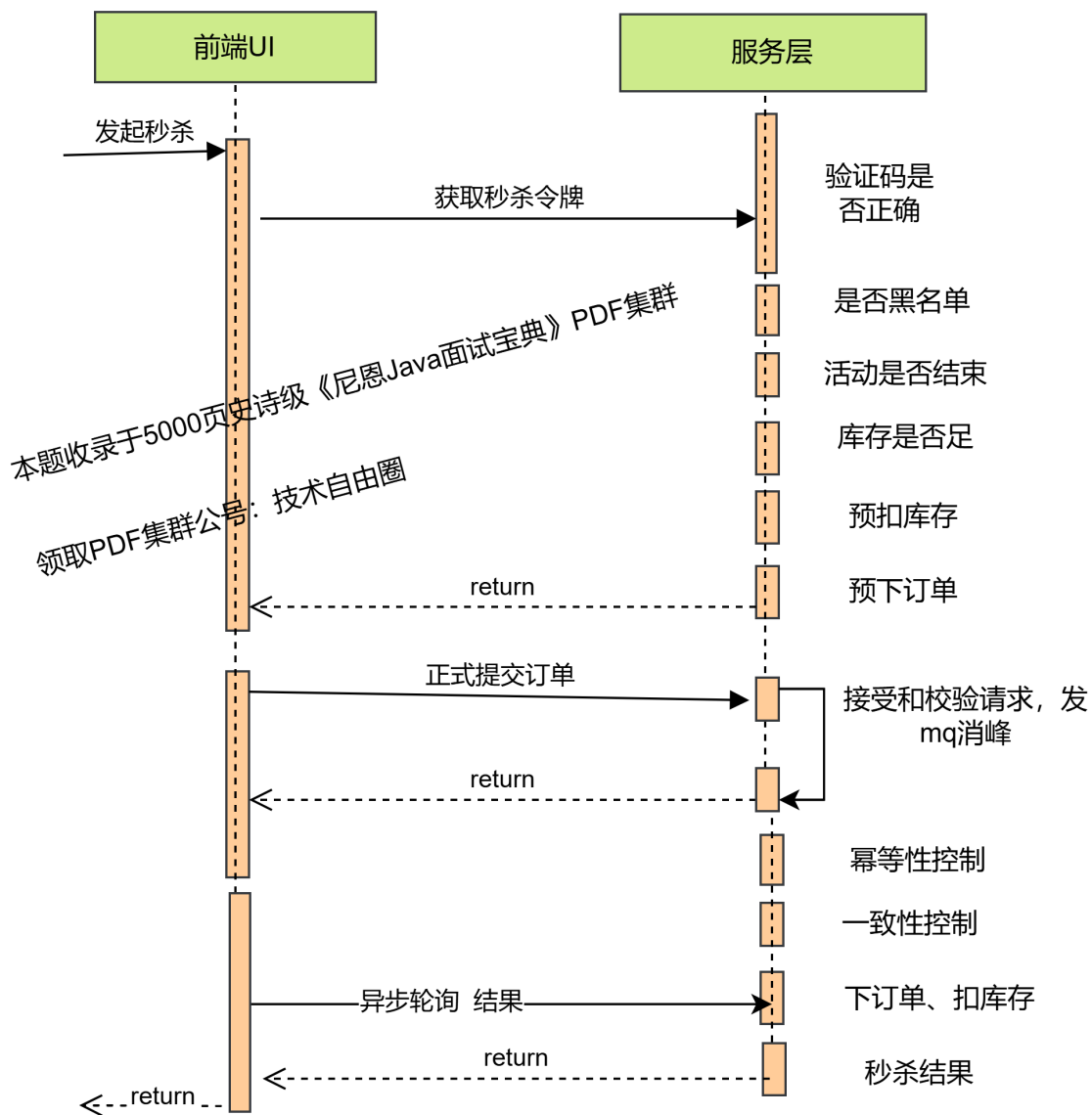
3. 扣减真实库存

第一个阶段 和 第二个阶段进行解耦， 是异步。

如果第一个阶段失败， 就没有必要进入第二阶段了。

3.3、三阶段异步下单流程

由于下单阶段很耗时， 在服务端 还可以解耦：



引入了MQ异步处理机制，同时可以返回 response 给前端，可以让io线程解除阻塞，去处理下一个请求。

前端可以短轮询查询秒杀结果。

短轮询查询秒杀结果

采用短轮询查询秒杀结果时，在页面上我们同样可以提示用户排队处理中，但是此时客户端会每隔几秒轮询服务器查询秒杀结果，相比于同步下单流程来说，无需长时间占用请求连接。

4 电商系统的分层架构

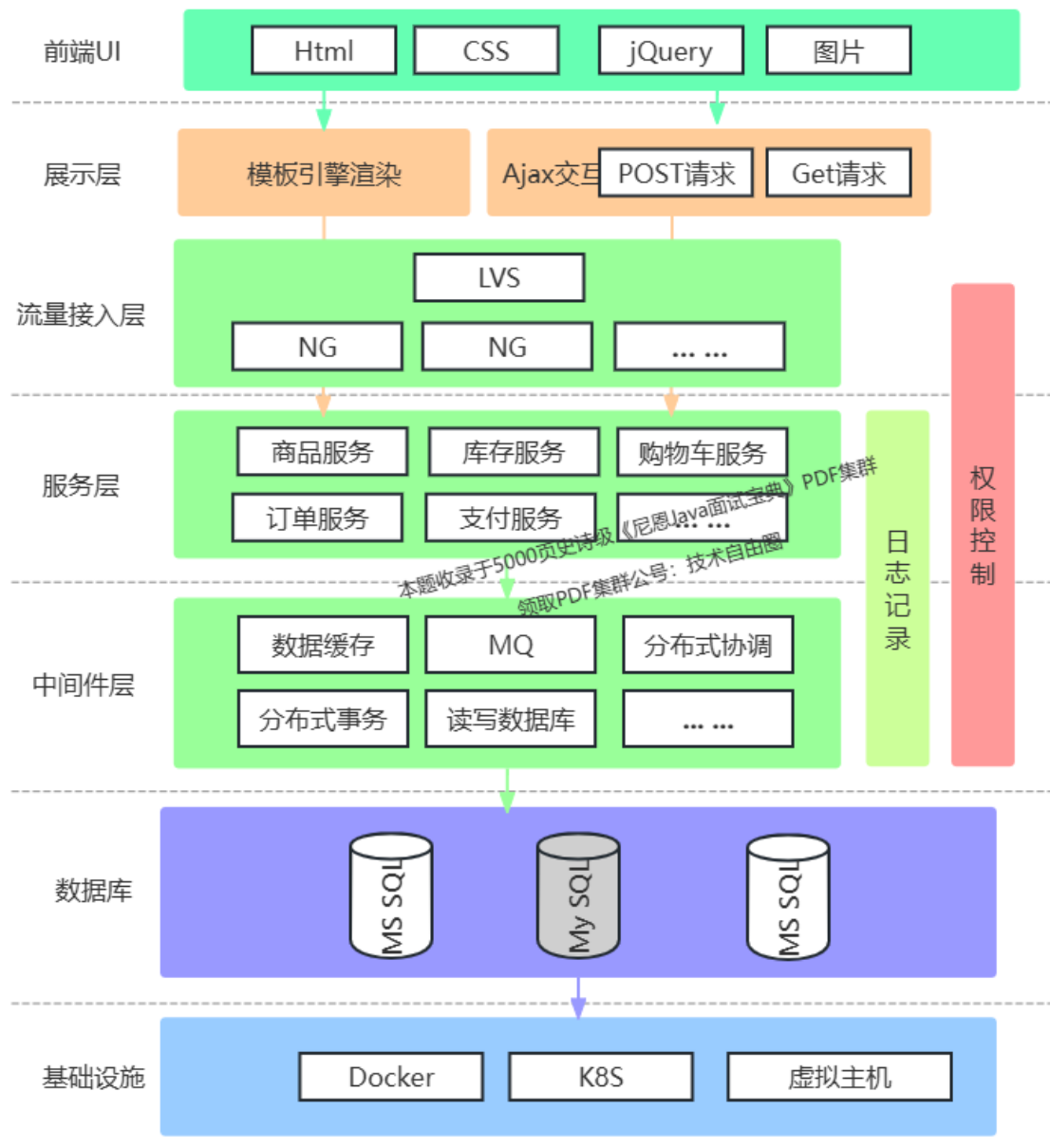
4.1 经典电商系统的分层架构

在电商领域，存在着典型的秒杀业务场景，那何谓秒杀场景呢。

简单的来说就是一件商品的购买人数远远大于这件商品的库存，而且这件商品在很短的时间内就会被抢购一空。

比如每年的618、双11大促，小米新品促销等业务场景，就是典型的秒杀业务场景。

我们可以将电商系统的架构简化成下图所示：



由图所示，我们可以简单的将电商系统的核心层分为：接入层、服务层和持久层。

接下来，我们就预估下每一层的并发量。

- 假如流量接入层使用的是高性能的Nginx，则我们可以预估Nginx最大的并发度为：10W+，这里是以万为单位。
- 假设服务层我们使用的是Tomcat，而Tomcat的最大并发度可以预估为800左右，这里是以百为单位。
- 假设持久层的缓存使用的是Redis，数据库使用的是MySQL，MySQL的最大并发度可以预估为1000左右，以千为单位。Redis的最大并发度可以预估为5W左右，以万为单位。

4.2 秒杀系统的分层架构

从分层的角度来说，秒杀系统架构可以分成3层，大致如下：

- 1) 客户端：负责内容提速和交互控制。
- 2) 接入层：负责认证、负载均衡、限流。
- 3) 业务层：负责保障秒杀的数据一致性。

1. 客户端负责内容提速和交互控制

客户端需要完成秒杀商品的静态化展示。无论是在桌面浏览器还是移动端APP上展示秒杀商品，秒杀商品的图片和文字元素都需要尽可能静态化，尽量减少动态元素，这样就可以通过CDN来提速和抗峰值。

另外，在客户端这一层的用户交互上需要具备一定的控制用户行为和禁止重复秒杀的能力。比如，当用户提交秒杀请求之后，可以将秒杀按钮置灰，禁止重复提交。

2. 接入层负责认证、负载均衡、限流

秒杀系统的特点是并发量极大，但实际的优惠商品有限，秒杀成功的请求数量很少，所以如果不在接入层进行拦截，则大量请求会造成数据库连接耗尽、服务端线程耗尽，导致整体雪崩。因此，必须在接入层进行用户认证、负载均衡、接口限流。

对于总流量较小的系统，可以在内部网关（如Zuul）完成认证、负载均衡、接口限流的功能，具体的分层架构如图11-2所示。

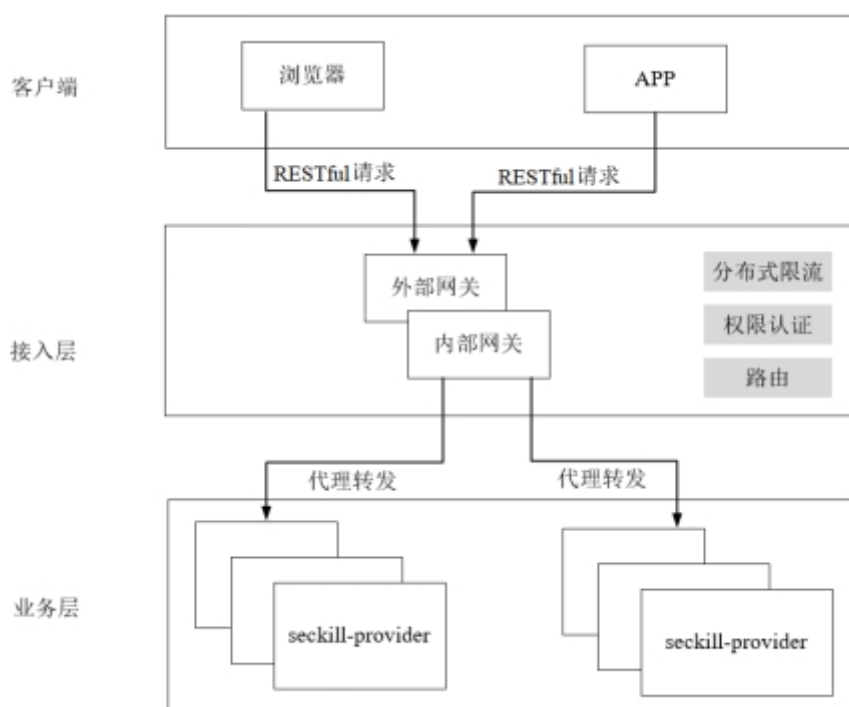


图11-2 内部网关（如Zuul）完成认证、负载均衡、接口限流

对于总流量较大的系统，会有一层甚至多层外部网关，因此，限流的职责会从内部网关剥离到外部网关，内部网关（如Zuul）仍然具备权限认证、负载均衡的能力，具体的分层架构如图11-3所示。

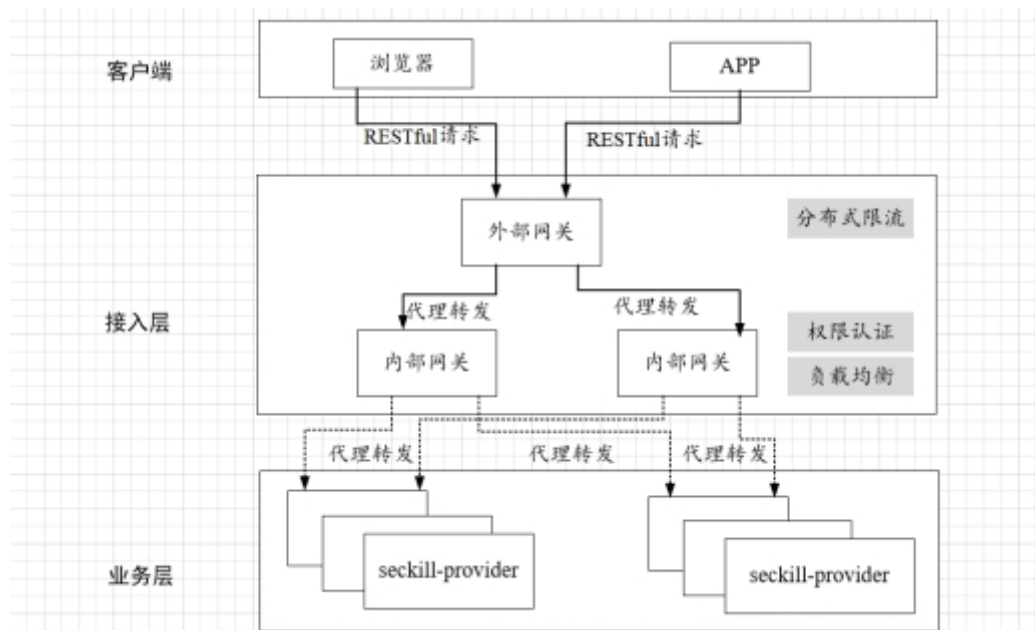


图11-3 外部网关与内部网关相结合完成认证、负载均衡、接口限流

3. 业务层负责保障数据一致性

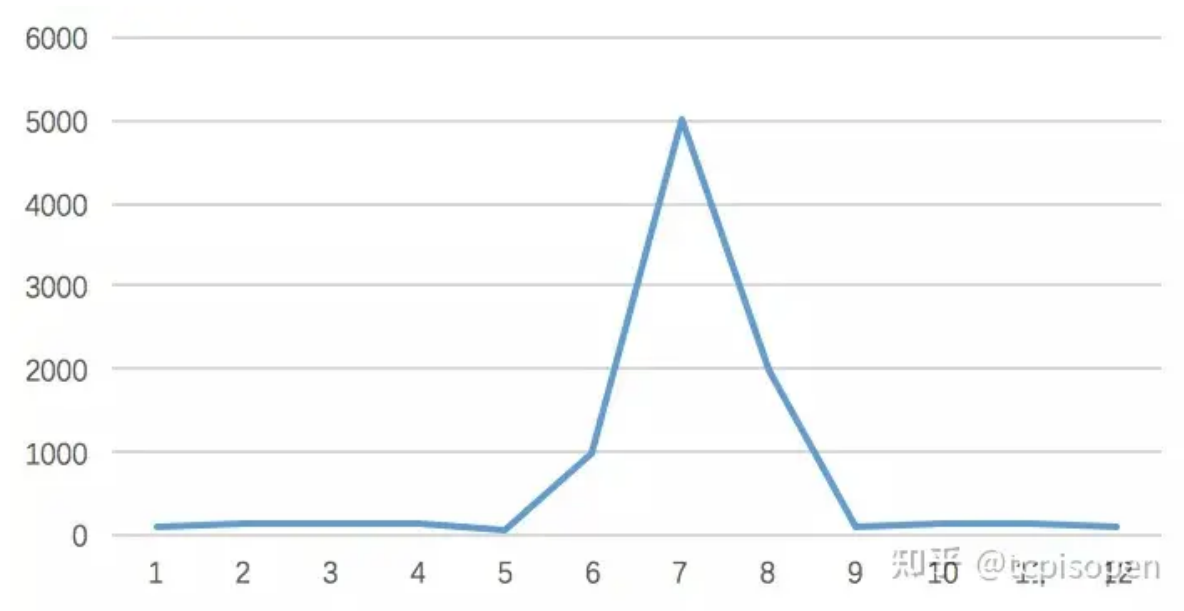
秒杀的业务逻辑主要是下订单和减库存，都是数据库操作。大家都知道，数据库层只能承担“能力范围内”的访问请求，是最脆弱的一层，也是需要事务保护的一层。在业务层，还需要防止超出库存的秒杀（超卖和少买），为了安全起见，可以使用分布式锁对秒杀的数据库操作进行保护。

4.3 秒杀架构的一些特殊方案

一般在秒杀时间点（比如：12点）前几分钟，用户并发量才真正突增，达到秒杀时间点时，并发量会达到顶峰。

但由于这类活动是大量用户抢少量商品的场景，必定会出现狼多肉少的情况，所以其实绝大部分用户秒杀会失败，只有极少部分用户能够成功。

正常情况下，大部分用户会收到商品已经抢完的提醒，收到该提醒后，他们大概率不会在那个活动页面停留了，如此一来，用户并发量又会急剧下降。所以这个峰值持续的时间其实是非常短的，这样就会出现瞬时高并发的情况，下面用一张图直观的感受一下流量的变化：



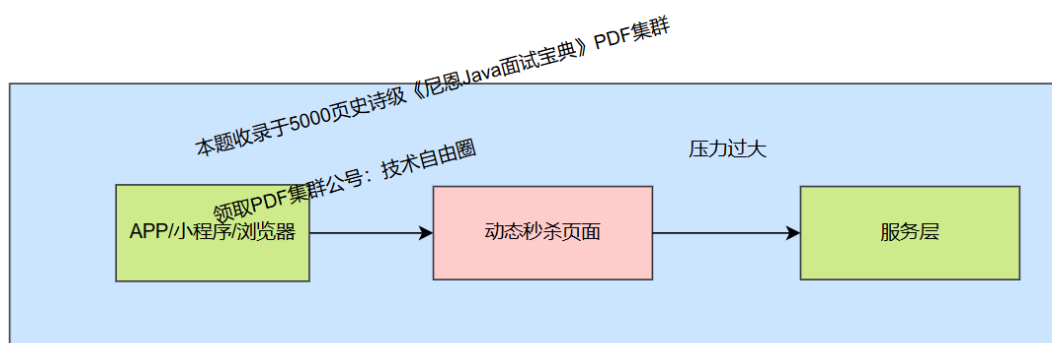
像这种瞬时高并发的场景，传统的系统很难应对，我们需要设计一套全新的系统。可以从以下几个方面入手：

1. 动静分离架构
2. CDN加速
3. 缓存
4. mq异步处理
5. 限流
6. 分布式锁
7. 系统扩容：系统扩容包括垂直扩容和水平扩容，增加设备和机器配置，绝大多数的场景有效。

5 秒杀的动静分离架构

活动页面是用户流量的第一入口，所以是并发量最大的地方。

如果这些流量都能直接访问服务端，恐怕服务端会因为承受不住这么大的压力，而直接挂掉。



活动页面绝大多数内容是固定的，比如：商品名称、商品描述、图片等。

为了减少不必要的服务端请求，通常情况下，会对活动页面做静态化处理。

用户浏览商品等常规操作，并不会请求到服务端。

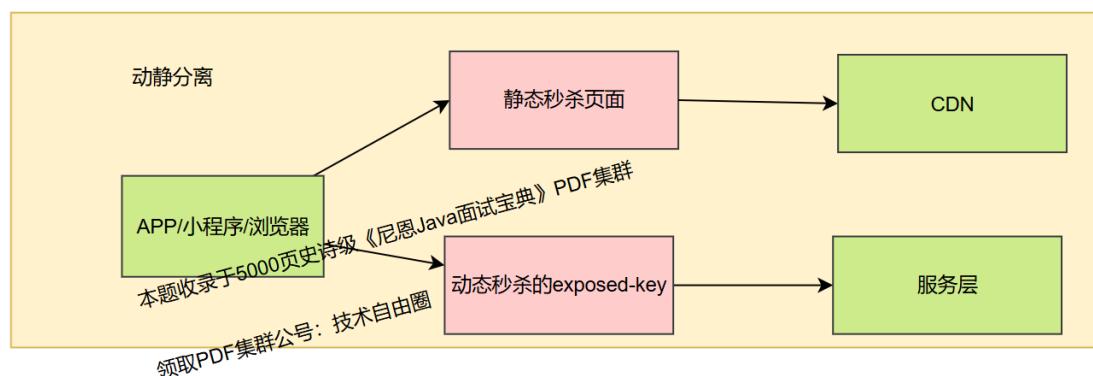
为了性能考虑 动静分离，分离出下面的两大部分：

- 静态 秒杀页面资源：一个html文件，包括 css、js和图片等，内容包括秒杀产品的介绍，详情，参数等等。静态秒杀静态资源文件提前缓存到CDN上，让用户能够就近访问秒杀页面。
- 动态秒杀的exposed-key，通过JS异步获取。秒杀暴露就是将符合条件的秒杀 暴露给用户，以便互联网用户能参与商品的秒杀。这个操作可以是商户手动完成，生产场景下的更合理的方式是系统定时任务去完成。秒杀暴露主要是生成一个 具备实效性的 exposed-key。

但只做页面静态化还不够，因为用户分布在全国各地，有些人在北京，有些人在成都，有些人在深圳，地域相差很远，网速各不相同。

如何才能让用户最快访问到活动页面呢？

这就需要使用CDN，它的全称是Content Delivery Network，即内容分发网络。

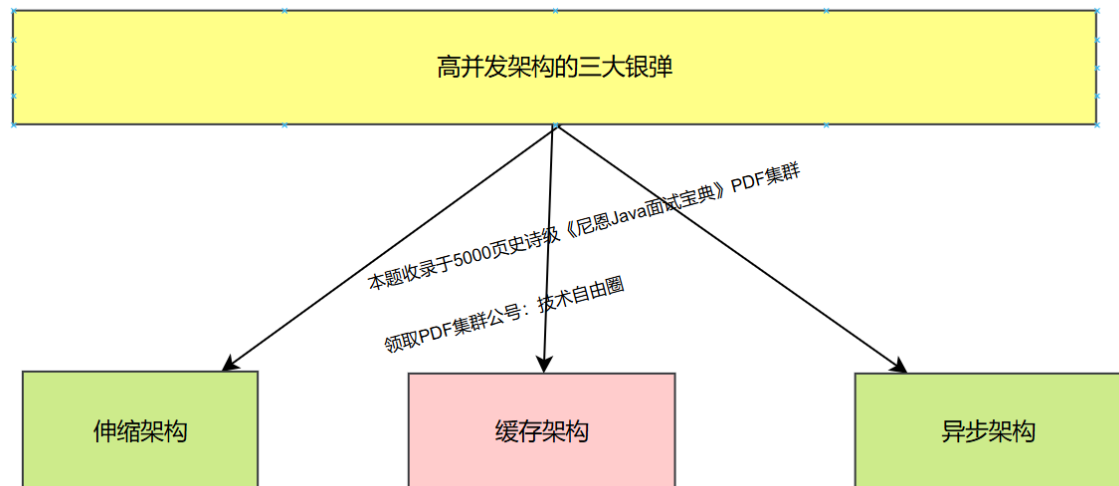


使用户就近获取所需内容，降低网络拥塞，提高用户访问响应速度和命中率。

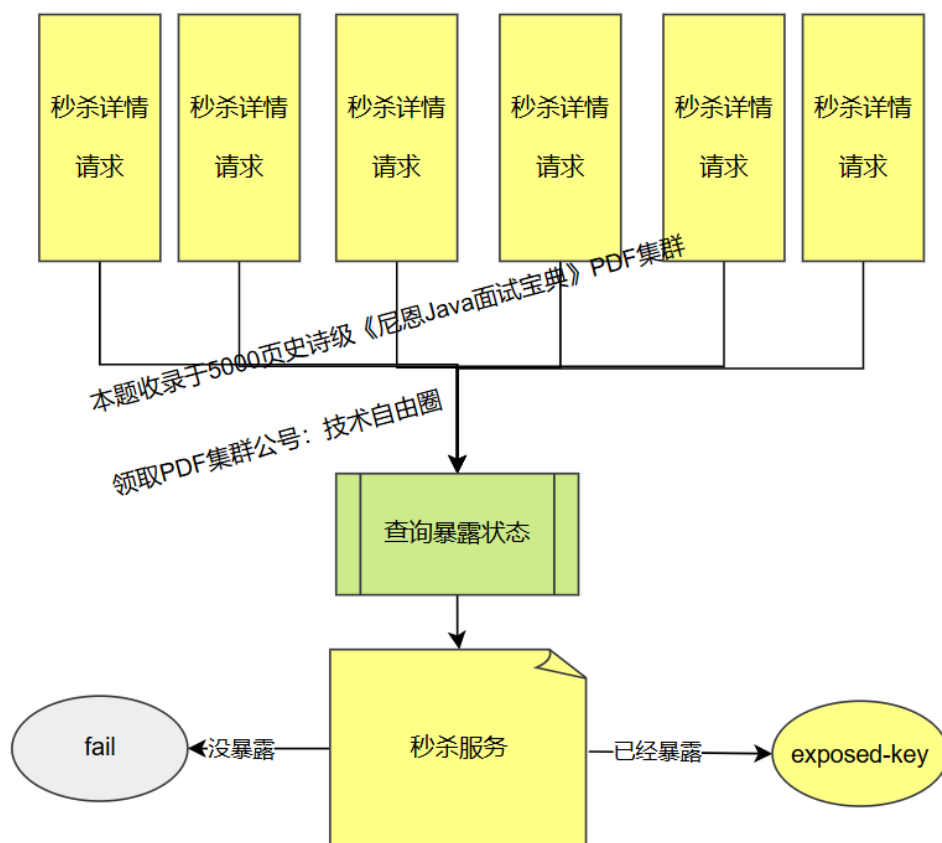
CDN服务器就是内容分发网络，把资源内容放在了全国各地的各服务器，通过中心平台的负载均衡、内容分发、调度等功能模块，使用户就近获取所需内容，一般都是到阿里云买CDN服务器。

尼恩提示：如果没有CDN，也可以放在Nginx中做动静分离。

6 秒杀的缓存架构



秒杀的开始前，系统一般会访问秒杀详情，这个是高并发读的场景。

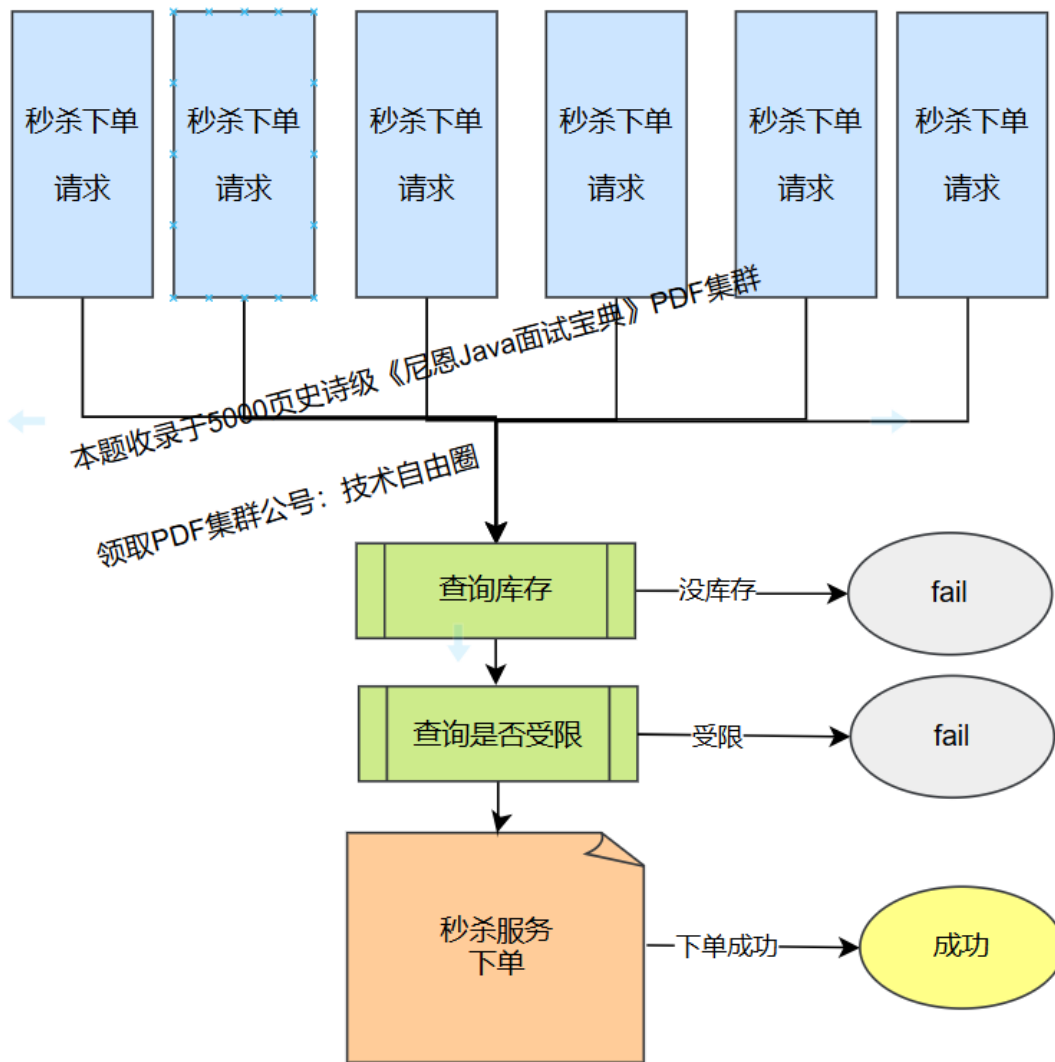


在下单的过程中，系统一般会先查一下库存是否足够，如果足够才允许下单，写数据库。如果不够，则直接返回该商品已经抢完。

在下单的过程中，大量用户抢少量商品，只有极少部分用户能够抢成功。

所以绝大部分用户在秒杀时，库存其实是不足的，系统会直接返回该商品已经抢完。

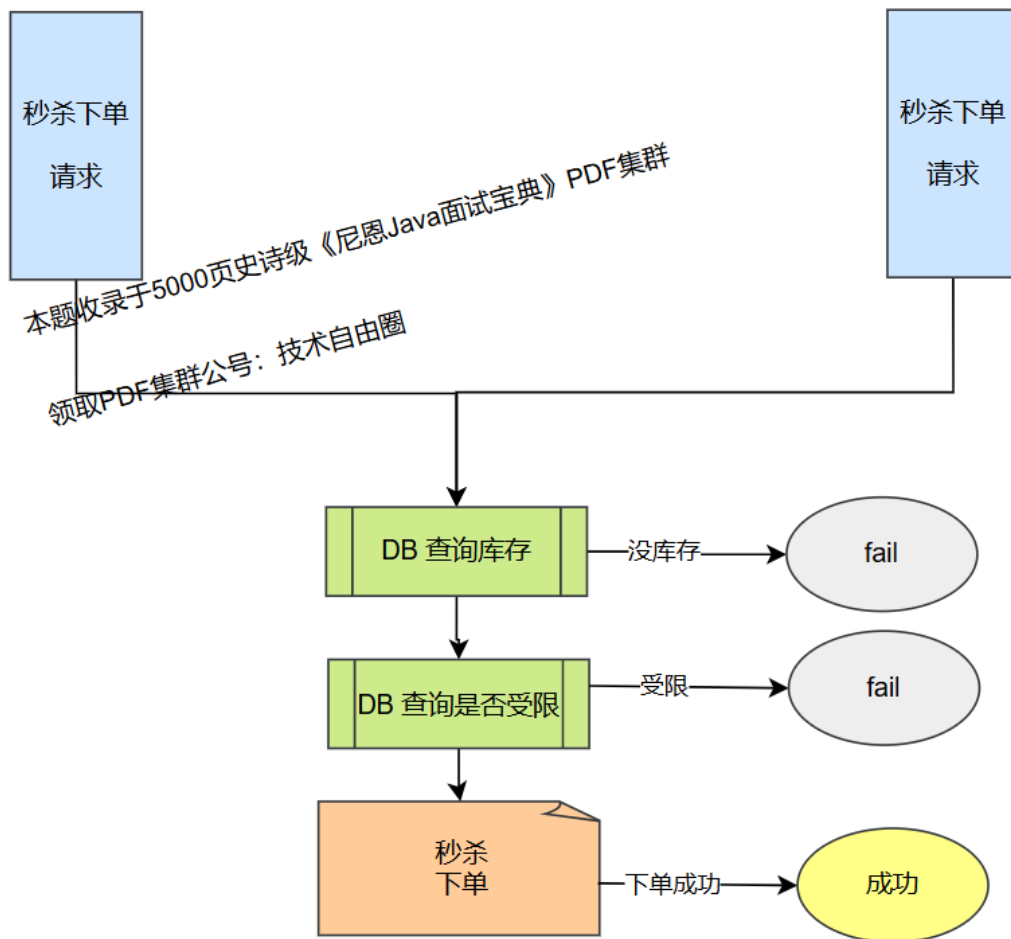
这是非常典型的：**读多写少**的场景。



6.1 读多写少大杀器：缓存架构

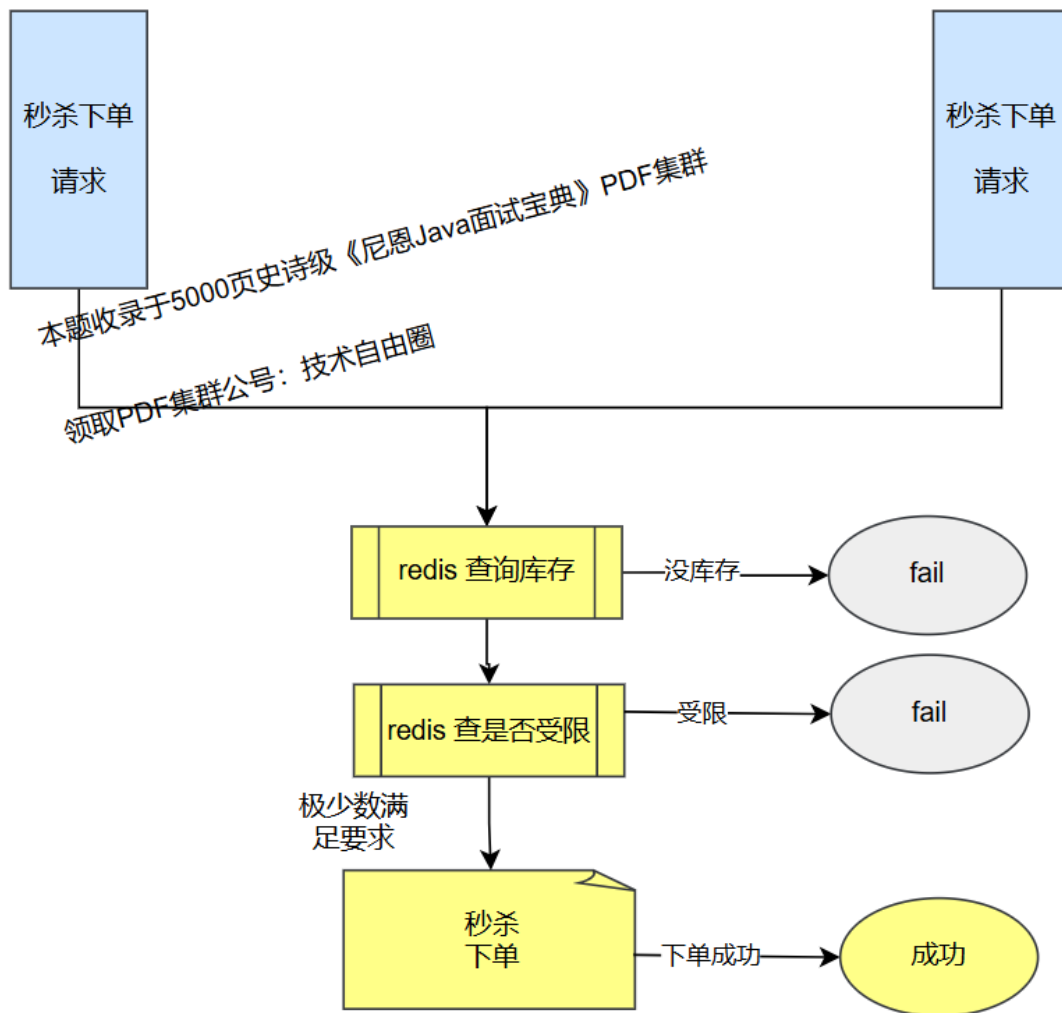
如果有数十万的请求过来，同时通过数据库查缓存是否足够，此时数据库可能会挂掉。

因为数据库的连接资源非常有限，比如：mysql，无法同时支持这么多的连接。



而应该改用缓存，比如：redis。

即便用了redis，也需要部署多个节点。

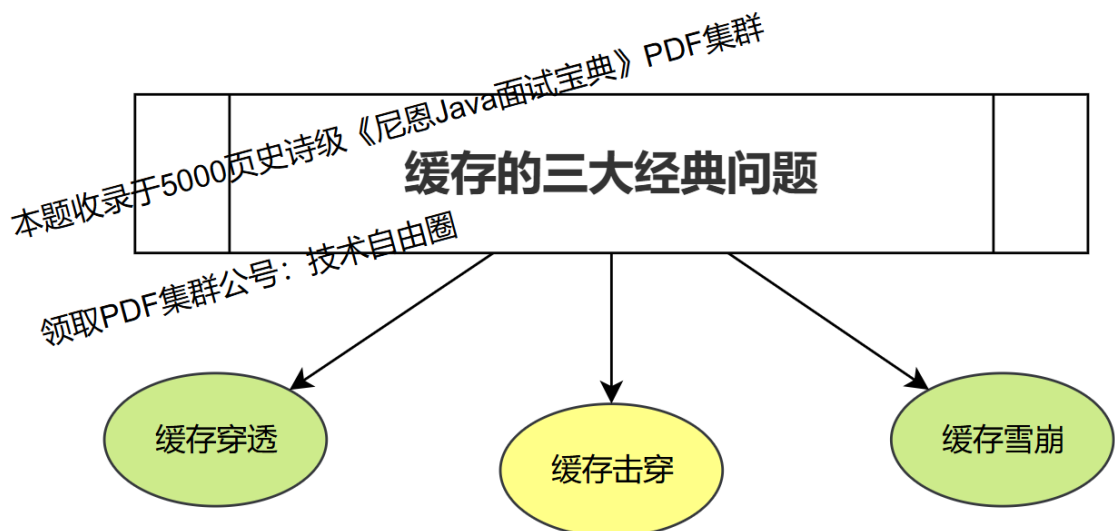


6.2 缓存的三大经典问题

通常情况下，我们需要在redis中保存商品信息，里面包含：商品id、商品名称、规格属性、库存等信息，同时数据库中也要有相关信息，毕竟缓存并不完全可靠。

用户在点击秒杀按钮，请求秒杀接口的过程中，需要传入的商品id参数，然后服务端需要校验该商品是否合法。

大致流程如下图所示：



根据商品id，先从缓存中查询商品，如果商品存在，则参与秒杀。

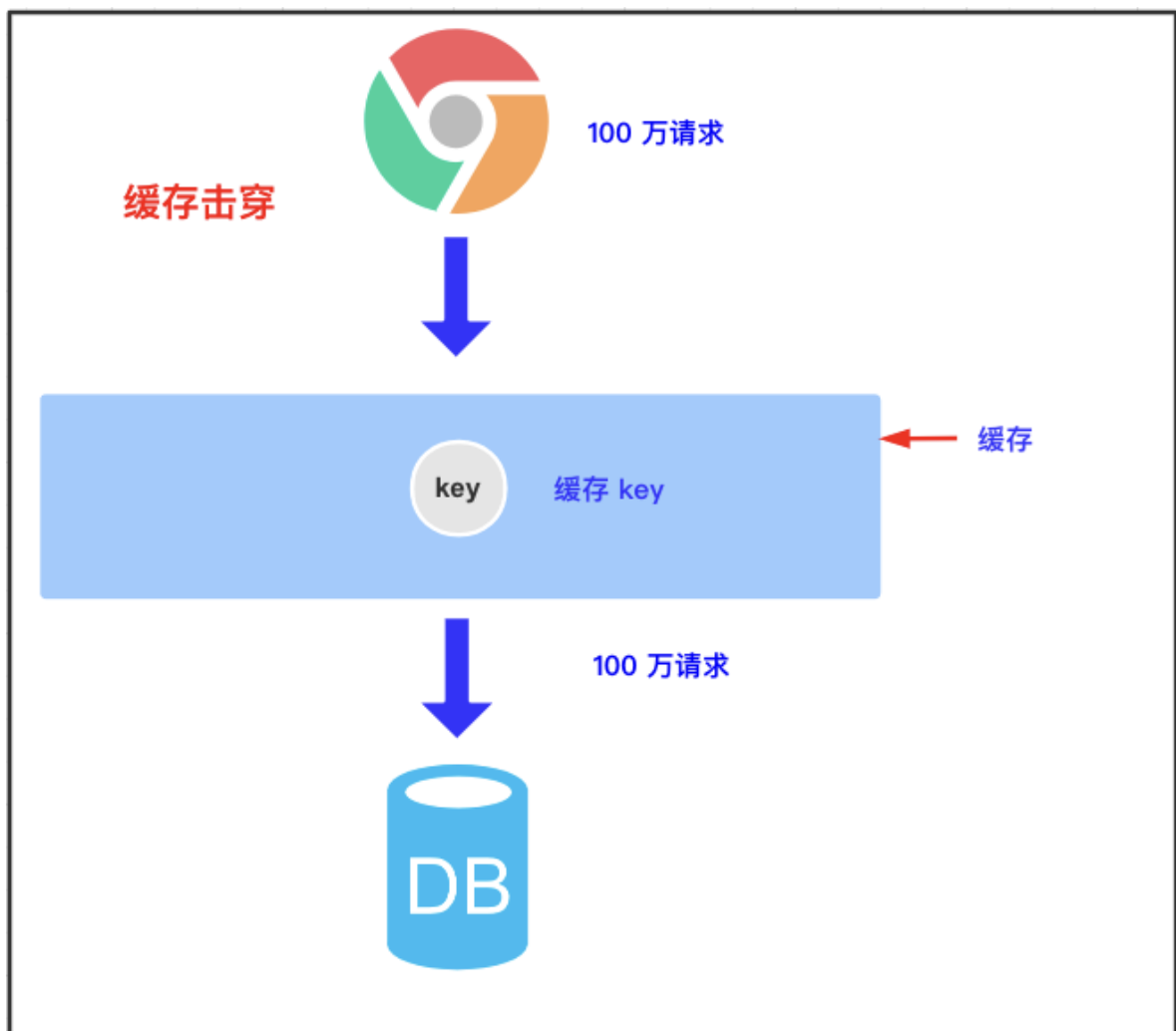
如果不存在，则需要从数据库中查询商品，如果存在，则将商品信息放入缓存，然后参与秒杀。

如果商品不存在，则直接提示失败。

这个过程表面上看起来是OK的，但是如果深入分析一下会发现一些问题。

6.3 缓存击穿

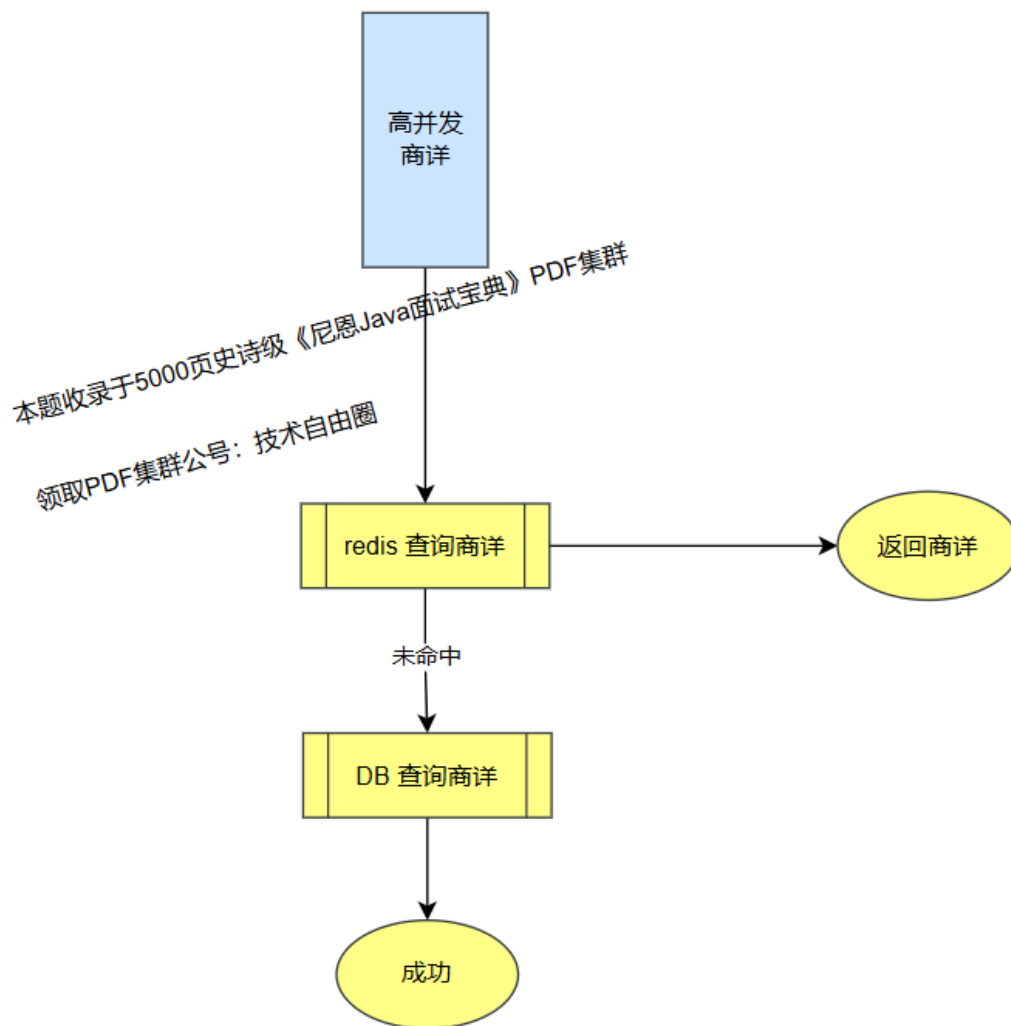
某个 key 设置了过期时间，但在正好失效的时候，有大量请求进来了，导致请求都到数据库查询了。



解决方案

大量并发时，只让一个请求可以获取到查询数据库的锁，其他请求需要等待，查到以后释放锁，其他请求获取到锁后，先查缓存，缓存中有数据，就不用查数据库。

比如商品A第一次秒杀时，缓存中是没有数据的，但数据库中有。



虽说上面有如果从数据库中查到数据，则放入缓存的逻辑。

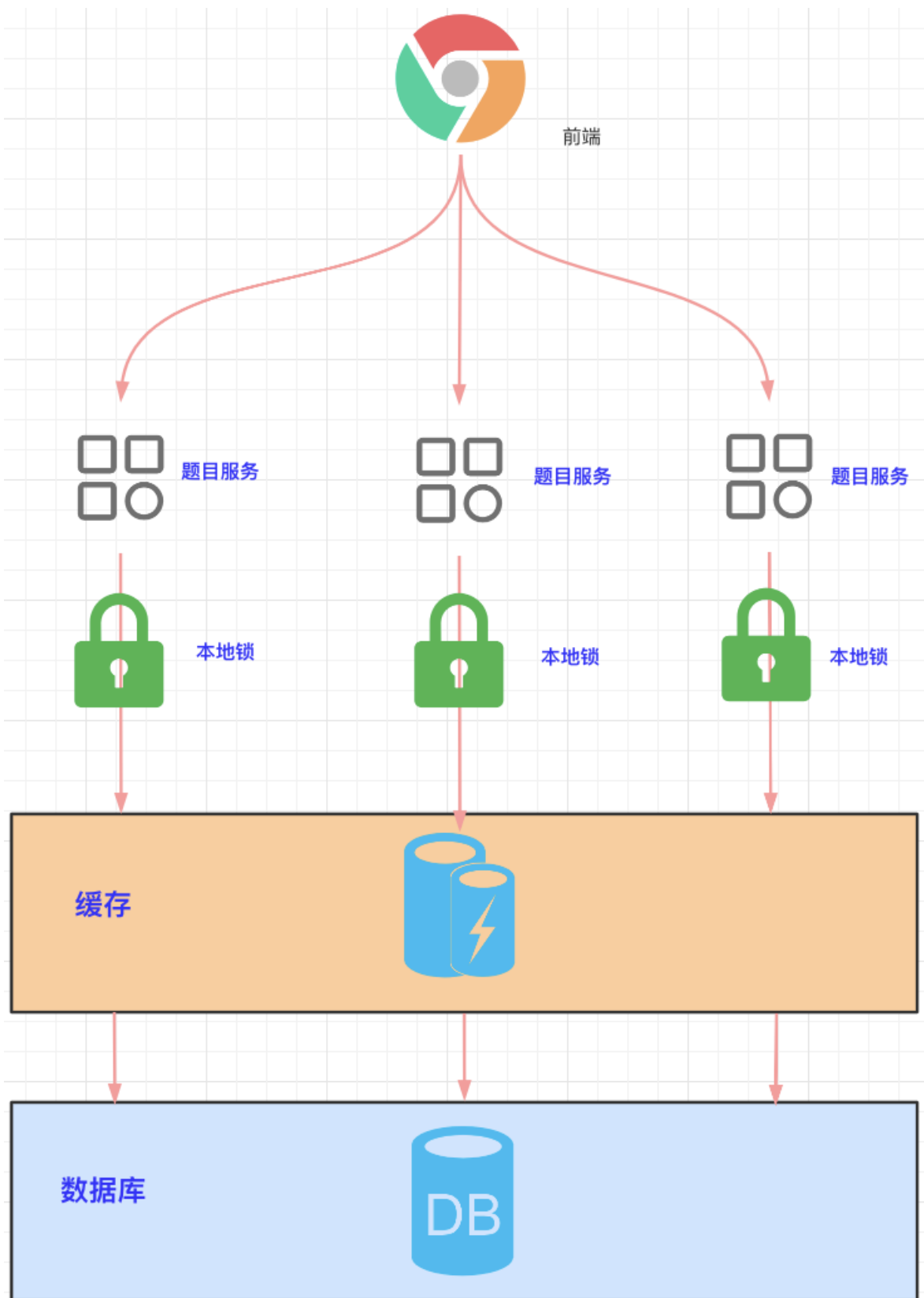
然而，在高并发下，同一时刻会有大量的请求，都在秒杀同一件商品，这些请求同时去查缓存中没有数据，然后又同时访问数据库。

结果悲剧了，数据库可能扛不住压力，直接挂掉。

如何解决这个问题呢？这就需要加锁。

本地锁的问题

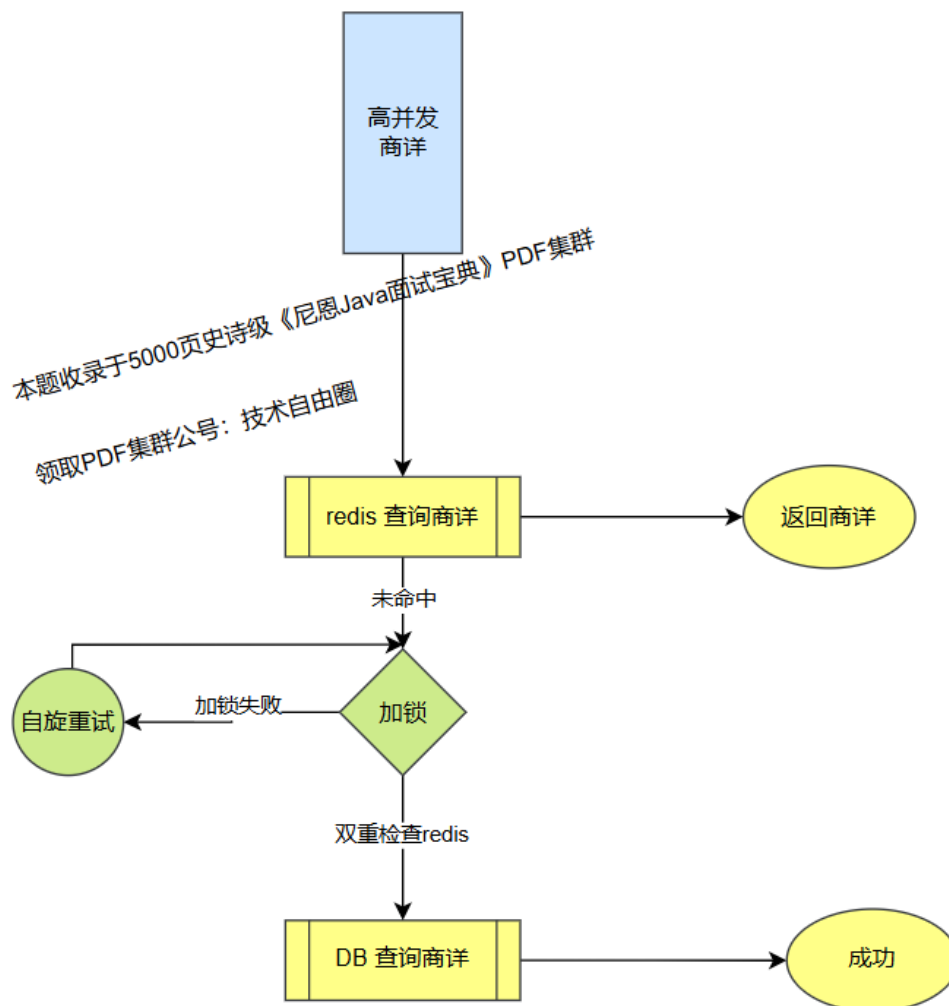
本地锁只能锁定当前服务的线程，如下图所示，部署了多个题目微服务，每个微服务用本地锁进行加锁。



本地锁在一般情况下没什么问题，但是当用来锁库存就有问题了：

- 1.当前总库存为 100，被缓存在 Redis 中。
- 2.库存微服务 A 用本地锁扣减库存 1 之后，总库存为 99。
- 3.库存微服务 B 用本地锁扣减库存 1 之后，总库存为 99。
- 4.那库存扣减了 2 次后，还是 99，就超卖了 1 个。

那如何解决本地加锁的问题呢？ 使用分布式锁。



当然，针对这种情况，最好在项目启动之前，先把缓存进行 **预热**。

即事先把所有的商品，同步到缓存中，这样商品基本都能直接从缓存中获取到，就不会出现缓存击穿的问题了。

是不是上面加锁这一步可以不需要了？

表面上看起来，确实可以不需要。但如果缓存中设置的过期时间不对，缓存提前过期了，或者缓存被不小心删除了，如果不加速同样可能出现缓存击穿。

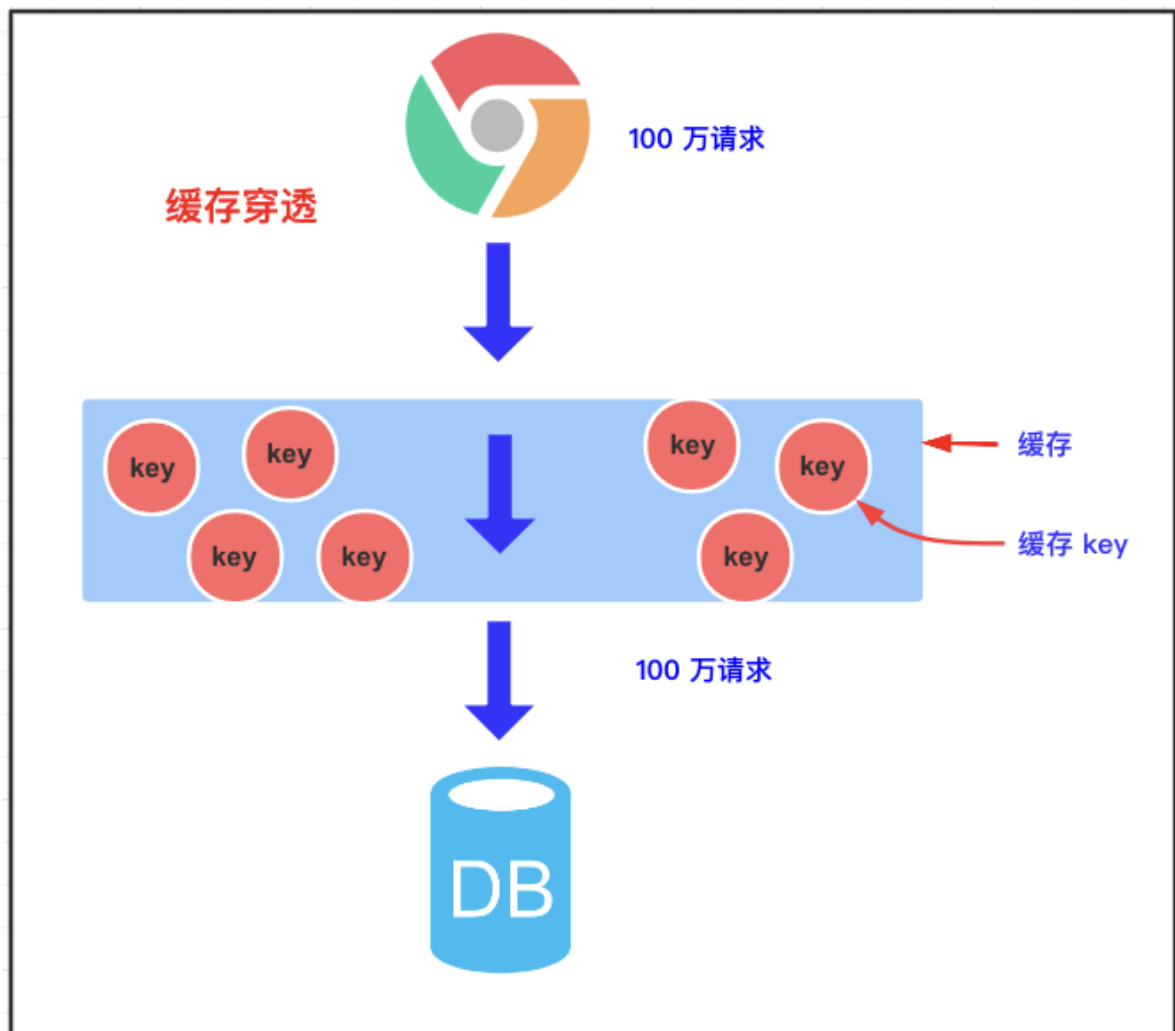
其实这里加锁，相当于买了一份保险。

6.4 缓存穿透

如果有大量的请求传入的商品id，在缓存中和数据库中都不存在，这些请求不就每次都会穿透缓存，而直接访问数据库了。

缓存穿透指一个一定不存在的数据，由于缓存未命中这条数据，就会去查询数据库，数据库也没有这条数据，所以返回结果是 `null`。

如果每次查询都走数据库，则缓存就失去了意义，就像穿透了缓存一样。



缓存穿透 带来的风险

利用不存在的数据进行攻击，数据库压力增大，最终导致系统崩溃。

缓存穿透为什么会产生缓存穿透

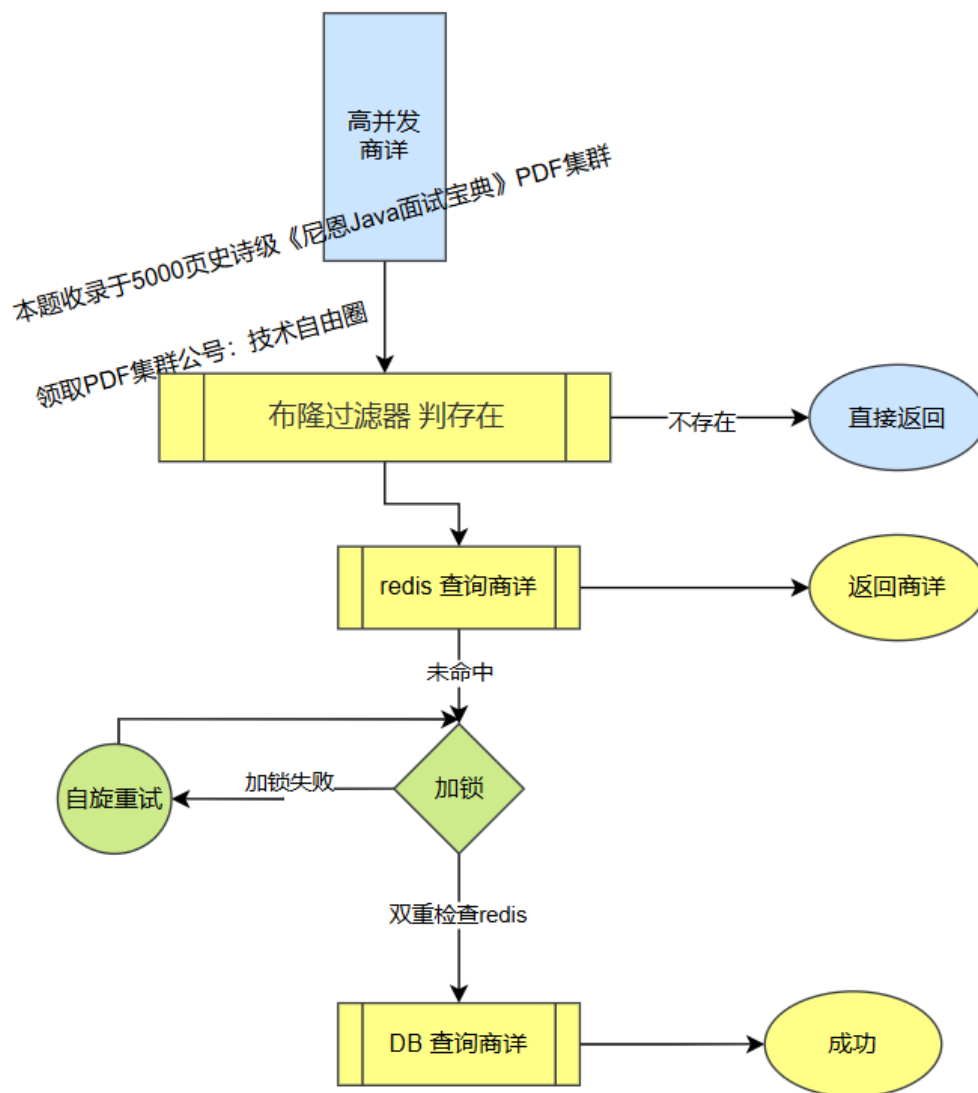
- 业务层误操作：缓存中的数据 and 数据库中的数据被误删除了，所以缓存和数据库中都没有数据；
- 恶意攻击：专门访问数据库中不存在的数据库。

缓存穿透解决方案

- 对结果 `null` 进行缓存，并加入短暂的过期时间。
- 使用布隆过滤器快速判断数据是否存在，避免从数据库中查询数据是否存在，减轻数据库压力。
- 前端进行请求检测。把恶意的请求（例如请求参数不合理、请求参数是非法值、请求字段不存在）直接过滤掉，不让它们访问后端缓存和数据库。

缓存穿透的一大利器：布隆过滤器

这时可以想到 布隆过滤器。



系统根据商品id，先从布隆过滤器中查询该id是否存在，如果存在则允许从缓存中查询数据，如果不存在，则直接返回失败。

虽说该方案可以解决缓存穿透问题，但是又会引出另外一个问题：布隆过滤器中的数据如何跟缓存中的数据保持一致？

这就要求，如果缓存中数据有更新，则要及时同步到布隆过滤器中。如果数据同步失败了，还需要增加重试机制，而且跨数据源，能保证数据的实时一致性吗？显然是不行的。所以布隆过滤器绝大部分使用在缓存数据更新很少的场景中。

6.5 缓存雪崩

某一时刻发生大规模的缓存失效的情况，例如缓存服务宕机、大量key在同一时间过期，这样的后果就是大量的请求进来直接打到DB上，db无响应，最后可能导致整个系统的崩溃，称为雪崩。

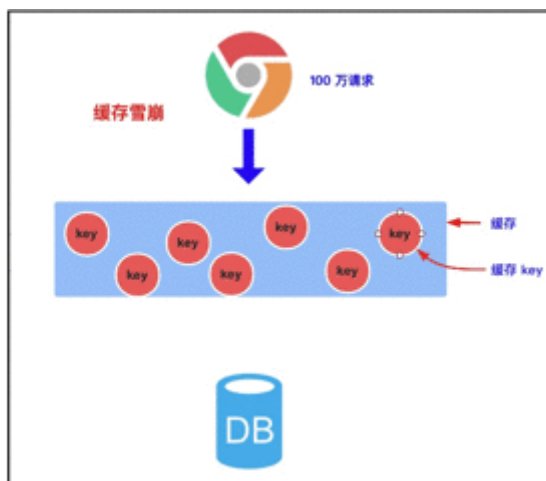
对于系统 A，假设每天高峰期每秒 5000 个请求，本来缓存高峰期可以扛住每秒 4000 个请求，

但是缓存机器意外发生了：

- 缓存全盘宕机，缓存挂了，
- 大量key在同一时间过期

此时 1 秒 5000 个请求全部落数据库，数据库必然扛不住，它会报一下警，然后db无响应，最后导致整个系统的崩溃。

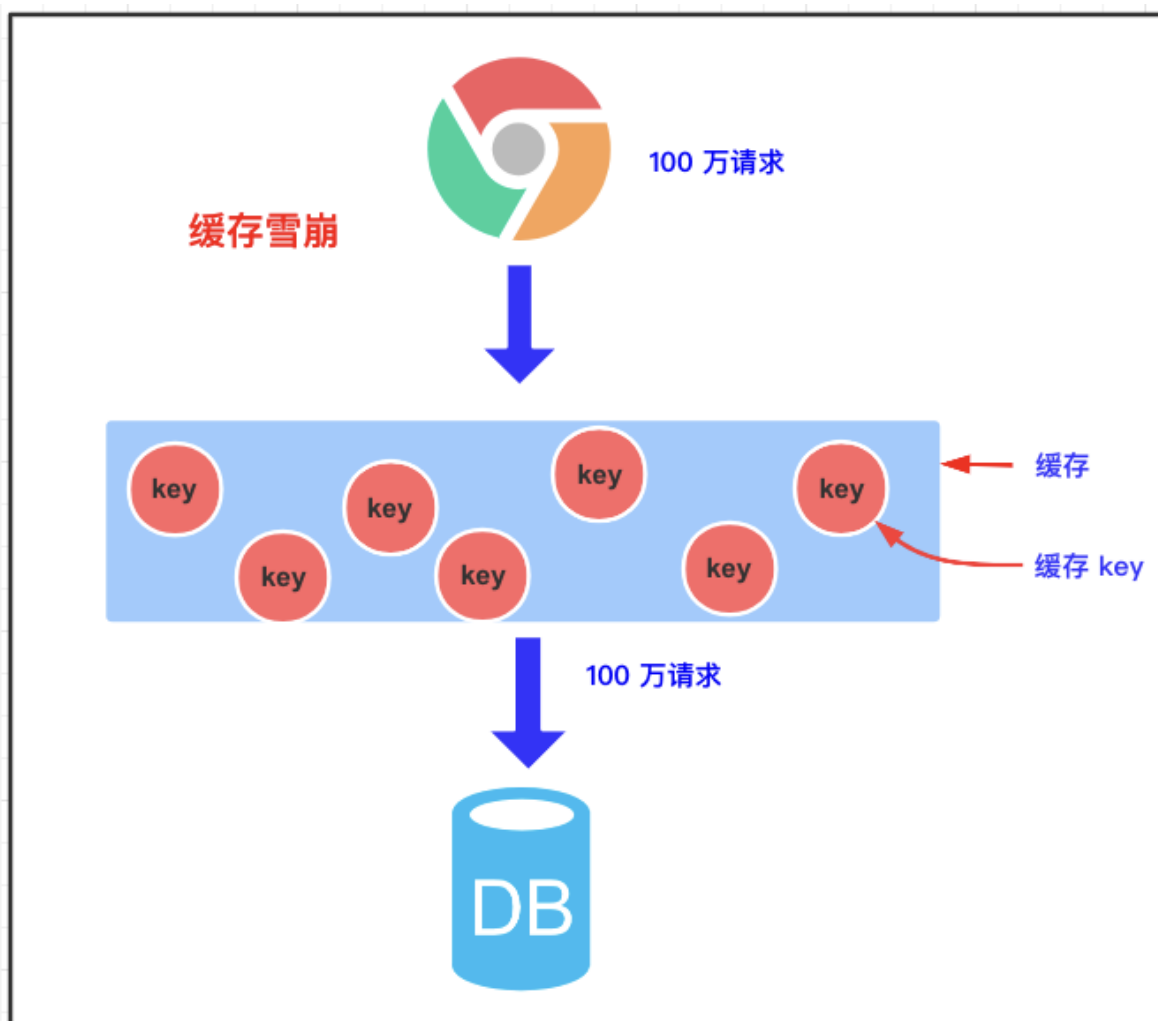
此时，如果没有采用什么特别的方案来处理这个故障，DBA 很着急，重启数据库，但是数据库立马又被新的流量给打死了。



缓存雪崩 带来的风险

尝试找到大量 key 同时过期的时间，在某时刻进行大量攻击，数据库压力增大，最终导致系统崩溃。

缓存雪崩是指我们缓存多条数据时，采用了相同的过期时间，比如 00:00:00 过期，如果这个时刻缓存同时失效，而有大量请求进来了，因未缓存数据，所以都去查询数据库了，数据库压力增大，最终就会导致雪崩。



缓存雪崩 解决方案

缓存雪崩是三大缓存问题里最严重的一种，我们来看看怎么预防和处理。

- **提高缓存可用性**

1. 集群部署：通过集群来提升缓存的可用性，可以利用Redis本身的Redis Cluster或者第三方集群方案如Codis等。
2. 多级缓存：设置多级缓存，设置一级缓存本地 guava 缓存，第一级缓存失效的基础上再访问二级缓存 redis，每一级缓存的失效时间都不同。

- **过期时间**

1. 均匀过期：为了避免大量的缓存在同一时间过期，可以把不同的 key 过期时间随机生成，避免过期时间太过集中。在原有的实效时间基础上增加一个碎挤汁，比如 1-5 分钟随机，降低缓存的过期时间的重复率，避免发生缓存集体失效。
2. 热点数据永不过期。

- **熔断降级**

1. 服务熔断：当缓存服务器宕机或超时响应时，为了防止整个系统出现雪崩，可以使用hystrix 类似的熔断，暂时停止业务服务访问db, 或者其他被依赖的服务，避免 MySQL 被打死。
2. 服务降级：当出现大量缓存失效，而且处在高并发高负荷的情况下，在业务系统内部暂时舍弃对一些非核心的接口和数据的请求，而直接返回一个提前准备好的 fallback（退路）错误处理信息。

7 库存扣减场景下的数据一致性架构

对于库存问题看似简单，实则里面还是有些东西。

真正的秒杀商品的场景，不是说扣完库存，就完事了，如果用户在一段时间内，还没完成支付，扣减的库存是要加回去的。

在高并发的秒杀活动中，库存扣减 可以分为以下三个阶段：

第一阶段：Redis预扣减库存

1. **预扣减启动**：当用户发起购买请求时，系统首先在Redis中检查并预扣减库存数量。
2. **并发处理**：通过 Redis可以高效地处理高并发请求，确保多个用户同时请求同一商品时不会导致DB 锁定或延迟。

第二阶段：数据库（DB）扣减库存

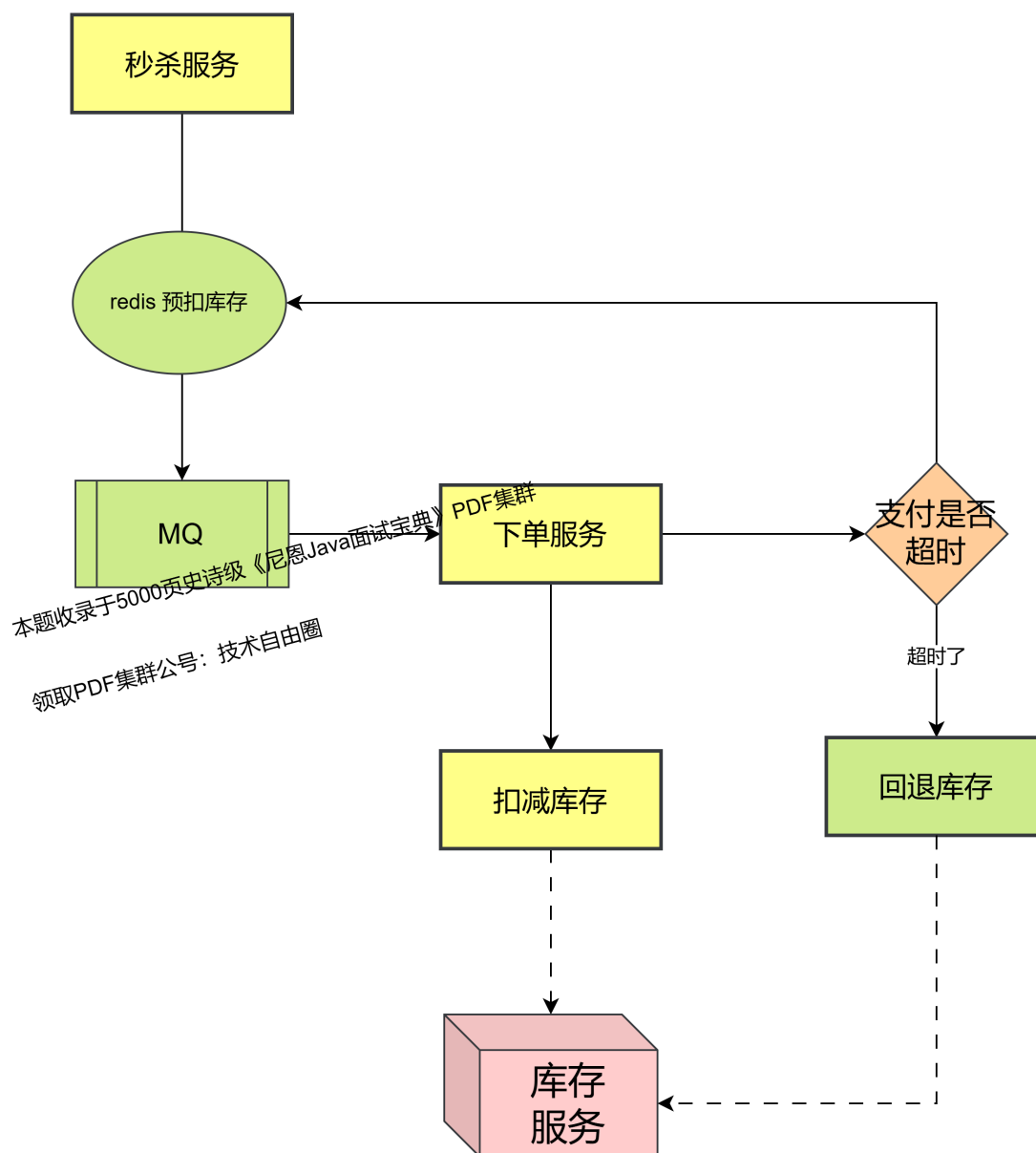
1. **最终扣减**：下单服务收到 MQ的 下单消息后，下单服务会在数据库中进行最终的库存扣减。
2. **数据一致性**：在数据库中更新库存确保数据的一致性和持久性。

第三阶段：支付超时，库存回退

1. **超时处理**：如果用户在规定时间内未完成支付，系统将识别支付超时。
2. **库存回退**：系统会将之前预扣减的库存数量从Redis中释放，并在数据库中恢复相应的库存数量，确保库存的准确性。

通过以上三个阶段，可以高效地管理秒杀活动中的库存，既保证了系统的响应速度，又维护了数据的一致性。

所以，库存扣减的主要流程如下：



扣减库存中除了上面说到的 预扣库存 和 回退库存 之外，还需要特别注意的是库存不足和库存超卖问题。

7.1 第一阶段 redis预扣减库存

第一阶段：Redis预扣减库存

1. **预扣减启动**：当用户发起购买请求时，系统首先在Redis中检查并预扣减库存数量。
2. **并发处理**：通过 Redis可以高效地处理高并发请求，确保多个用户同时请求同一商品时不会导致DB 锁定或延迟。

可以在java代码中，通过 redis的 `incr` 方法 操作库存， `incr` 方法是原子性的。

伪代码如下：

```

boolean exist = redisClient.query(productId,userId);
if(exist) {
    return -1;
}
int stock = redisClient.queryStock(productId);
if(stock <=0) {
    return 0;
}
redisClient.incrby(productId, -1);
redisClient.add(productId,userId);
return 1;

```

代码流程如下：

1. 先判断该用户有没有秒杀过该商品，如果已经秒杀过，则直接返回-1。
2. 查询库存，如果库存小于等于0，则直接返回0，表示库存不足。
3. 如果库存充足，则扣减库存，然后将本次秒杀记录保存起来。然后返回1，表示成功。

这段代码有问题。其实有一个问题，有什么问题呢？

由于查询库存和更新库存非原则操作，则会出现库存为负数的情况，即 **库存超卖**。

为了解决上面的问题，我们可以使用 lua脚本，是能够保证原子性的，它跟redis一起配合使用，能够完美解决上面的问题。

lua脚本有段非常经典的代码：

```

StringBuilder lua = new StringBuilder();
lua.append("if (redis.call('exists', KEYS[1]) == 1) then");
lua.append("    local stock = tonumber(redis.call('get', KEYS[1]));");
lua.append("    if (stock == -1) then");
lua.append("        return 1;");
lua.append("    end;");
lua.append("    if (stock > 0) then");
lua.append("        redis.call('incrby', KEYS[1], -1);");
lua.append("        return stock;");
lua.append("    end;");
lua.append("    return 0;");
lua.append("end;");
lua.append("return -1;");

```

该代码的主要流程如下：

1. 先判断商品id是否存在，如果不存在则直接返回。
2. 获取该商品id的库存，判断库存如果是-1，则直接返回，表示不限制库存。
3. 如果库存大于0，则扣减库存。
4. 如果库存等于0，是直接返回，表示库存不足。

7.2 第二阶段 数据库扣减库存

第二阶段：数据库（DB）扣减库存

1. **最终扣减**：下单服务收到 MQ 的 下单消息后，下单服务会在数据库中进行最终的库存扣减。
2. **数据一致性**：在数据库中更新库存确保数据的一致性和持久性。

下单服务 使用数据库扣减库存，是最简单的实现方案了，假设扣减库存的sql如下：

```
update product set stock=stock-1 where id=123;
```

这种写法对于扣减库存是没有问题的，但如何控制库存不足的情况下，不让用户操作呢？

这就需要在update之前，先查一下库存是否足够了。

伪代码如下：

```
int stock = mapper.getStockById(123);
if(stock > 0) {
    int count = mapper.updateStock(123);
    if(count > 0) {
        addOrder(123);
    }
}
```

大家有没有发现这段代码的问题？没错，查询操作和更新操作不是原子性的，会导致在并发的场景下，出现库存超卖的情况。

超卖或少卖问题，是秒杀场景的经典问题：比如10万次请求同时发起秒杀请求，正常需要进行10万次库存扣减，但是由于某种原因造成了多减库存或者少减库存，这就会出现超卖或少卖问题。

解决超卖或者少卖问题有效的办法之一就是利用分布式锁对同一个商品的并行数据库操作予以串行化。

秒杀场景的分布式锁应该具备的条件如下：

- 1) 一个方法在同一时间只能被一个机器的一个线程执行。
- 2) 高可用地获取锁与释放锁。
- 3) 高性能地获取锁与释放锁。
- 4) 具备可重入特性。
- 5) 具备锁失效机制，防止死锁。
- 6) 具备非阻塞锁特性，即没有获取到锁将直接返回获取锁失败。

常用的分布式锁有两种：ZooKeeper分布式锁和Redis分布式锁。

使用ZooKeeper分布式锁来保护秒杀的数据库操作的架构图大致如图11-5所示。

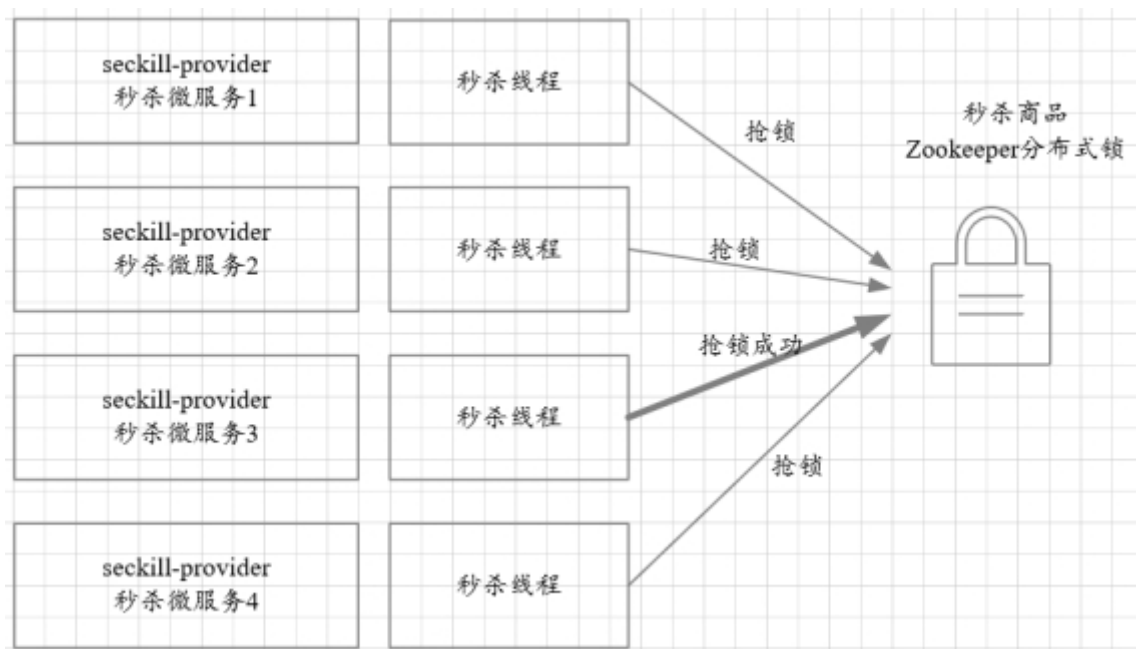


图11-5 使用ZooKeeper分布式锁来保护秒杀数据库操作

实际上，除了提供分布式锁外，ZooKeeper还具有提供高可靠的分布式计数器、高可靠的分布式ID生成器的基础能力。

ZooKeeper分布式计数器、分布式锁、分布式ID生成器等基础知识，也是大家必须系统学习和掌握的知识，但是不在这里介绍，如果对这一块不了解，请翻阅本书姊妹篇书籍《Java高并发核心编程 卷1（加强版）：NIO、Netty、Redis、ZooKeeper》。

ZooKeeper分布式锁虽然高可靠，但是性能不高，不能满足秒杀场景分布式锁的第三个条件（高性能地获取锁与释放锁），所以在秒杀的场景建议使用Redis分布式锁来保护秒杀的数据库操作。

7.3 redis分布式锁 实现库存扣减一致

ZooKeeper分布式锁 是CP类型，性能低。

如果要高并发，这就需要用redis分布式锁了。

基础的 setNx命令加锁

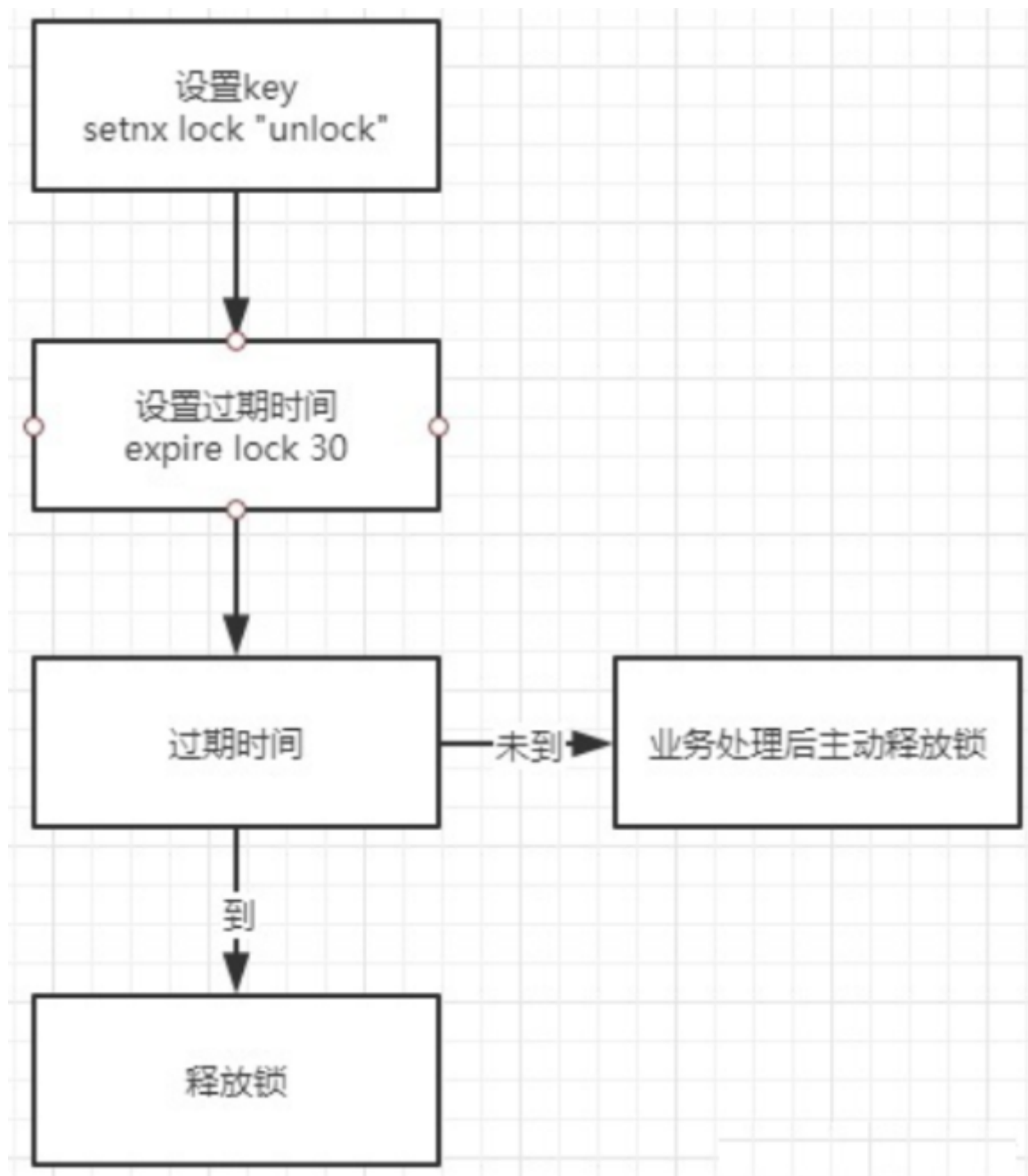
使用redis的分布式锁，首先想到的是 `setNx` + `expire` 命令。

```
if (jedis.setnx(lockKey, val) == 1) {  
    jedis.expire(lockKey, timeout);  
}
```

通过Redis的setnx、expire命令可以实现简单的锁机制：

- key不存在时创建，并设置value和过期时间，返回值为1；成功获取到锁；
- 如key存在时直接返回0，抢锁失败；
- 持有锁的线程释放锁时，手动删除key；或者过期时间到，key自动删除，锁释放。

线程调用setnx方法成功返回1认为加锁成功，其他线程要等到当前线程业务操作完成释放锁后，才能再次调用setnx加锁成功。



以上简单redis分布式锁的问题：

用setnx、expire 命令其实可以加锁，但setnx和后面的expire设置超时时间是分开的，并非原子操作。假如加锁成功了，但是设置超时时间失败了，该lockKey就变成永不失效的了。

在高并发场景中，该问题会导致非常严重的后果。

换句话说：

如果出现了这么一个问题：如果 `setnx` 是成功的，但是 `expire` 设置失败，一旦出现了释放锁失败，或者没有手工释放，那么这个锁永远被占用，其他线程永远也抢不到锁。

所以,需要保障setnx和expire两个操作的原子性，要么全部执行，要么全部不执行，二者不能分开。

解决的办法有两种：

- 使用set的命令时，同时设置过期时间，不再单独使用 expire命令

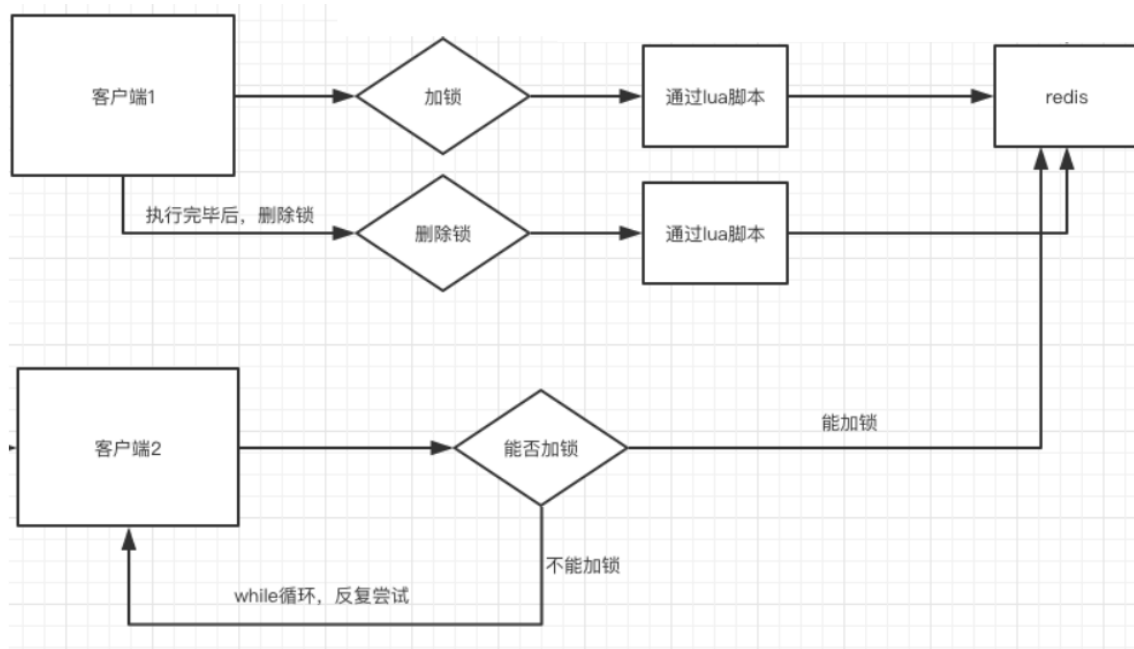
- 使用lua脚本，将加锁的命令放在lua脚本中原子性的执行

那么，有没有保证原子性的加锁命令呢？

基于纯Lua脚本的分布式锁的执行流程

加锁和删除锁的操作，使用纯lua进行封装，保障其执行时候的原子性。

基于纯Lua脚本实现分布式锁的执行流程，大致如下：



加锁的Lua脚本： lock.lua

```
--- -1 failed
--- 1 success
---
local key = KEYS[1]
local requestId = KEYS[2]
local ttl = tonumber(KEYS[3])
local result = redis.call('setnx', key, requestId)
if result == 1 then
    --PEXPIRE:以毫秒的形式指定过期时间
    redis.call('expire', key, ttl)
else
    result = -1;
    -- 如果value相同，则认为是同一个线程的请求，则认为重入锁
    local value = redis.call('get', key)
    if (value == requestId) then
        result = 1;
        redis.call('expire', key, ttl)
    end
end
-- 如果获取锁成功，则返回 1
return result
```

解锁的Lua脚本： unlock.lua：

```

--- -1 failed
--- 1 success

-- unlock key
local key = KEYS[1]
local requestId = KEYS[2]
local value = redis.call('get', key)
if value == requestId then
    redis.call('del', key);
    return 1;
end
return -1

```

7.4 redisson 分布式锁 实现库存扣减一致

使用redis分布式锁，还有续期问题等。

在使用 Redisson 实现分布式锁时，会自动解决续期问题。

Redisson 分布式锁的使用

1. **添加依赖**：在 Maven 或 Gradle 项目中添加 Redisson 的依赖。

```

<!-- Maven 依赖 -->
<dependency>
    <groupId>org.redisson</groupId>
    <artifactId>redisson</artifactId>
    <version>3.16.3</version>
</dependency>

```

2. **配置 Redisson**：配置 Redisson 客户端以连接到 Redis 服务器。

```

config = new Config();
config.useSingleServer().setAddress("redis://127.0.0.1:6379");
RedissonClient redissonClient = Redisson.create(config);

```

3. **获取分布式锁**：使用 RedissonClient 获取锁。

```

java复制代码RLock lock = redissonClient.getLock("anyLock");
lock.lock(); // 获取锁
try {
    // 执行需要加锁的业务逻辑
} finally {
    lock.unlock(); // 释放锁
}

```

锁的续期问题

在实际应用中，锁的持有时间可能会超过设定的租约时间（lease time），这时需要考虑锁的续期问题。

Redisson 提供了自动续期机制，但了解其工作原理和手动续期的方法也是非常重要的。

Redisson 内部实现了 Watch Dog 机制，当一个客户端获取到锁后，默认会启动一个看门狗线程，该线程会每隔 10 秒自动续期，续期时间为锁的默认超时时间的一半（默认30秒）。

Watch Dog 是 Redisson 的一个看门狗线程，当一个客户端获取到锁后，这个看门狗线程会定期检查并自动续期锁的租约时间。

Watch Dog 工作原理：

1. **获取锁时启动 Watch Dog**：当客户端获取锁时，Redisson 会启动一个看门狗线程。
2. **定期续期**：Watch Dog 会每隔一段时间（默认10秒）检查锁的状态，并自动续期。续期时间为锁的默认租约时间的一半（默认30秒）。
3. **续期机制**：在租约时间快到期时，Watch Dog 会发送续期请求，将锁的租约时间延长。这样即使业务逻辑执行时间较长，也不会因为锁超时而被其他线程获取。

Watch Dog 示例代码：

```
java复制代码// 配置 Redisson
Config config = new Config();
config.useSingleServer().setAddress("redis://127.0.0.1:6379");
RedissonClient redissonClient = Redisson.create(config);

// 获取分布式锁
RLock lock = redissonClient.getLock("anyLock");

// 获取锁（默认30秒租约时间）
lock.lock();
try {
    // 执行业务逻辑
    // watch Dog 会每隔10秒自动续期，续期时间为30秒
} finally {
    lock.unlock(); // 释放锁
}
```

Redisson获取锁有两种方式，lock获取和tryLock获取，其中：

（1）lock获取锁时，如果第一次尝试获取失败，会一直等待，直到获取锁。Redisson的lock获取锁时，会为锁配置过期时间，当锁超过过期时间后，会自动释放，避免死锁，可以重新获得锁；

（2）lock()原始方法获取（不自定义过期时间）会激活看门狗。即通过lock()方法成功获取锁后，逻辑执行的时间超过看门狗超时时间：10秒，会自动为锁续期（增加）10秒的过期时间，使锁的过期时间保持在30秒（默认的锁过期时间）；

（3）lock(long, TimeUnit)带参方法可以指定锁过期时间，但是，不会触发看门狗自动续期；锁过期后，自动释放锁；

```
// 获取锁时指定持有时间
RLock lock = redissonClient.getLock("anyLock");
lock.lock(10, TimeUnit.SECONDS); // 锁会在10秒后自动释放，不会触发看门狗
```

(4) tryLock方法返回标识位，获取成功，返回true，获取失败，返回false；tryLock尝试获取锁，无法获取锁时会等待，同时，tryLock有最大的等待时间，如果获取锁的等待时间超过最大等待时间，会放弃获取锁，并返回false；tryLock不会触发看门狗，无法自动为锁续期；

(5) tryLock可以同时指定等待锁的最大时间，以及锁过期时间，如果等待最大时间大于锁过期时间，则锁被提前释放，重新生成锁；

(6) Redisson看门狗默认巡查周期为10秒，锁续期时间（过期时间）30秒；

7.5 redisson 分段锁 实现库存扣减一致

本文已经超高3w字了，redisson 分段锁 的文章，请参见下面的链接：

[绝命一问：秒杀Redis分段锁，如何设计？](#)

7.6 第三阶段：支付超时，库存回退

通常情况下，如果用户秒杀成功了，下单之后，在15分钟之内还未完成支付的话，该订单会被自动取消，回退库存。

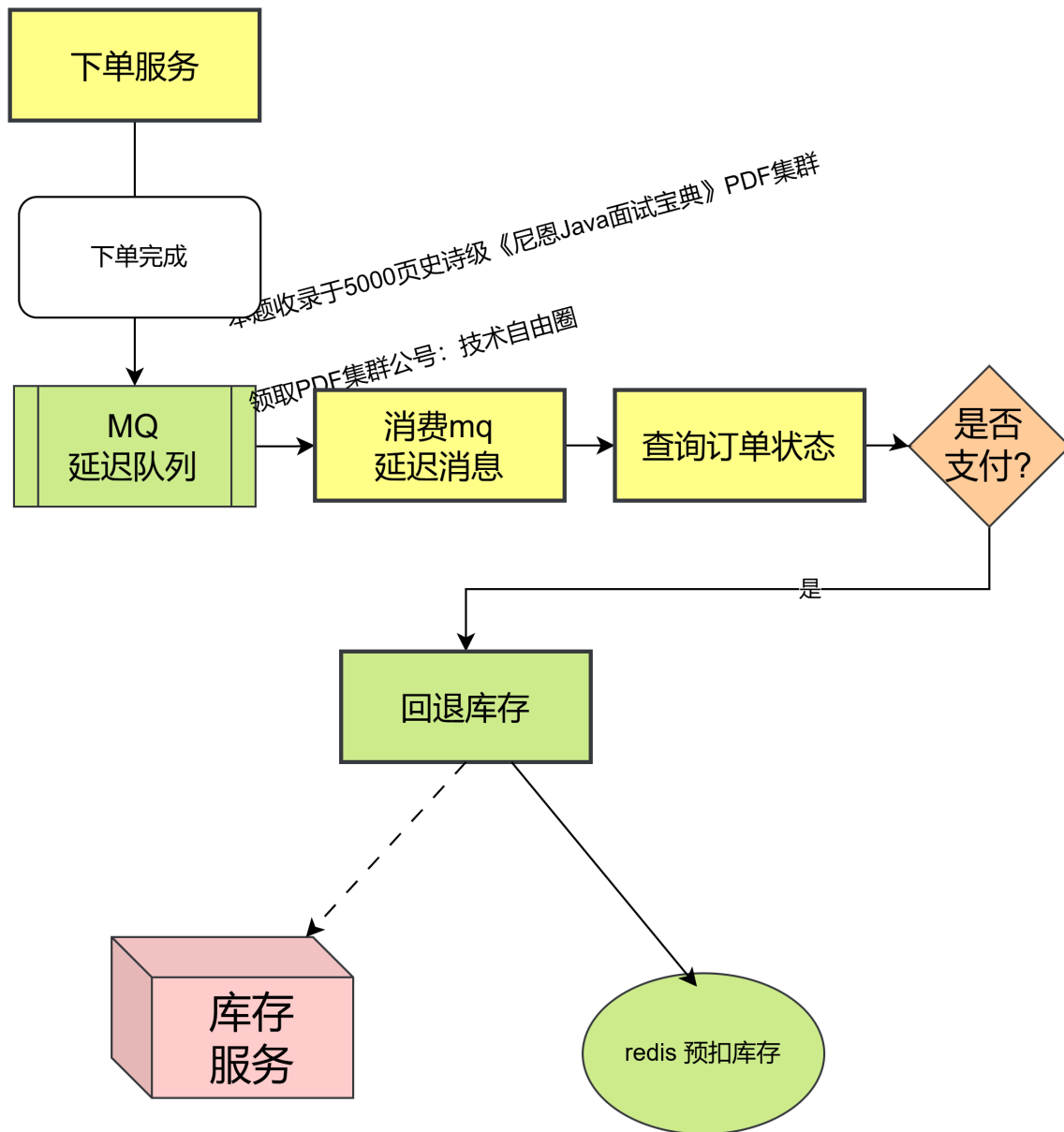
那么，在15分钟内未完成支付，订单被自动取消的功能，要如何实现呢？

我们首先想到的可能是job，因为它比较简单。

但job有个问题，需要每隔一段时间处理一次，实时性不太好。还有更好的方案？

答：使用延迟队列。

我们都知道rocketmq，自带了延迟队列的功能。



下单时消息生产者会先生成订单，此时状态为待支付，然后会向延迟队列中发一条消息。

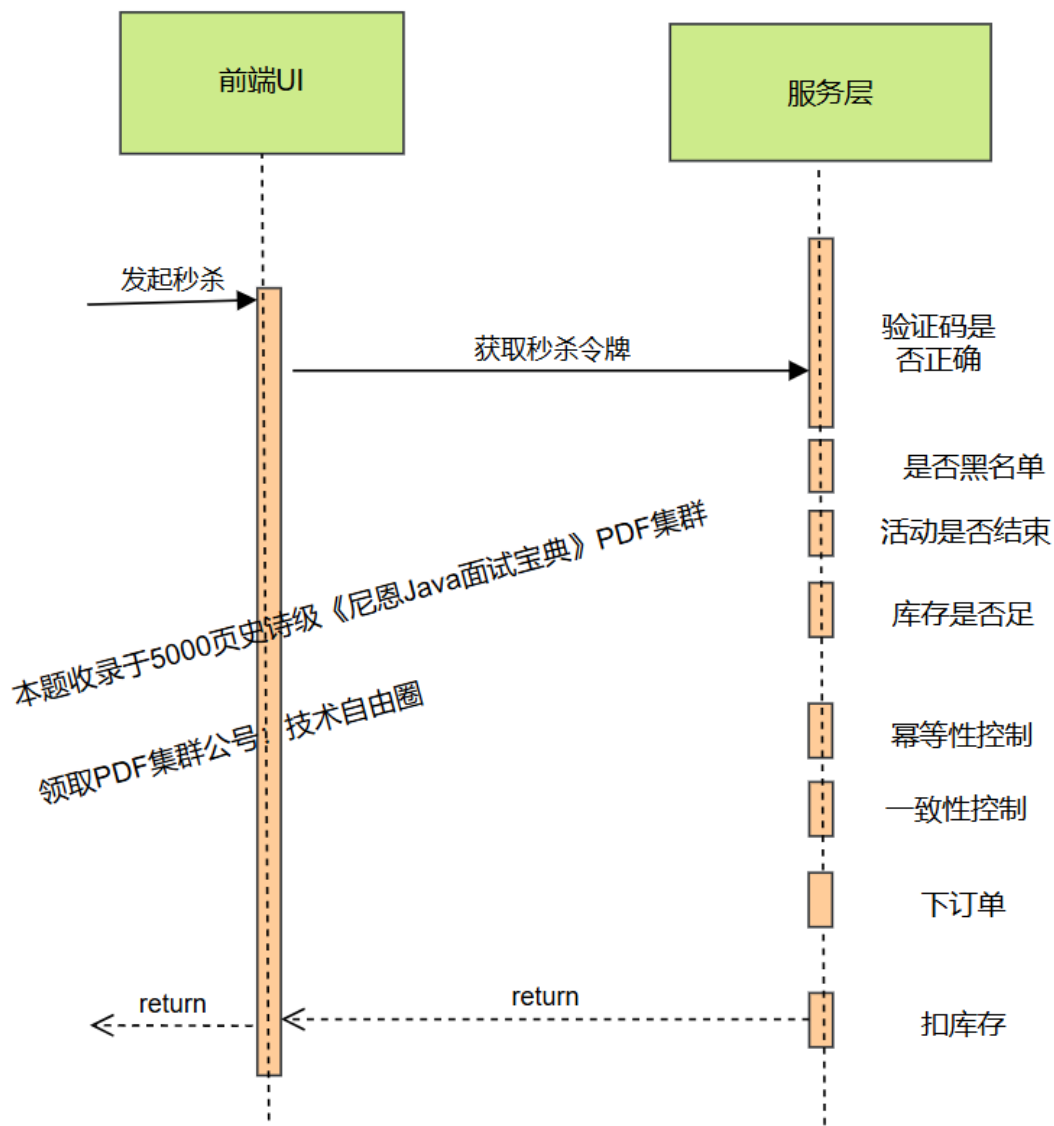
达到了延迟时间，消息消费者读取消息之后，会查询该订单的状态是否为待支付。

如果是待支付状态，则会更新订单状态为取消状态。如果不是待支付状态，说明该订单已经支付过了，则直接返回。

8 秒杀的mq异步架构

[得物面试：每秒上万次下单请求，秒杀如何处理？](#)

我们都知道在真实的秒杀场景中，有三个核心流程：

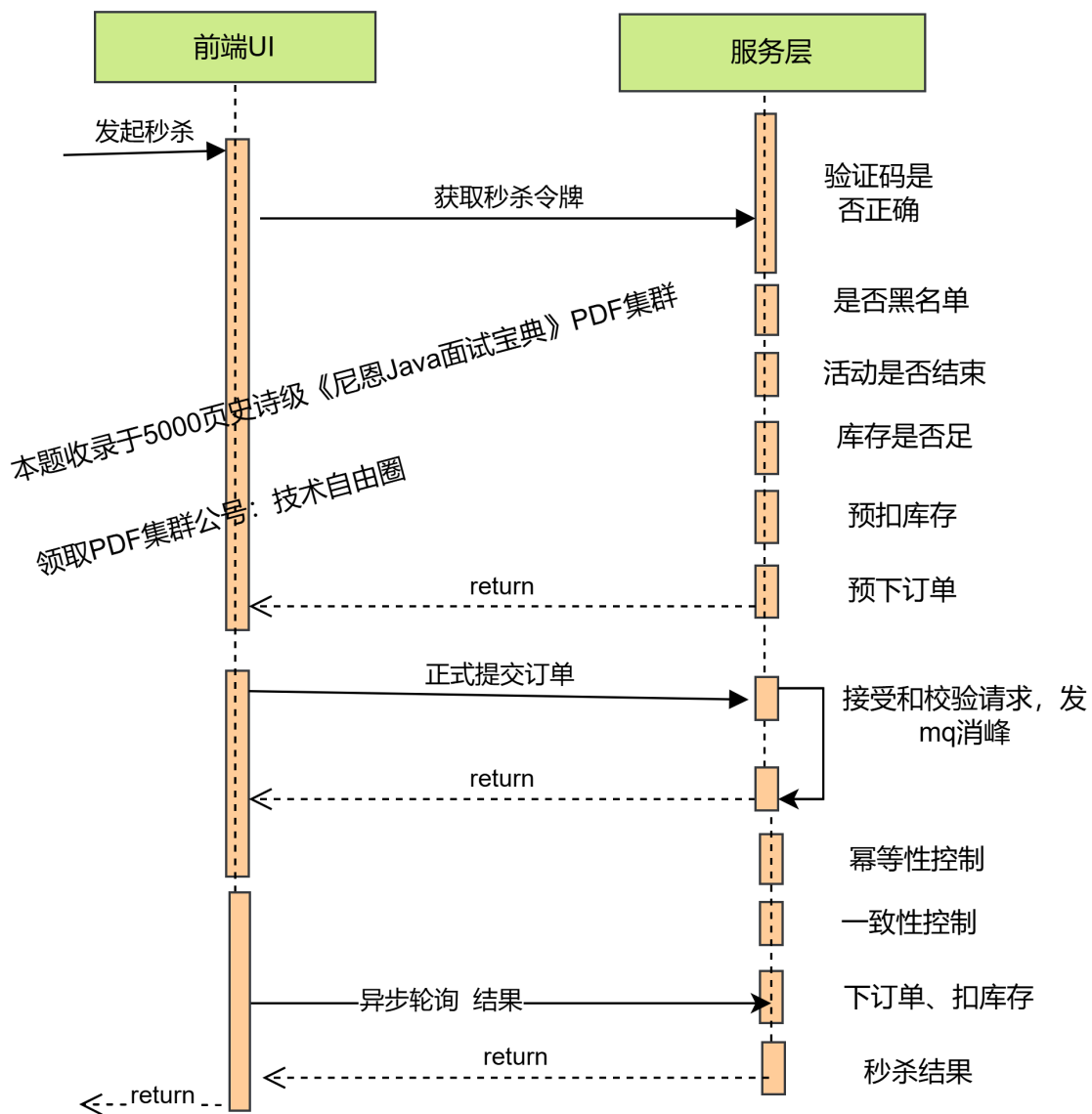


而这三个核心流程中，真正并发量大的是秒杀功能，下单和支付功能实际并发量很小。

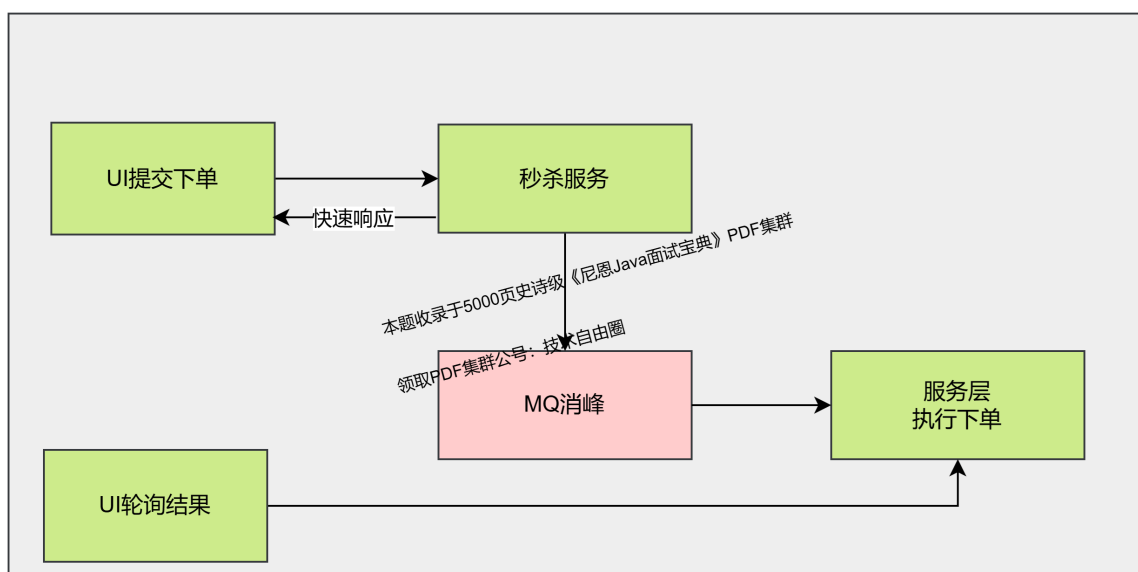
所以，我们在设计秒杀系统时，有必要把下单和支付功能从秒杀的主流程中拆分出来，特别是下单功能要做成mq异步处理的。

而支付功能，比如支付宝支付，是业务场景本身保证的异步。

于是，秒杀后下单的流程变成 两阶段下单，或者 三阶段下单：



使用mq，服务之间的关系，如下图：



如果使用mq，需要关注以下几个问题：

- 消息0丢失架构

8.1 MQ的消息0丢失架构

秒杀成功了，往mq发送下单消息的时候，有可能会失败。

原因有很多，比如：网络问题、broker挂了、mq服务端磁盘问题等。这些情况，都可能会造成消息丢失。

那么，如何防止消息丢失呢？

[滴滴面试：Rocketmq消息0丢失，如何实现？](#)

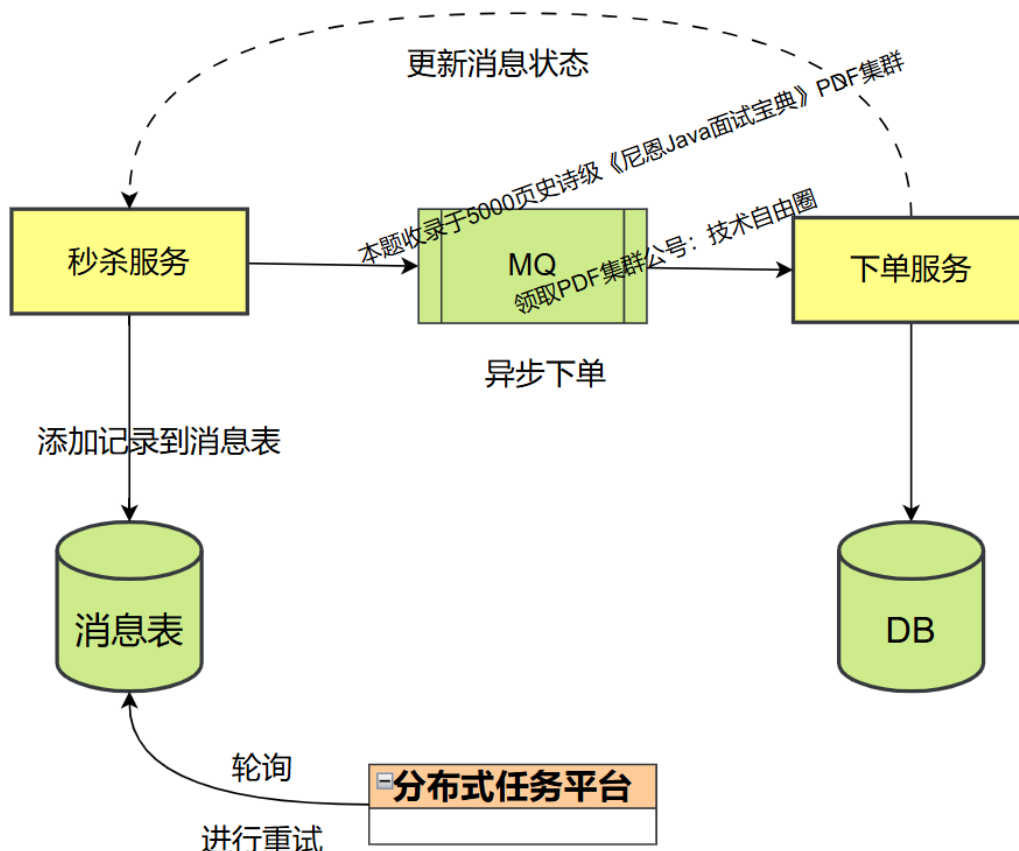
MQ三板斧 不一定能解决根本问题，因为百密一疏的原因，任何环节的异常，都有可能致数据丢失。

有没有业务维度的 终极保护措施呢？有，本地消息表+定时扫描

文章之中提到：**业务维度的 终极0丢失保护措施：本地消息表+定时扫描**

本地消息表+定时扫描 方案，和本地消息表事务机制类似，也是采用 本地消息表+定时扫描 相结合的架构方案。

大概流程如下图



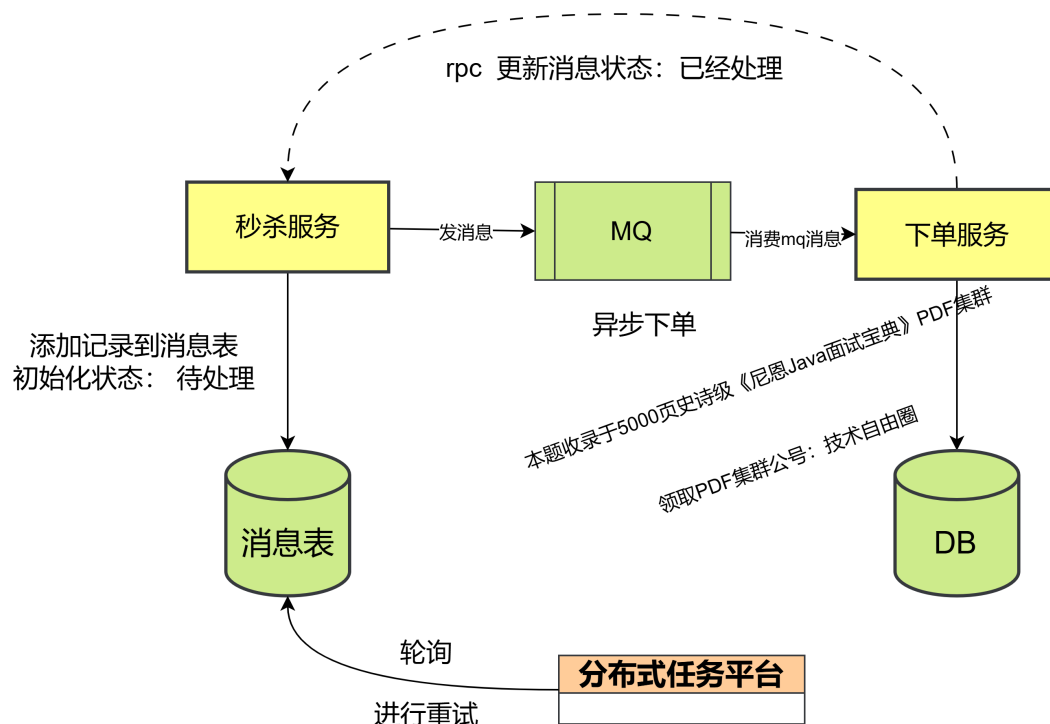
1、设计一个本地消息表，可以存储在DB里，或者其它存储引擎里，用户保存消息的消费状态

2、Producer 发送消息之前，首先保证消息的发生状态，并且初始化为待发送；

- 3、如果消费者（如库存服务）完成的消费，则通过RPC，调用Producer 去更新一下消息状态；
- 4、Producer 利用定时任务扫描 过期的消息（比如10分钟到期）， 再次进行发送。

在这里尼恩想说的是：本地消息表+定时扫描 的架构方案，是业务层通过额外的机制来保证消息数据发送的完整性，是一种很重的方案。

方案的设计如下：



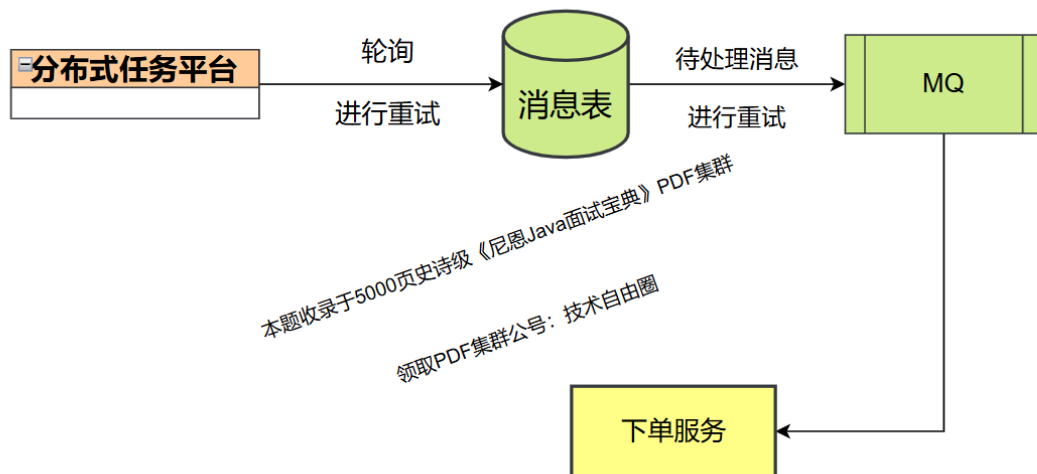
在生产者发送mq消息之前，先把该条消息写入消息发送表，初始状态是待处理，然后再发送mq消息。

消费者消费消息时，处理完业务逻辑之后，再回调生产者的一个接口，修改消息状态为已处理。

如果生产者把消息写入消息发送表之后，再发送mq消息到mq服务端的过程中失败了，造成了消息丢失。

这时候，要如何处理呢？

答：使用job，增加重试机制。



用job每隔一段时间去查询消息发送表中状态为待处理的数据，然后重新发送mq消息。

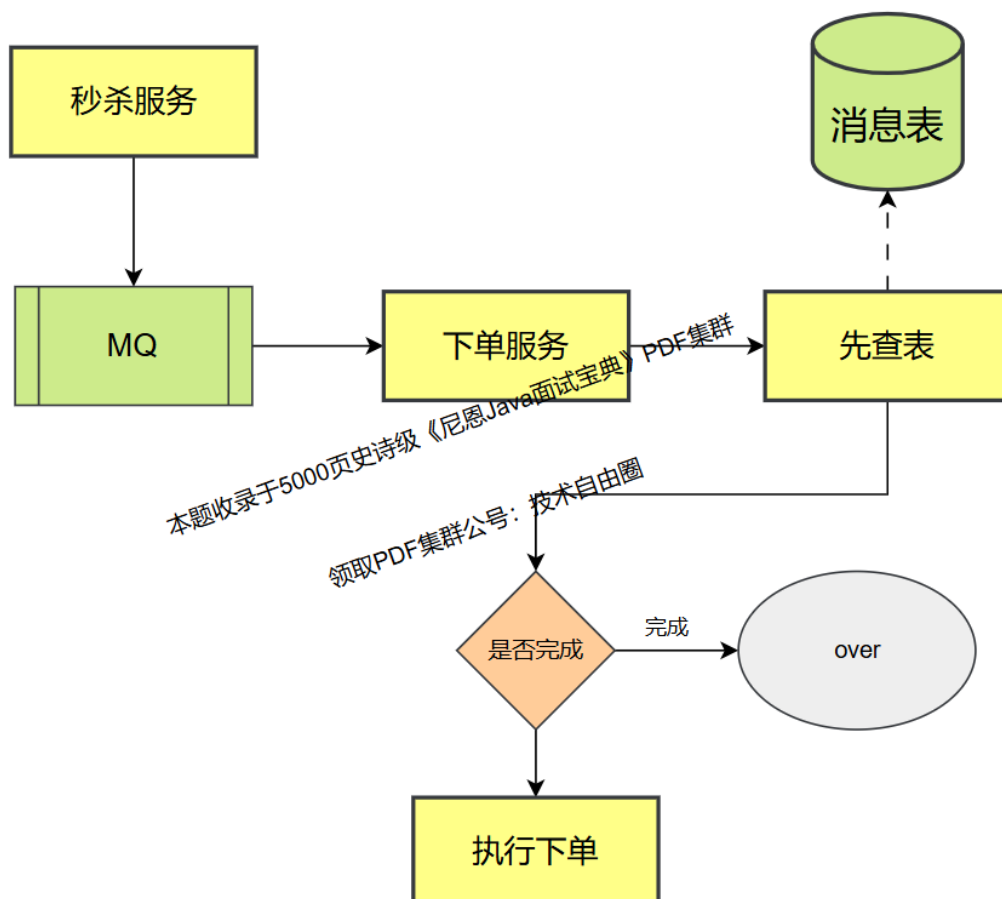
8.2 MQ消息的幂等性架构

本来消费者消费消息时，在ack应答的时候，如果网络超时，本身就可能会消费重复的消息。

但由于消息发送者增加了重试机制，会导致消费者重复消息的概率增大。

那么，如何解决重复消息问题，实现消息处理的幂等性呢？

答：处理之前，先查表。

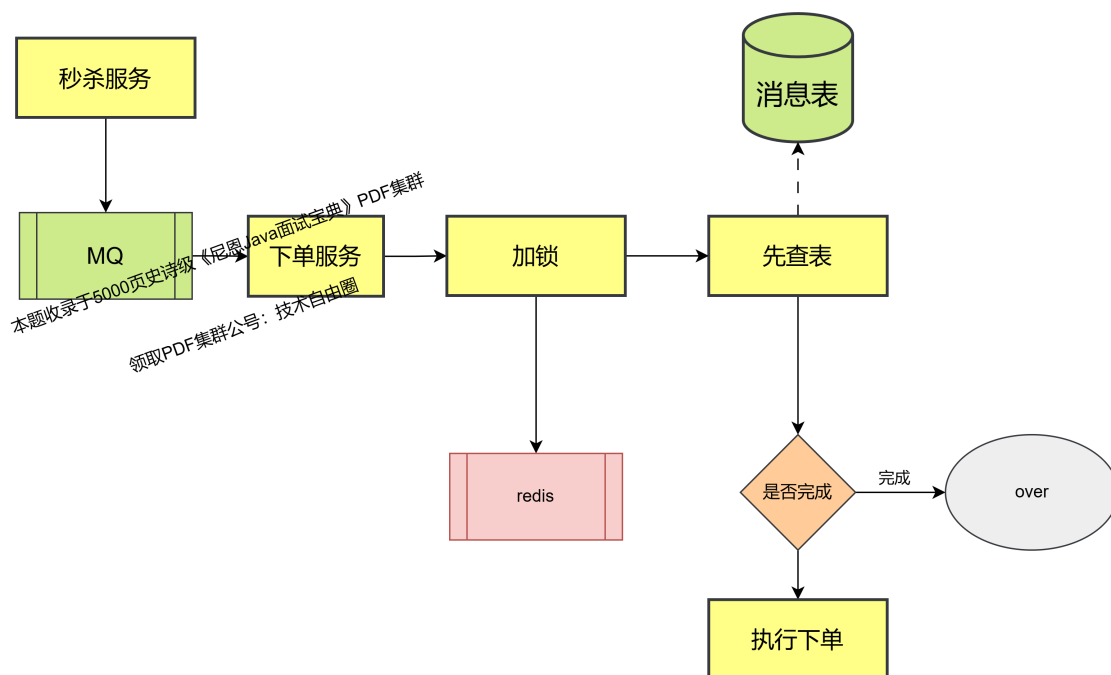


消费者读到消息之后，先判断一下消息处理表，是否存在该消息，如果存在，表示是重复消费，则直接返回。

如果不存在，则进行下单操作，接着将该消息写入消息处理表中，再返回。

有个比较关键的点是：下单和写消息处理表，需要保证原子操作。如何保证这个原子性，可以使用分布式锁：

[最系统的幂等性方案：一锁二判三更新](#)



这个方案，也是一个非常好的幂等性方案，建议大家好好掌握。

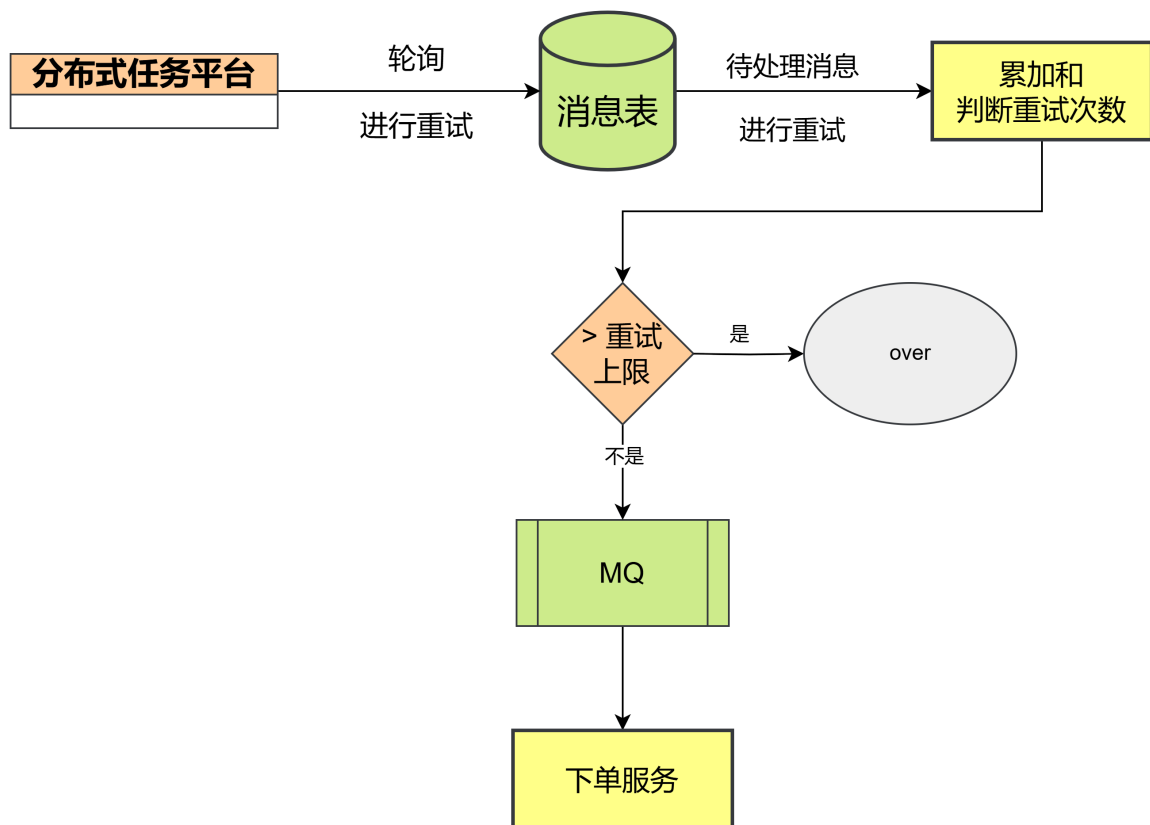
[最系统的幂等性方案：一锁二判三更新](#)

8.3 垃圾消息问题

这套方案表面上看起来没有问题，但如果出现了消息消费失败的情况。

比如：由于某些原因，消息消费者下单一直失败，一直不能回调状态变更接口，这样job会不停的重试发消息。

最后，会产生大量的垃圾消息。那么，如何解决这个问题呢？



每次在job重试时，需要先判断一下消息发送表中该消息的发送次数是否达到最大限制，如果达到了，则直接返回。

如果没有达到，则将次数加1，然后发送消息。

这样如果出现异常，只会产生少量的垃圾消息，不会影响到正常的业务。

8.4 消息积压问题

首先，如何确定RocketMQ有大量的消息积压？

在正常情况下，使用MQ都会要尽量保证他的消息生产速度和消费速度整体上是平衡的，但是如果部分消费者系统出现故障，就会造成大量的消息积累。

所以消息积压是个需要时刻关注的问题。

如何对消息积压进行监控和预警呢？

对于RocketMQ来说，有个最简单的方式来确定消息是否有积压。那就是使用web控制台，就能直接看到消息的积压情况。

在web控制台的主题页面，可以通过Consumer管理按钮实时看到消息的积压情况。

TestTopic订阅组						
订阅组	MyConsumerGroup		延迟	0	最后消费时间	2020-11-28 16:53:35
Broker	队列	消费者终端	代理者位点	消费者位点	差值	上次时间
worker1	0		9	9	0	2020-11-28 16:53:35
worker1	1		10	10	0	2020-11-28 16:53:35
worker1	2		7	7	0	2020-11-28 16:53:35
worker1	3		7	7	0	2020-11-28 16:53:35

@稀土掘金技术社区

另外，也可以通过mqadmin指令在后台检查各个Topic的消息延迟情况。

还有RocketMQ也会在他的 `${storePathRootDir}/config` 目录下落地一系列的json文件，也可以用来跟踪消息积压情况。

其次，如何处理大量积压消息？

其实我们回顾下RocketMQ的负载均衡的内容就不难想到解决方案。

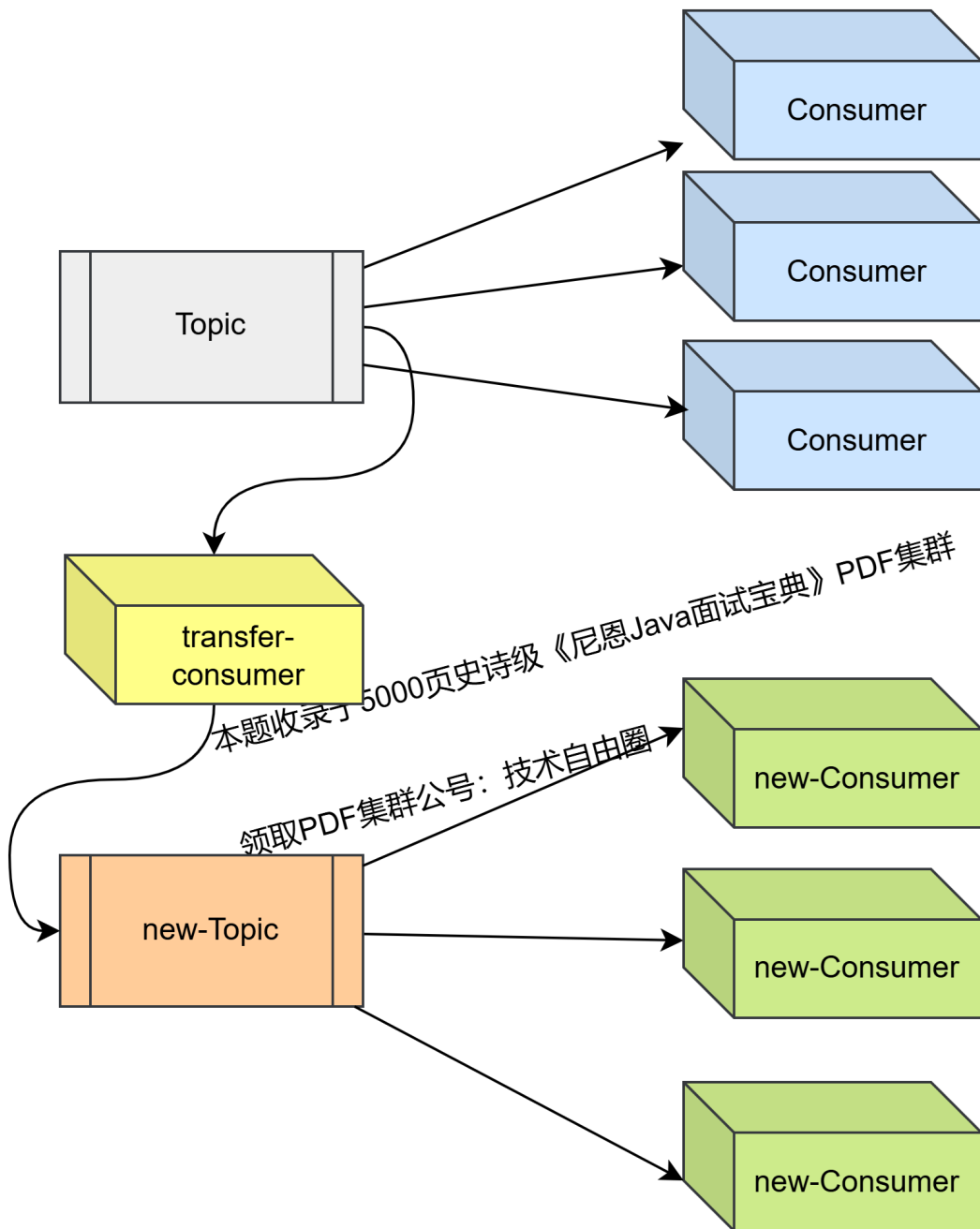
如果Topic下的MessageQueue配置的是足够多的，那每个Consumer实际上会分配多个MessageQueue来进行消费。

所以，第一招就是 增加 Consumer。但是，如果 Consumer的节点个数设置成跟MessageQueue的个数相同，此时再继续增加Consumer的服务节点就没有用了。

所以，如果Topic下的MessageQueue配置的不够多的话，就不能用上面这种增加Consumer节点个数的方法了。

这时怎么办呢？

这时，可以创建一个新的new-Topic，配置足够多的MessageQueue，同时，开启一批 new-consumer 消费者节点。



接下来，进行消息的转储。

紧急上线一组新的 transfer-consumer 消费者，只负责转储，就是消费老Topic中的积压消息，并转储到新的Topic中，这个速度是可以很快的。

然后新的Topic上，就可以通过增加消费者个数来提高消费速度了

解决积压之后，再根据情况恢复成正常情况。

[阿里面试：如何保证RocketMQ消息有序？如何解决RocketMQ消息积压？](#)

9 秒杀高可用架构

9.1 可用性和高可用概念

高可用，英文单词High Availability，缩写HA，它是分布式系统架构设计中一个重要的度量。

高可用（High Availability）的定义：（From 维基百科）是 IT 术语，指系统无中断地执行其功能的能力，代表系统的可用性程度，是进行系统设计时的准则之一。

行业内一般用几个9表示可用性指标，对应用的可用性程度一般衡量标准有三个9到五个9；

一般我们的系统至少要到 4 个 9（99.99%）的可用性才能谈得上高可用。业界通常用多个9来衡量系统的可用性，如下表：

可用性级别	系统可用性	宕机时间			
		每年	每月	每周	每天
不可用	90%	36.5 天	73 小时	16.8 小时	144 分钟
基本可用	99%	87.6 小时	7.3 小时	1.68 小时	14.4 分钟
较高可用	99.90%	8.76 小时	43.8 分钟	10.1 分钟	1.44 分钟
高可用	99.99%	52.56 分钟	4.38 分钟	1.01 秒	8.64 秒
极高可用	99.999%	5.26 分钟	26.28 秒	6.06 秒	0.86 秒

既然有可用率，有一定会存在不可用的情况。

不可用一般分为有计划的和无计划的，有计划的如日常维护、系统升级等，无计划的如设备故障、突发断电等。

服务不可能 100% 可用。

因此要提高我们的高可用设计，就要尽最大可能的去增加我们服务的可用性，提高可用性指标。

一句话来表述就是：高可用就是让我们的服务在任何情况下都尽最大可能能够对外提供服务。

9.2 高可用系统设计思想

高可用系统的设计，需要有一套比较科学的工程管理套路，要从产品、开发、运维、基建等全方位去考量和设计，高可用系统的设计思想包括但不限于：

- **做好产品层面的高可用设计**，主要是兜底策略。
- **做好研发规范**，系统都是研发人员设计和编码写出来的，因此首先要对研发层面有一个规范和标准
- **做好容量规划和评估**，主要是让开发人员对系统要抗住的量级有一个基本认知，方便进行合理的架构设计和演进。
- **做好服务层面的高可用**，主要是负载均衡、弹性伸缩、异步解耦、故障容错、过载保护等。
- **做好存储层面的高可用**，主要是冗余备份（热备、冷备）、失效转移（确认，转移，恢复）等。
- **做好运维层面的高可用**，主要是发布测试、监控告警、容灾、故障演练等。
- **做好多IDC机房层面的高可用**，比如秒杀的多IDC机房双活高可用架构
- **做好应急预案**，主要是在出现问题后怎么快速恢复，不至于让我们的异常事态扩大。

9.3 产品层面的高可用设计

产品层面的高可用架构解决方案，基本上就是指我们的兜底产品策略。这里说的兜底策略，也可叫做柔性降级策略，更多则是通过产品层面上来考虑。

- 比如，当我们的页面获取不到数据的时候，或者无法访问的时候，要如何友好的告知用户，比如【稍后重试】之类的。
- 比如 当我们的真实的页面无法访问的时候，那么需要产品提供一个默认页面，如果后端无法获取真实数据，那么直接渲染默认页面。
- 比如服务器需要停机维护，那么产品层面给一个停机页面，所有用户只会弹出这个停机页面，不会请求后端服务
- 比如抽奖商品给一个默认兜底商品
- ...

比如，当并发量太高时，我们会为用户弹出下面的排队页面，来提示用户进行等待。



再例如，如果redis 的库存已经销售完毕，如果后续仍然有用户发起秒杀请求，则后续的请求我们可以不再处理，直接向用户返回商品已售完的提示。



9.4 研发规范层面

研发规范层面这个是大家容易忽视的一个点，从设计文档到编码到发布上线，需要一个规范流程和套路，来让我们更好的去研发和维护一个高可用的系统：

第一阶段：设计阶段

- 规范好相关方案设计文档的模板和提纲，让团队内部保持统一，
- 方案设计后一定要进行评审，新项目一定要评审，重构项目一定要评审，大的系统优化或者升级一定要评审，其他的一般研发工作量超过一周的建议要评审的。

第2阶段：编码阶段

- 工程的 layout 目录结构规范，团队内部保持统一，尽量简洁
- 日志规范，不要随便打日志，要接入远程日志
- 遵循团队内部的代码规范，一般公司都有对应语言的规范，如果没有则参考官方的规范，代码规范可以大大减少 bug 并且提高可用性。
- 执行代码规范
- 要能够分布式链路追踪
- 单测覆盖率，代码编写完需要有一定的单测来保证代码的健壮性，同时也能保障我们后续调整逻辑或者优化的时候可以保证代码的稳定。包括增量覆盖率、全量覆盖率，具体的覆盖率要达到多少可以根据团队内部的实际情况来定，在我们团队，定的规则是 50% 的覆盖率。

第3阶段：发布上线阶段

参考下面运维部署层面那一章节的灰度发布和接口测试相关说明

9.5 应用服务层面的高可用设计

应用服务层面的高可用设计, 包括:

- 应用层的无状态和负载均衡设计
- 应用层的弹性伸缩设计
- 应用层的异步解耦和削峰设计（消息队列）
- 应用层的故障和容错设计
- 应用层的资源隔离设计

9.5.1 应用层的无状态和负载均衡设计

一般要做到系统的高可用，我们的应用服务的常规设计都是无状态的，

这也就意味着，我们可以部署多个实例来提高我们系统的可用性，而这多个实例之间的流量分配，就需要依赖我们的负载均衡能力。

无状态 + 负载均衡 既可以让我们的系统提高并发能力，也可以提高我们系统的可用性。

如果我们的业务服务使用的是各种微服务框架来开发的，那么大概率在这个微服务框架里面就会包含了服务发现和负载均衡的能力。

这是一整套流程，包括服务注册和发现、负载均衡、健康状态检查和自动剔除。当我们的任何一个服务实例出现故障后会被自动剔除掉，当我们有新增一个服务实例后会自动添加进来提供服务。

如果我们不是使用的微服务框架来开发的，那么就需要依赖负载均衡的代理服务，比如 LVS、Nginx 来帮我们实现负载均衡。

9.5.2 应用层的弹性伸缩设计

弹性伸缩设计是应对突峰流量的非常有效的手段之一，同时也是保障我们服务可用性的必要手段。

微服务Provider的弹性伸缩设计也叫作自动伸缩、自动扩容、自动缩容。

自动伸缩是和手动伸缩相对而言的。

自动伸缩就是通过运维工具实现资源监控，然后根据资源的紧张程度自动开启新的Java服务（微服务Provider），或者自动关闭一些空闲的Java服务。

传统的微服务Provider扩容策略是手动的，由运维人员手动进行。

- 一旦发现现有的微服务Provider能力不够，运维人员就会开启新的Java服务并且动态地加入集群；
- 一旦发现现有的微服务Provider能力有富余，运维人员就会关闭部分Java服务，这些Provider会动态地离开集群。

手动伸缩的问题是无法面对突发流量。

一旦出现突发流量，等到运维人员收到监控系统的资源预警信息后再去进行微服务Provider扩容，中间会有较大的时间延迟，在这个时间延迟内，系统可能已经发生雪崩了。所以，对于会出现突发流量的系统需要用到自动扩容、自动缩容的策略。

怎么实现弹性伸缩呢？常见的微服务Provider的自动伸缩策略有以下两种：

- 1) 通过Kubernetes HPA组件实现自动伸缩。
- 2) 通过微服务Provider自动伸缩伺服组件实现自动伸缩。

弹性伸缩针对的是我们的无状态的应用服务而言的，因为服务是无状态的，因此可以随时根据请求量的大小来进行扩缩容，流量大就扩容来应对大量请求，流量小的时候就缩容减少资源占用。

现阶段都是云原生时代，大部分的公司都是采用容器化（K8s）部署，那么基于这个情况的话，弹性伸缩就非常容易了，只需要配置好 K8s 的弹性条件就能自动根据 CPU 的使用率来实现。

Kubernetes HPA组件实现自动伸缩 的原理和实操，请参见尼恩的Java视频，第28章：

第28章：穿透云原生K8S+Jenkins+SpringCloud底层原理和实操

基础篇

00: 云原生，你真的懂了吗？

01: 云原生为何如此重要？来看看其巨大的企业价值、技术深度

02: Docker圣经：Docker,containerd, containerd-shim和runc之间的关系

03: Docker圣经：Docker的Linux 命名空间、控制组和UnionFS 三大技术底座

04: 云原生基础：云原生概念、云原生发展历史、云原生架构师的庞大需求

05: 架构师视角解读：常用的本地缓存组件、以及本地缓存的优点、缺点

06: K8S运行时环境：Minikube的一键启动、核心组件、核心架构学习，以及安装的前置条件

07: K8S运行时环境：生产场景（HA）高可用集群的两大架构，以及HA高可用集群搭建的核心流程

08: 云原生学习路线：总览一下SpringCloud+K8S云原生的学习重点，学习次序

K8S学习圣经

09: K8S学习圣经：结合案例学习，Kubernetes 资源对象的CRUD操作

10: K8S学习圣经：POD的什么周期的2个大阶段，3个小

阶段，以及什么是Pause容器？

11: K8S学习圣经：POD的InitContainer容器实操，start、stop两个Hook实操

12: K8S学习圣经：临时容器的原理，以及如何使用临时容器进行线上排错

13: K8S学习圣经：静态POD的原理，以及如何查看、修改静态POD的资源路径

14: K8S学习圣经：大白话介绍POD的就绪探针、存活探针、启动探针

15: K8S学习圣经：6大副本管理对象之Deployment、RS、DS、StatefulSet、JOB、HPA功能介绍

16: K8S学习圣经：Deployment、RS、DS、StatefulSet实操演示

17: K8S学习圣经：水平自动伸缩autoscale HPA的底层原理和两个前置条件(重点)

18: K8S学习圣经：使用HPA对SpringCloud微服务进行自动伸缩(重点)

19: K8S学习圣经：SVC四层流量分发、负载均衡实操

20: K8S学习圣经：kube-proxy工作的三大模式UserSpace、Iptables、Ipvs底层原理

进阶篇

21: 架构师视角解读：七层流量负载均衡Ingress的底层原理、组成部分、核心流程

22: 七层负载均衡：IngressController部署实操、Ingress

资源对象使用实操

23: 七层负载均衡：IngressController基于hostNetwork + DaemonSet实现高性能和高并发

24: 灰度架构方案：蓝绿发布、金丝雀发布、滚动发布、AB测试的原理和使用场景

25: 灰度架构实操：使用SVC实操一下蓝绿发布、金丝雀发布、滚动发布

26: 微服务云原生：使用K8S+Harber手动部署SpringCloud应用的核心流程和实操（1）

27: 微服务云原生：使用K8S+Harber手动部署SpringCloud应用的核心流程和实操（2）

28: 灰度架构实操：SpringCloud+Jenkins+ K8s Ingress 自动化灰度发布（1）

29: 灰度架构实操：SpringCloud+Jenkins+ K8s Ingress 自动化灰度发布（2）

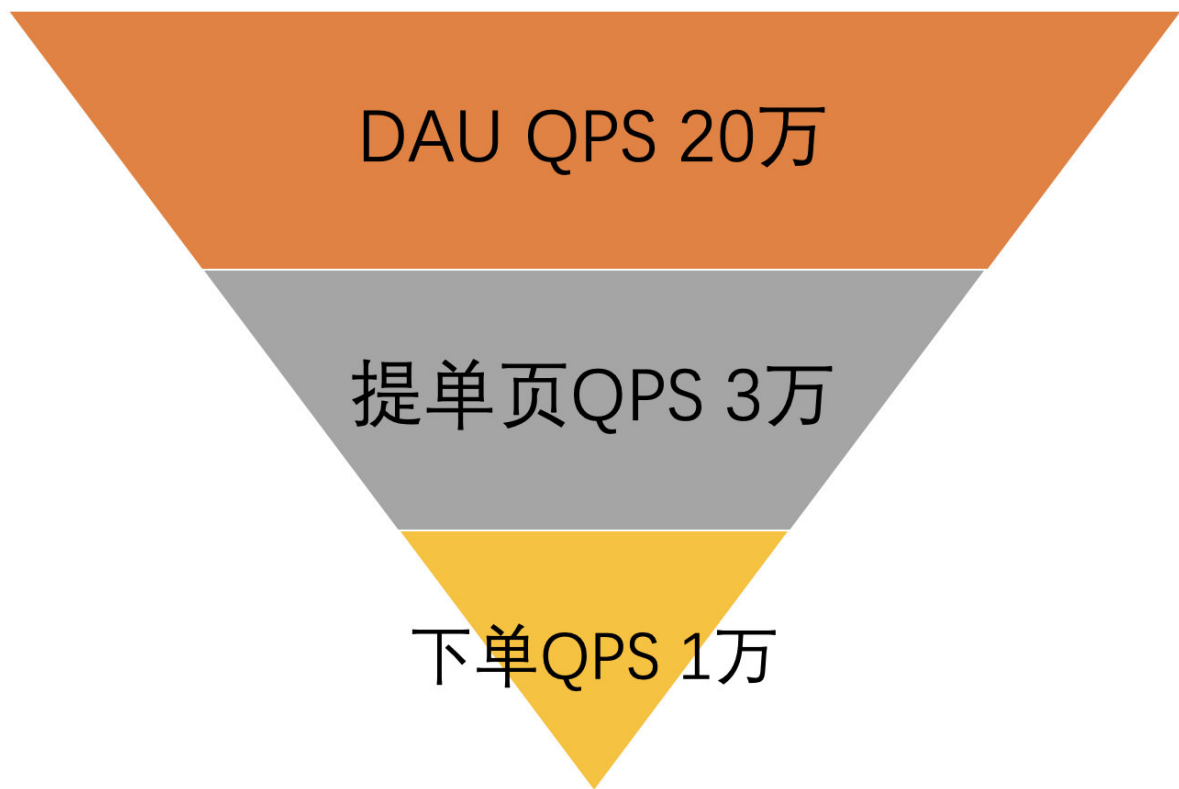
30: 灰度架构实操：SpringCloud+Jenkins+ K8s Ingress 自动化灰度发布（3）

9.5.3 应用层的异步解耦和削峰设计（消息队列）

要想我们的系统能够高可用，那么从架构层面来说，

要做到分层、分模块来设计，而分层分模块之后，那么各个模块之间，还可以进行异步处理、解耦处理。目的是为了不相互影响，通过异步和解耦可以使我们的架构大大的提升可用性。

经过异步解耦和削峰设计，最终我们的流量应该是呈漏斗状。



架构层面的异步解耦的方式就是采用消息队列（比如常见的 Kafka、Rocketmq），并且同时消息队列还有削峰的作用，这两者都可以提高我们的架构可用性：

- 异步解耦：采用消息队列之后，可以把同步的流程转换为异步的流程，消息生成者和消费者都只需要和消息队列进行交互，这样不仅做了异步处理，还讲消息生成者和消费者进行了隔离。
异步处理的优势在于，不管消息的后续处理的业务服务是否 ok，只要消息队列还没满，那么就可以执行对外提供服务，而消费方则可以根据自身处理能力来消费消息后进行处理。
解耦的优势在于，如果消费方异常，那么并不影响生产方，依然可以对外提供服务，消息消费者恢复后可以继续从消息队列里面消费数据后执行业务逻辑
- 削峰：采用消息队列之后，还可以做到削峰的作用，当并发较高的时候，甚至是流量突发的时候，只要消息生产者能够将消息写入到消息队列中，那么这个消息就不会丢，后续处理逻辑可以慢慢的去消息队列里面消费这些突发的流量数据。这样就不会因为有突发流量而把整个系统打垮。

9.5.4 应用层的故障和容错设计

任何服务，一定会存在失败的情况，不可能有 100% 的可用，服务在线上运行过程中，总会遇到各种各样意想不到的问题会让你的服务出现状况，因此业界来评价可用性 SLA 都是说多少个 9，比如 4 个 9(99.99%)的可用性。

为此，我们的设计建议遵循"design for failure"的设计原则，设计出一套可容错的系统，需要做到尽早返回、自动修复，细节如下

- 遵循 fail fast 原则，Fail fast 原则是说，当我们的主流程的任何一步出现问题的时候，应该快速合理地结束整个流程，尽快返回错误，而不是等到出现负面影响才处理。
- 具备自我保护的能力。当我们依赖的其他服务出现问题的时候，要尽快的进行降级、兜底等各种异常保护措施，避免出现连锁反应导致整个服务完全不可用。比如当我们依赖的数据存储出现问题，我们不能一直重试从而导致数据完全不可用。

9.5.5 应用层的资源隔离设计

资源隔离设计的目标是：系统里，某块故障时，不会耗尽系统所有资源（如线程资源）。

船舶工业上为了使船不容易沉没，使用舱壁将船舶划分为几个部分，以便在船体遭到破坏的情况下可以将船舶各个部件密封起来。

泰坦尼克号沉没的主要原因之一就是其舱壁设计不合理，水可以通过上面的甲板进入舱壁的顶部，导致整个船体淹没。

可见，资源隔离的架构设计，在造船工业中是多么重要。

资源隔离设计的理论可以用在很多领域，比如在服务层的RPC调用过程中，使用舱壁模式可以保护有限的系统资源不被耗尽。

在一个基于微服务的应用程序中，通常需要调用多个微服务提供者的接口才能完成一个特定任务。

不使用舱壁模式，所有的RPC调用都从同一个线程池中获取线程，一个具体的实例如图6-4所示。在该实例中，微服务提供者Provider A对依赖Provider B、Provider C、Provider D的所有RPC调用都从公共的线程池中获取线程。

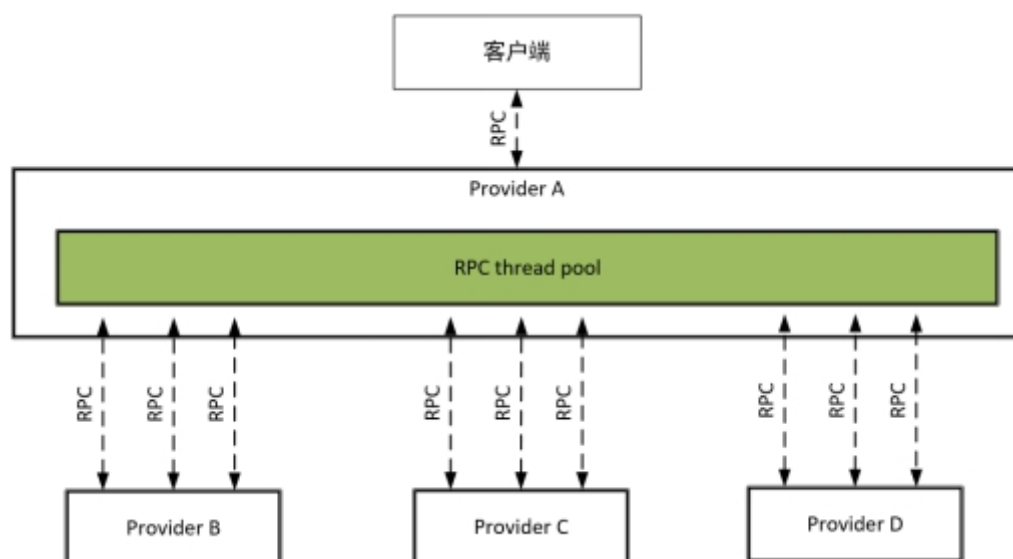


图6-4 公共的RPC线程池，from 尼恩《Java 高并发核心编程 卷3 加强版》，清华大学出版社出版。

在高服务器请求的情况下，对某个性能较低微服务提供者的RPC调用很容易“霸占”整个公共的RPC线程池，对其他性能正常的微服务提供者的RPC调用往往需要等待线程资源的释放。

最后，整个Web容器（Tomcat）会崩溃。现在假定Provider A的RPC线程个数为1000，而并发量非常大，其中有500个线程来执行Provider B的RPC调用，如果Provider B不小心宕机了，那么这500个线程都会超时，此时剩下的服务Provider C、Provider D的总共可用的线程为500个，随着并发量的增大，剩余的500个线程估计也会被Provider B的RPC耗尽，然后Provider A进入瘫痪，最后导致整个系统的所有服务都不可用，这就是服务的雪崩效应。

为了最大限度地减少Provider之间的相互影响，一个很好的做法是对于不同的微服务提供者设置不同的RPC调用线程池，让不同RPC通过专门的线程池请求到各自的Provider微服务提供者，像舱壁一样对Provider进行隔离。

对于不同的微服务提供者设置不同的RPC调用线程池，这种模式就叫作舱壁模式，如图6-5所示。

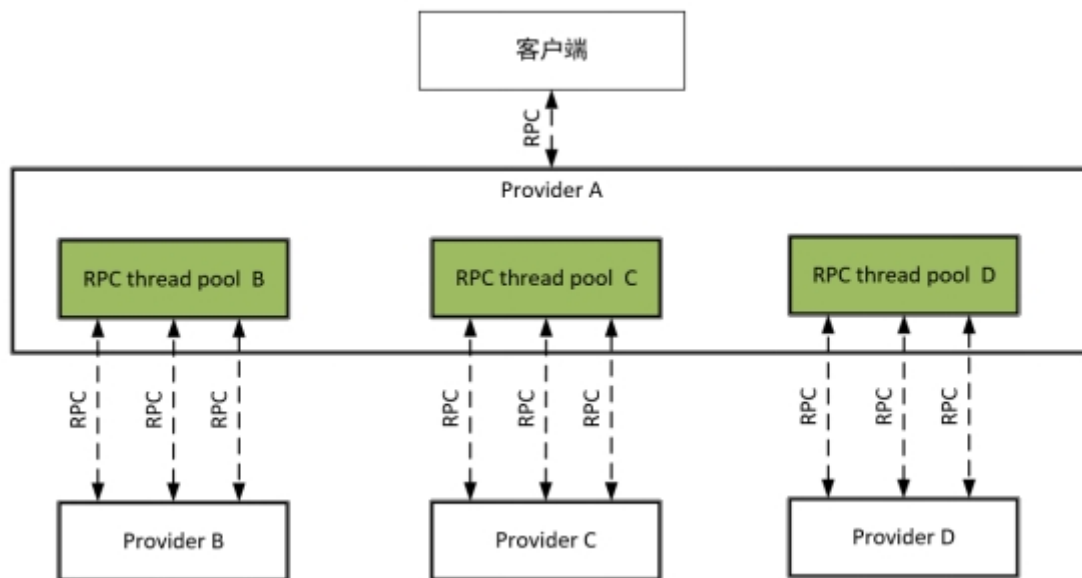


图6-5 舱壁模式的RPC线程池，from 尼恩 《Java 高并发核心编程 卷3 加强版》，清华大学出版社出版。

使用舱壁模式可以避免对单个Provider的RPC消耗掉所有资源，从而防止由于某一个服务性能底而引起的级联故障和雪崩效应。

在Provider A中，假定对服务Provider B的RPC调用分配专门的线程池，该线程池叫作Thread Pool B，其中有10个线程，只要对Provider B的RPC并发量超过了10，后续的RPC就走降级服务，就算服务的Provider B挂了，最多也就导致Thread Pool B不可用，而不会影响系统中的其他服务的RPC。

一般来说，RPC线程与Web容器的IO线程也是需要隔离的。

如图6-6所示，当Provider A的用户请求涉及Provider B和Provider C的RPC的时候，Provider A的IO线程会将任务交给对应的RPC线程池里面的RPC线程来执行，Provider A的IO线程就可以去干别的事情去了，当RPC线程执行完远程调用的任务之后，就会将调用的结果返回给IO线程。如果RPC线程池耗尽了，IO线程池也不会受到影响，从而实现RPC线程与Web容器的IO线程的相互隔离。

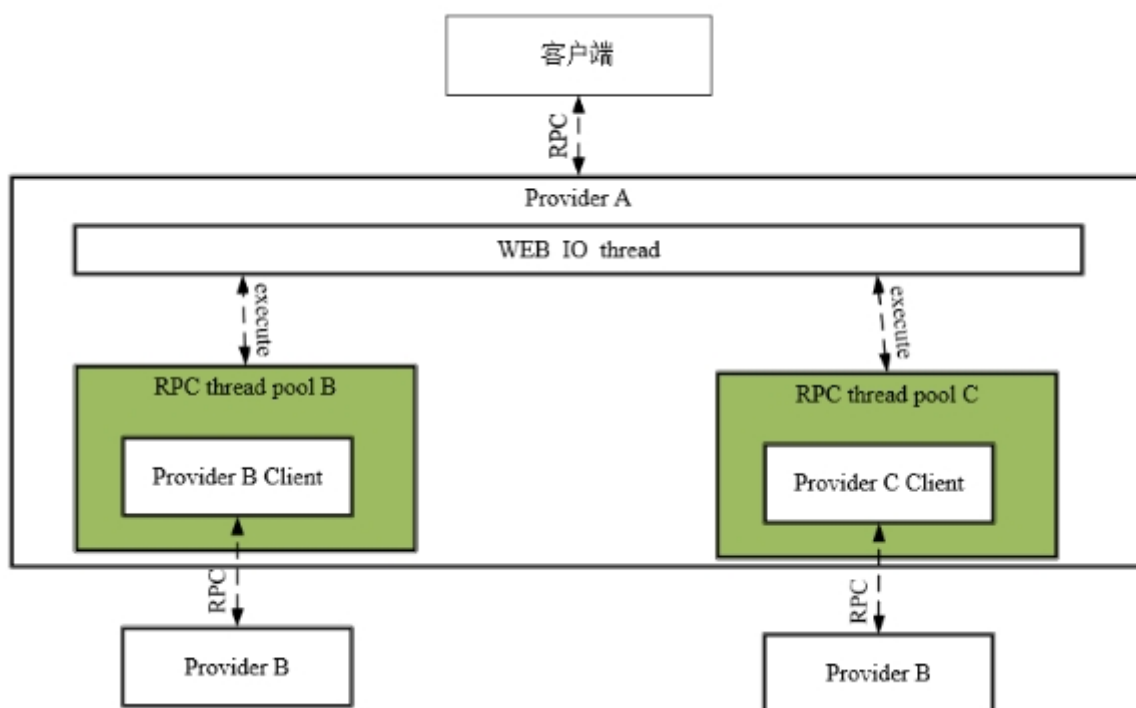


图6-6 RPC线程与Web容器的IO线程相互隔离

Hystrix提供了两种RPC隔离方式：线程池隔离和信号量隔离。

由于信号量隔离不太适合使用在RPC调用的场景，所以这里重点介绍线程池隔离。

虽然线程在就绪状态、运行状态、阻塞状态、终止状态间转变时需要由操作系统调度，这会带来一定的性能消耗，但是Netflix详细评估了使用异步线程和同步线程带来的性能差异，结果表明在99%的情况下异步线程带来的延迟仅为几毫秒，这种性能的损耗对于用户程序来说是完全可以接受的。

资源隔离设计的理论可以用在很多领域，其他领域，可以来尼恩的技术自由圈交流。

9.6 过载保护设计（限流、熔断、降级）

系统无法高可用的一个重要原因就在于：突发的流量过来，导致我们的服务超载运行。

如何实现 服务超载运行？

首先要做的当然是快速扩容，并且我们事先就要预留好一定的冗余。

然而，就算我们扩容了，但是还是会超载，比如超过了下游依赖的存储的最大容量、或者超过了下游依赖的三方服务的最大容量。

那么这个时候，我们就需要执行我们的过载保护。

过载保护策略，主要包括限流、熔断、降级，过载保护是为了保证服务部分可用从而不至于整个服务完全不可用。

- 限流。

限流是指对进入系统的请求进行限流处理，如果请求量超过了我们系统最大处理能力或者超过了我们指定的处理能力，那么直接拒绝请求，通过这种丢弃部分请求的方式可以保证整个系统有一定的可用性，从而不至于让整个系统完全不可用。

怎么判别超过最大处理能力呢？一般就是针对 QPS 来判别，如果 QPS 超过阈值，那么就拒绝请求。

限流有很多细节的策略，比如针对接口限流、针对服务限流、针对用户限流。

- 熔断。

熔断，断路（开路）的价值在于限制故障影响范围。

我们希望控制、减少或中断和故障系统之间的通信，从而降低故障系统的负载，有利于系统的恢复。

一般我们的服务都会有很多下游依赖，如果下游依赖的服务出现问题，比如开始超时甚至响应非常慢的情况下，如果我们不做任何处理，那么会导致我们的整个请求都被卡住从而超时，那么我们的业务服务对外就无法提供任何正常的功能了。

为此，熔断策略就可以解决这个问题，熔断就是当我们依赖的下游服务出现问题的时候，可以快速对其进行熔断（不发起请求），这样我们的业务服务至少可以提供部分功能。

- 降级。

降级是指我们划分好系统的核心功能和非核心功能，然后当我们的系统超过最大处理能力之后，直接关闭掉非核心的功能，从而保障核心功能的可用。

关闭掉非核心的功能后可以使我们的系统释放部分资源，从而可以有资源来处理核心功能。

熔断和降级这两个策略，看着比较像。

字面的意思上来看都是要快速拒绝掉请求。

但是他们是两个维度的设计，降级的目的是应对系统自身的故障，而熔断的目的是应对我们系统依赖的外部服务故障的情况。

接下来，单列几章，专门介绍这个。

10 秒杀的限流架构

高并发的流量涌入进来，比如突然间100万QPS，系统废了。应该10万QPS进入系统，其他90万QPS被拒绝了。

如何系统的最大吞吐上限为10Wqps

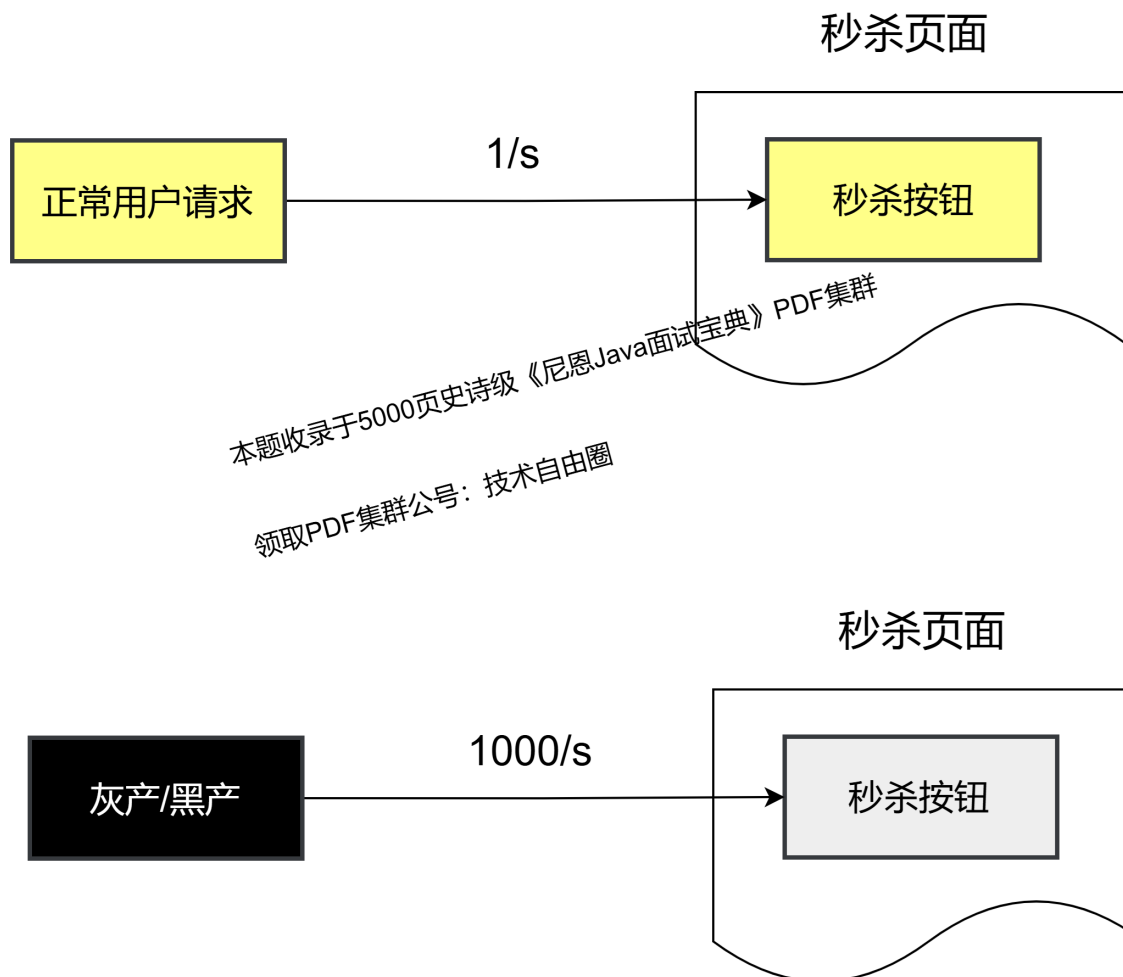
- 当高并发的流量涌入进来，比如突然间100万QPS，系统废了。
- 应该10万QPS进入系统，其他90万QPS被拒绝了。

通过秒杀活动，如果我们运气爆棚，可能会用非常低的价格买到不错的商品（这种概率堪比买福利彩票中大奖）。

但有些高手，并不会像我们一样老老实实，通过秒杀页面点击秒杀按钮，抢购商品。

他们可能在自己的服务器上，模拟正常用户登录系统，跳过秒杀页面，直接调用秒杀接口。

如果是我们手动操作，一般情况下，一秒钟只能点击一次秒杀按钮。



但是如果是灰产/黑产，他们使用分布式程序/分布式脚本，大批量的发送请求，一秒钟可以请求成千接口。

这种差距实在太明显了，如果不做任何限制，绝大部分商品可能是被机器抢到的，而非正常的用户，有点不太公平。

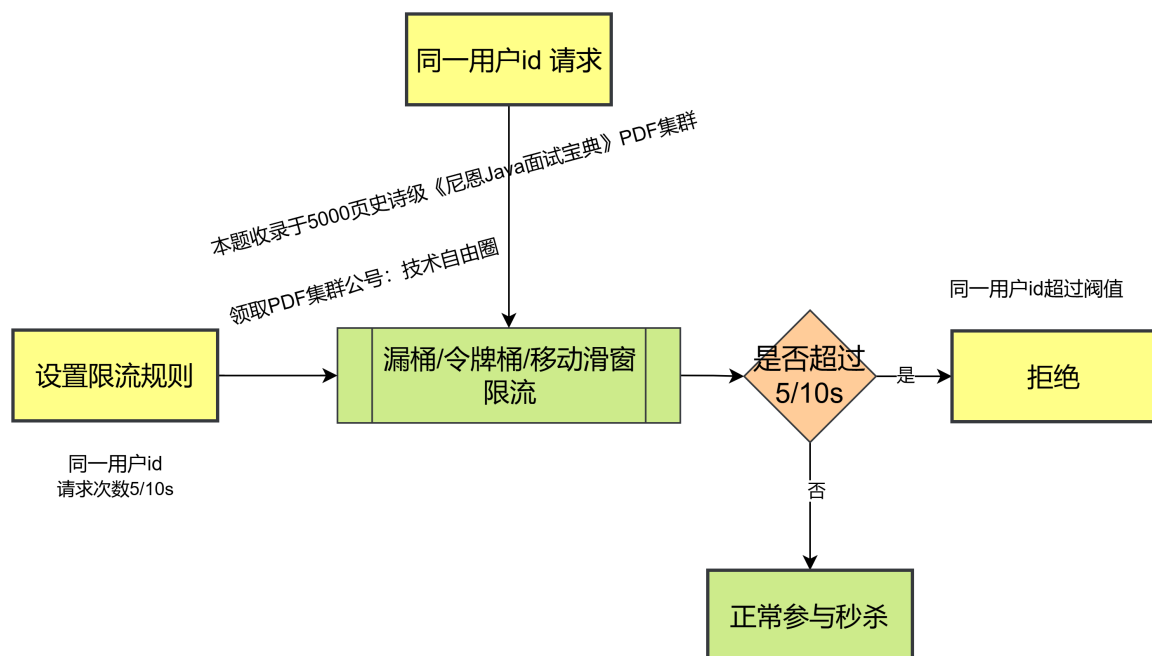
所以，我们有必要识别这些非法请求，做一些限制。那么，我们该如何现在这些非法请求呢？

目前有两种常用的限流方式：

1. 基于nginx限流
2. 基于redis限流

10.1 对同一用户限流

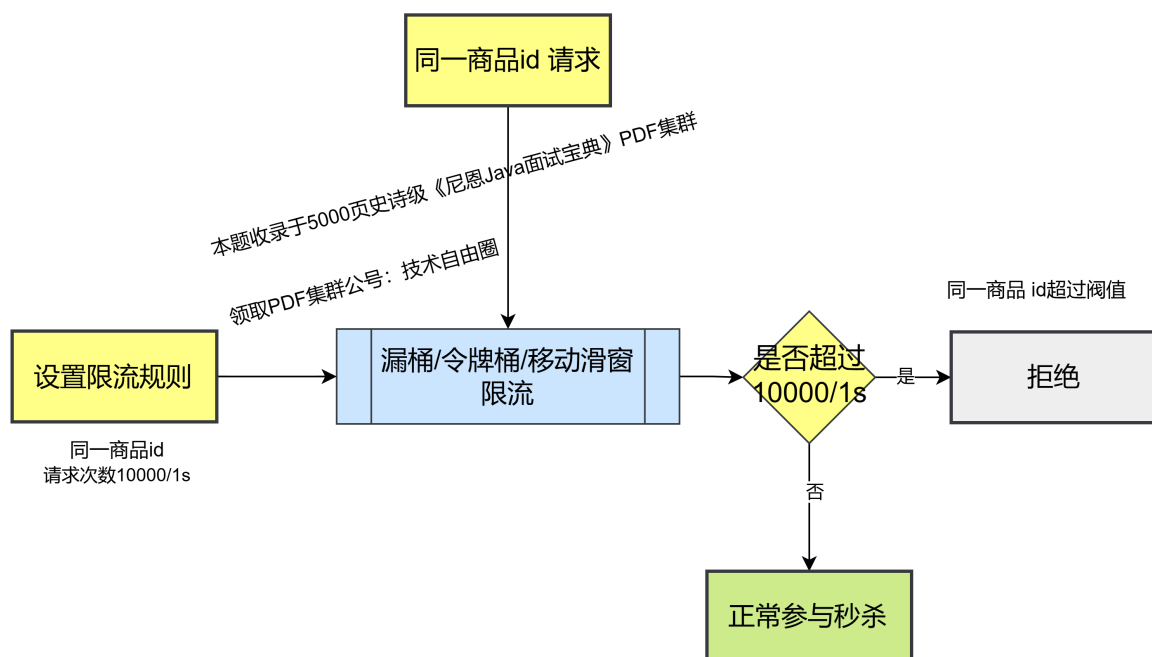
为了防止某个用户，请求接口次数过于频繁，可以只针对该用户做限制。



限制同一个用户id，比如每分钟只能请求5次接口。

10.2 对同一商品id 限流

有时候只对某个用户限流是不够的，有些高手可以模拟多个用户请求，这种nginx就没法识别了。这时需要加同一商品id限流功能。



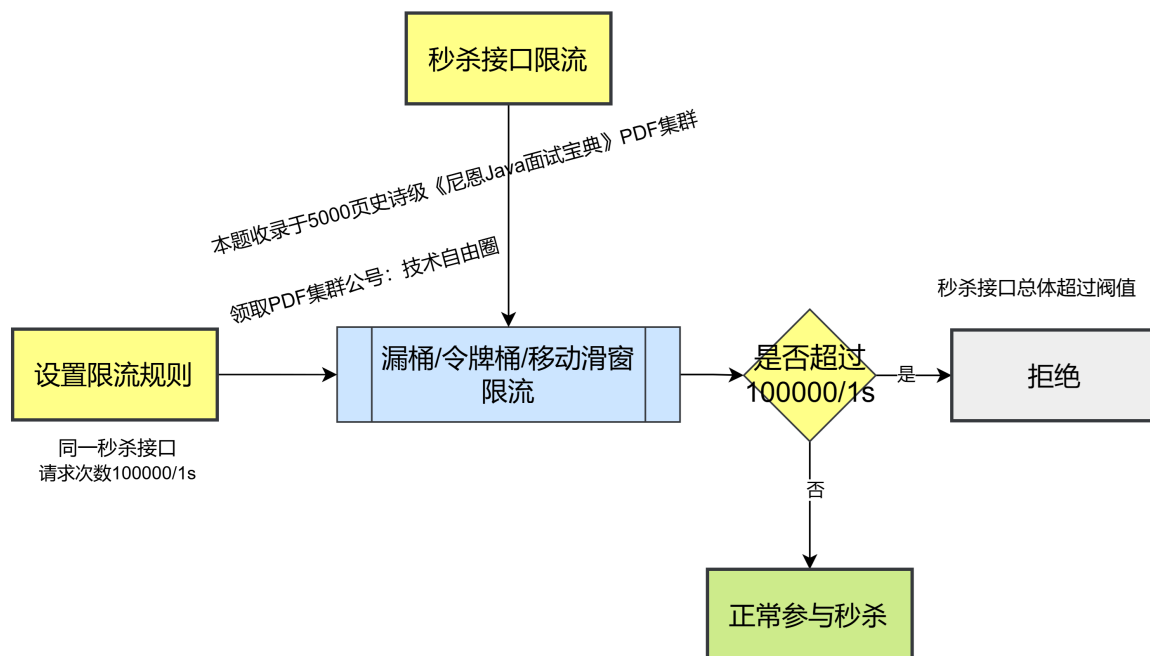
限制同一个商品id，比如每秒只能请求10000次接口。

但这种限流方式可能会有误杀的情况，比如同一个公司或网吧的出口ip是相同的，如果里面有多正常用户同时发起请求，有些用户可能会被限制住。

10.3 对接口限流

别以为限制了用户和ip就万事大吉，有些高手甚至可以使用代理，每次都请求都换一个ip。

这时可以限制请求的接口总次数。



在高并发场景下，这种限制对于系统的稳定性是非常有必要的。但可能由于有些非法请求次数太多，达到了该接口的请求上限，而影响其他的正常用户访问该接口。看起来有点得不偿失。

11 秒杀的熔断架构

A服务调用B服务的某个功能，由于网络不稳定或B服务宕机，导致功能时间超长。

若这样的次数太多，会拖垮我们的A服务。

为了避免a被b拖垮，我们就可以直接将B断路(A不再请求B接口)，凡是调用B的直接返回降级数据，不必等待B的超长执行。这样B的故障问题，就不会级联影响到A。

依赖服务，出了一些故障，每次请求都报错，熔断它，后续的请求过来直接不接收了，拒绝访问，10分钟之后再尝试去看看接口是否恢复。

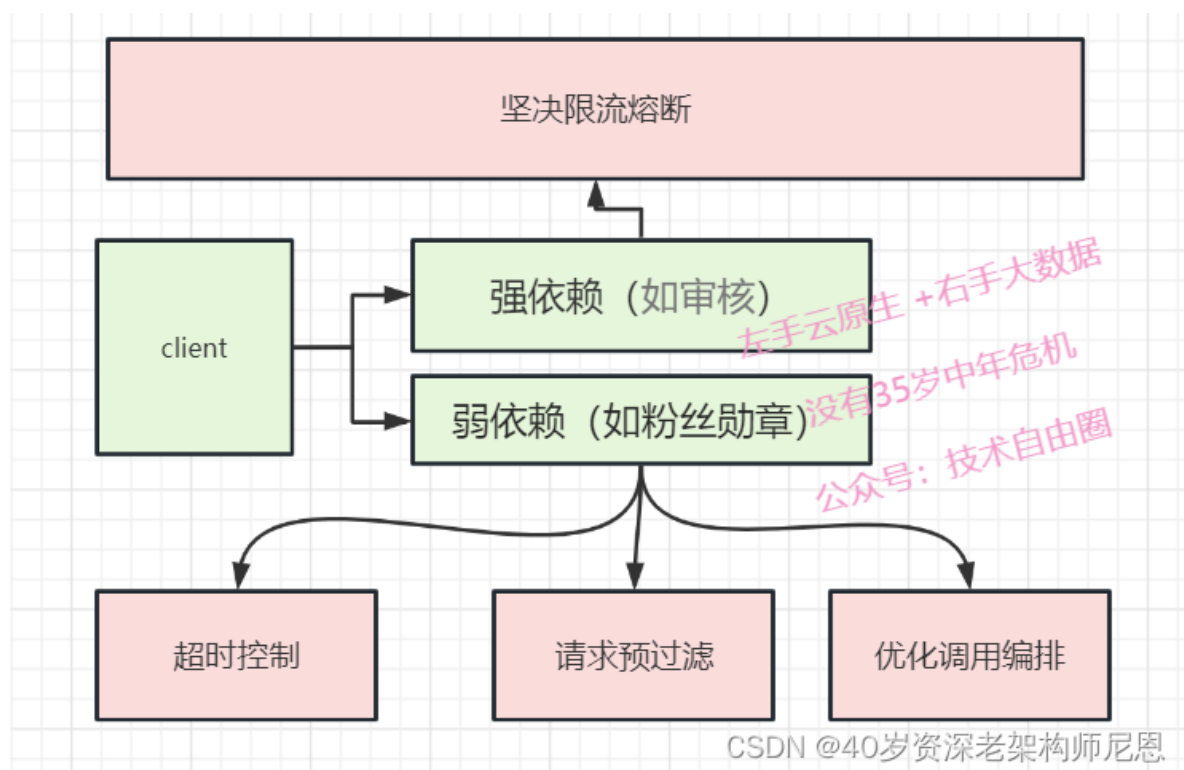
熔断策略的落地

为了尽可能 保证系统可用，在功能层面，做了级别划分：

把依赖划分为强依赖（如审核）、弱依赖（如粉丝勋章）。

首先，在如果强依赖出现异常，下坚决限流熔断，尽可能 保证 强依赖的可用性。

另外，对于弱依赖，通过超时控制、请求预过滤、优化调用编排，持续优化提升非核心功能的可用性，



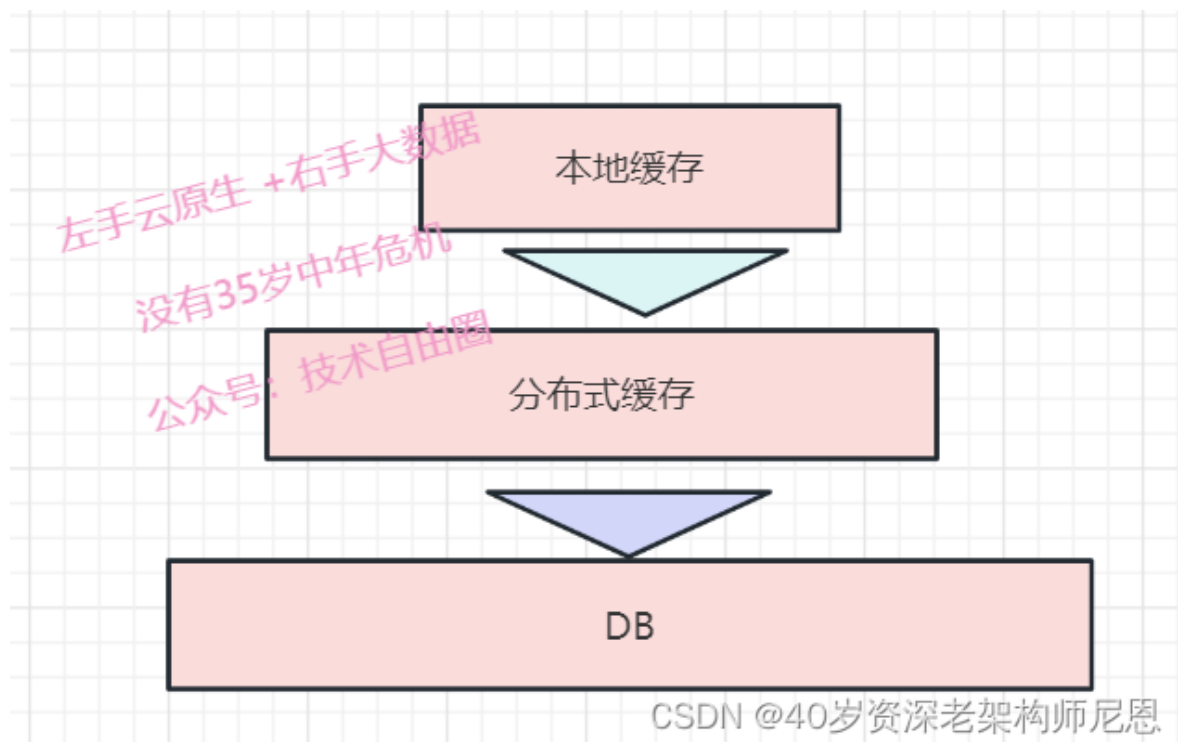
12 秒杀的降级架构

整个网站处于流量高峰期，服务器压力剧增，根据当前业务情况及流量，对一些服务和页面进行有策略的降级[停止服务，所有的调用直接返回降级数据]。

以此缓解服务器资源的压力，以保证核心业务的正常运行，同时也保持了客户和大部分客户的得到正确的响应。

缓存降级与DB降级

基础服务层集成了多级缓存，在上一级缓存未命中或者出现网络错误后，降级至下一级缓存，缓存没有命中，就降级到DB，保证系统的可用性。



降级和熔断的异同

相同点

- 为了保证集群大部分服务的可用性和可靠性，防止崩溃，牺牲小我
- 用户最终都是体验到某个功能不可用

不同点

- 熔断是被调用方故障，触发的系统主动规则
- 降级是基于全局考虑，停止一些正常服务，释放资源

需要注意的是，无论是限流、降级还是熔断，对业务都是有损的。所以，不到万不得已，不要进行降级和熔断。

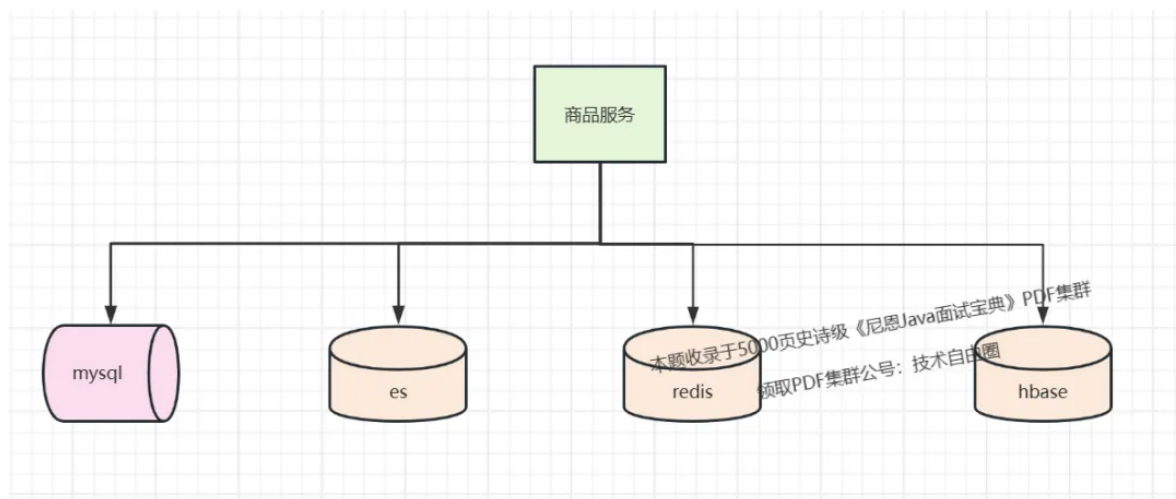
13 秒杀的存储架构

大家都知道，当一个表（比如订单表）达到500万条或2GB时，需要考虑水平分表。

秒杀系统，往往有着 百亿级数据存储架构，怎么设计？

咱们的生产需求上，百亿级数据存储架构，一般来说，需要具备以下多种能力/多种形式的异构存储架构：

- 1 高并发的在线ACID事务（分库分表）
- 2 高并发的在线搜索（倒排表副本）
- 3 海量数据的离线处理（高可用+全量副本）
- 4 冗余表双写能力（不同业务维度的副本）



里边经典的存储，比如 未来便于商品的聚合搜索 + 高速搜索，采用两大优化方案：

- 把商品数据冗余存储在Elasticsearch中，实现高速搜索
- 把商品数据冗余存储在redis 中，实现高速缓存

另外，在存储架构设计上，会考虑到的冗余表双写能力，也就是 高并发的 多业务维度 在线ACID 事务处理能力，比如在海量订单场景，

- 用户维度的在线ACID 事务订单处理能力，需要进行用户维度的分库分表。
- 商家维度的在线ACID 事务订单处理能力，需要进行商家维度的分库分表。

如果不需要 不同业务维度的 在线ACID 事务订单处理能力，那么冗余表双写能力 这个 确实没有必要。

问题来了，这是引入这么多的副本，有好处，也有坏处：

- 好处是满足各种各样的处理要求
- 坏处是我们要维护多个副本之间的数据一致。

所以接下来要解决：百亿级数据存储架构，多副本之间的数据一致如何实现？

既然有了多个副本，那么，如何保持很高的数据一致性？

比如：

- 要求 mysql 与 es 做到秒级别的数据同步。
- 要求 mysql 与 redis 做到秒级别的数据同步。
- 要求 mysql 与 hbase 做到秒级别的数据同步。

接下来，以 mysql 与 es 的数据一致，作为业务场景进行分析，其他的场景比如mysql 与 redis 的数据一致性方案，都是差不多的。

上面的架构方案，是一个大型工程，在此文中，没有办法展开啦。

从0到1，百亿级数据存储架构，40岁老架构师尼恩团队，计划用一个系列的文章帮大家实现这个架构难题，这个系列还会录成视频，并辅导大家写入简历。

这个系列包括：

- 高并发搜索ES圣经：从0到1，从入门到 ElasticSearch 工业级使用
- **100亿级任务调度篇：从0到1，从入门到 XXLJOB 工业级使用（已经发布）**
- 100亿级海量存储篇：从0到1，从入门到 HBase 工业级使用
- 100亿级离线计算篇：从0到1，从入门到 Flink 工业级使用

已经发布文章包括：

[字节面试：百亿级存储，怎么设计？只是分库分表？](#)

[100亿级任务调度篇：从0到1，从入门到 XXLJOB 工业级使用](#)

[高并发搜索ES圣经：从0到1，从入门到 ElasticSearch 工业级使用](#)

[超级底层：10WQPS/PB级海量存储HBase/RocksDB，底层LSM结构是什么？](#)

[百亿级存储架构：ElasticSearch+HBase 海量存储架构与实现](#)

14 运维部署层面的架构

开发阶段-灰度发布、接口测试设计

灰度发布、接口测试、接口拨测系列设计包括但不限于：

- 灰度发布，我们服务发布上线的时候，要有一个灰度的过程，先灰度 1-2 个服务实例，然后逐步放量观察，如果一切 ok，再逐步灰度，直到所有实例发布完毕
- 接口测试，每次服务发布上线的时候，服务提供的各种接口，都要有接口测试用例，接口测试用例跑过之后，服务才能发布上线，目的是为了查看我们对外提供的接口是否能够正常，避免服务发布上线后才发现问题

灰度发布和接口测试，一般在大公司里面会有相关的 DevOps 流程来保证。

开发阶段-监控告警设计

监控告警的设计，在大公司来说，根本不是问题，因为一定会有比较专门一拨人去做这种基础能力的建设，会有对应的配套系统，业务开发的同学只需要配置或使用即可。

那如果说公司内部没有相关基础建设，那么就需要自己分别来接入对应的系统了。

监控系统

一般在监控系统这方面的开源解决方案包括但不限于这些：

- ELK (Elasticsearch、Logstash、Kibana) 日志收集和分析
 - 我们的日志记录不能都本地存储，因为微服务化后，日志散落在很多机器上，因此必须要有一个远程日志记录的系统，ELK 是不二人选
- Prometheus 监控收集
 - 可以监控各种系统层面的指标，包括自定义的一些业务指标
- OpenTracing 分布式全链路追踪
 - 一个请求的上下游这么多服务，怎么能够把一个请求的上下游全部串起来，那么就要依靠 OpenTracing，可以把一个请求下的所有链路都串起来并且有详细的记录
- OpenTelemetry 可观测系统标准
 - 最新的标准，大一统，集合了跟踪数据 (Traces)，指标数据 (Metrics)，日志数据 (Logs) 来观测分布式系统状态的能力

我们会依托开源系统进行自建或者扩展，甚至直接使用都行，然后我们的监控的指标一般会包括：

- 基础设施层的监控：主要是针对网络、交换机、路由器等低层基础设备，这些设备如果出现问题，那么依托其运行的业务服务肯定就无法稳定的提供服务，我们常见的核心监控指标包括网络流量（入和出）、网络丢包情况、网络连接数等。
- 操作系统层的监控：这里需要包含物理机和容器。常见的核心指标监控包括 CPU 使用率、内存占用率、磁盘 IO 和网络带宽等。
- 应用服务层的监控：这里的指标会比较多，核心的比如主调请求量、被调请求量、接口成功率、接口失败率、响应时间（平均值、P99、P95 等）等。
- 业务内部的自定义监控：每个业务服务自己的一些自定义的监控指标。比如电商系统这里的：浏览、支付、发货等各种情况的业务指标
- 端用户层的监控：前面的监控更多的都是内部系统层面的，但是用户真正访问到页面，中间还有外网的情况，用户真正获取到数据的耗时、打开页面的耗时等这些信息也是非常重要的，但是这个一般就是需要客户端或者前端去进行统计了。

告警系统

这些系统接入完了之后，还只是做到监控和统计，当出现问题的时候，还需要进行实时告警，因此还要有一个实时告警系统，如果没有实时报警，系统运行异常后我们就无法快速感知，这样就无法快速处理，就会给我们的业务带来重大故障和灾难。告警设计需要包括：

- 实时性：实现秒级监控；
- 全面性：覆盖所有系统业务；
- 实用性：预警分为多个级别，监控人员可以方便实用地根据预警严重程度做出精确的决策；
- 多样性：预警方式提供推拉模式，包括短信，邮件，可视化界面，方便监控人员及时发现问题

开发阶段-安全性、防攻击设计

安全性、防攻击设计的目的是为了防刷、防黑产、防黑客，避免被外部恶意攻击，这个一般有几个策略：

- 在公司级别的流量入口做好统一的防刷和鉴权的能力，比如再统一接入层做好封装
- 在业务服务内部，做好相关的业务鉴权，比如登录态信息、比如增加业务鉴权的逻辑

部署阶段-多机房部署（容灾设计）

一般的高可用策略，都是针对一个机房内来服务层面来设计的，但是如果整个机房都不可用了，比如地震、火灾、光纤挖断等。。。那么这个情况怎么办？

这就需要我们的服务和存储都能够进行容灾了，容灾的一个常见方案就是多机房部署了。

- 服务的多机房部署，这个比较容易，因为我们的服务都是无状态的，因此只要名字服务能够发现不同机房的服务，就可以实现调用，这里需要注意的是名字服务（或者说负载均衡服务）要能够有就近访问的能力。
- 存储的多机房部署，这个会比较难搞一点，因为存储是有状态的，部署在不同的机房就涉及到存储的同步和复制问题。

条件不允许的情况下，我们保证多机房部署业务服务就可以了。

线上运行阶段-故障演练（混沌实验）

故障演练在大公司是一个常见的手段；在业界，Netflix 早在 2010 年就构建了混沌实验工具 Chaos Monkey，混沌实验工程对于提升复杂分布式系统的健壮性和可靠性发挥了重要作用。

简单的故障演练就是模拟机房断电、断网、服务挂掉等场景，然后看我们的整个系统运行是否正常。

系统的就要参考混沌实验工程来进行详细的规划和设计，这个是一个相对比较大的工程，效果挺好，但是需要有大量人力去开发这种基础建设。

线上运行阶段-接口拨测系列设计

接口拨测，和巡检类似，就是服务上线后，每隔一个固定时间（比如 5s）调用后端的各种接口，如果接口异常则进行告警

针对接口拨测，一般也会有相关配套设施来提供相关的能力去实现，如果没有提供，那么我们可以自己写一个接口拨测（巡检）的服务，定期去调用重要的接口。

15 秒杀的多IDC机房双活高可用架构

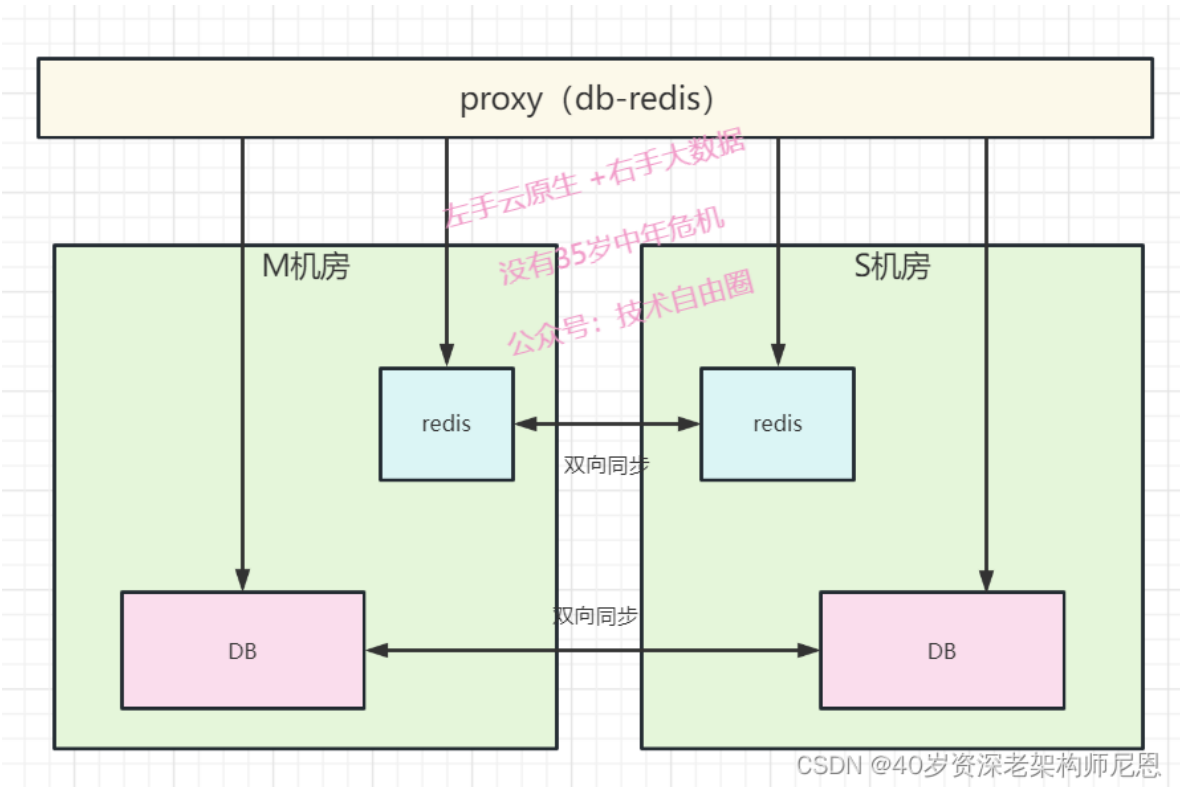
同城读双活 + 双机房独立部署 架构

评论系统是一个同城读双活+ 双机房独立部署 的架构。

DB 和 redis 均支持多副本，具备水平扩容的弹性。

双机房独立部署：

数据库与缓存均是双机房独立部署的，通过 db-proxy，或者 db-redis进行访问



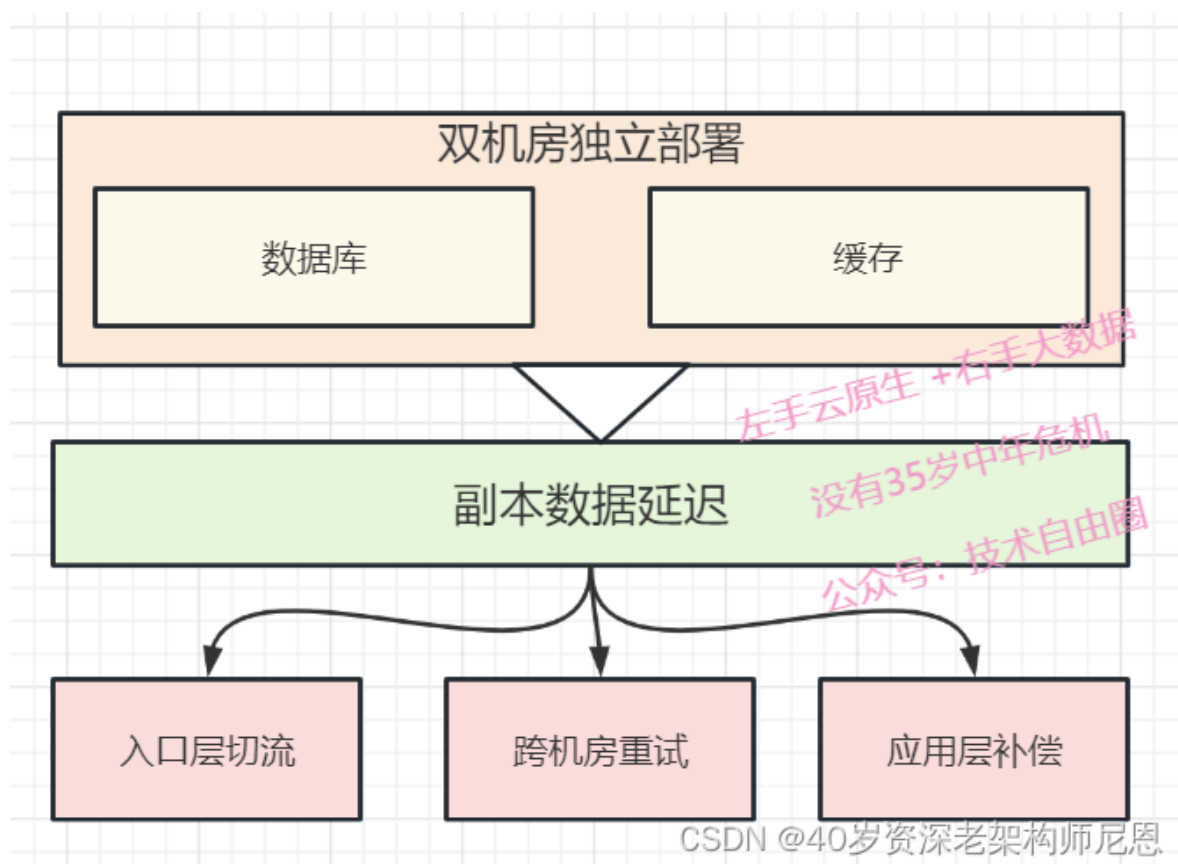
副本数据延迟优化架构

双机房架构场景下，存在跨机房数据延迟问题，

采用如下的策略解决：

- 入口层切流
- 应用层补偿
- 跨机房重试

尽可能保证极端情况下，用户没有延迟感。



16 秒杀的应急预案架构

前面做了这么多保障，但是终究架不住线上的各种异常情况，

如果真出问题了，让我们的服务异常，无法提供服务后，我们还需要最后一根救命稻草，那就是应急预案，将服务异常的损失降低到最小。

应急预案就是我们需要事先规划好，我们业务系统在各个层级出现问题后，我们需要第一时间怎么恢复，制定好相关规则和流程，当出现异常状况后可以按照既有的流程去执行，这样避免出现问题上手忙脚乱导致事态扩大。

说在最后：有问题找老架构取经

CPU100%，如何处理？

面试的时候如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

在面试之前，建议大家系统化的刷一波 5000页《[尼恩Java面试宝典](#)》V174，在刷题过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

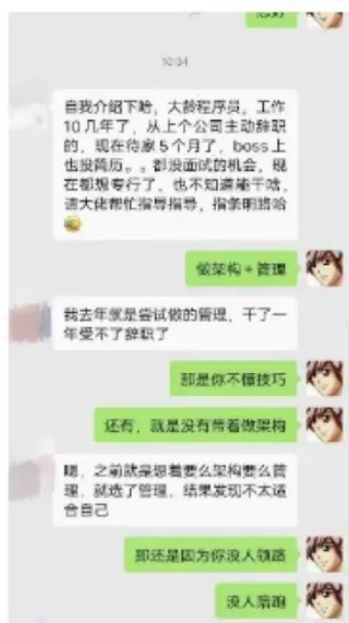
另外，如果没有面试机会，可以找尼恩来帮扶、领路。

- 大龄男的最佳出路是 架构+ 管理
- 大龄女的最佳出路是 DPM，



架构师尼恩6

大龄程序员，找不到出路了



54分钟前

架构师尼恩6：大龄男最佳出路： 架构+管理

架构师尼恩6：大龄女最佳出路： DPM

女程序员如何成为DPM，请参见：

[DPM（双栖）陪跑，助力小白一步登天，升格 产品经理+研发经理](#)

领跑模式，尼恩已经指导了大量的就业困难的小伙伴上岸。

前段时间，领跑一个40岁+就业困难小伙伴拿到了一个**年薪100W**的offer，小伙伴实现了 **逆天改命**。

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》** 377页 PDF
v1 版，发布日期：2023年03月15日
- 2 《Docker 学习圣经》** 145页 PDF
v2 版，发布日期：2023年03月12日
- 3 《Kubernetes 学习圣经》** 530页 PDF
v1 版，发布日期：2023年04月11日
- 4 《响应式圣经》** 114页 PDF
v2 版，发布日期：2023年02月14日
- 5 《缓存之王 Caffeine 红宝书》** 175页 PDF
v2 版，发布日期：2022年11月30日
- 6 《队列之王 Disruptor 红宝书》** 75页 PDF
v1 版，发布日期：2022年10月05日
- 7 《Prometheus+grafna 红宝书》** 100页 PDF
v1 版，发布日期：2022年10月08日

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》** 609页 PDF
加强版，发布日期：2023年01月01日
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》** 448页 PDF
加强版，发布日期：2022年11月14日
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》** 481页 PDF
加强版，发布日期：2022年11月14日

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题** 4000页 PDF
v60 版，发布日期：2023年03月11日

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班



已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

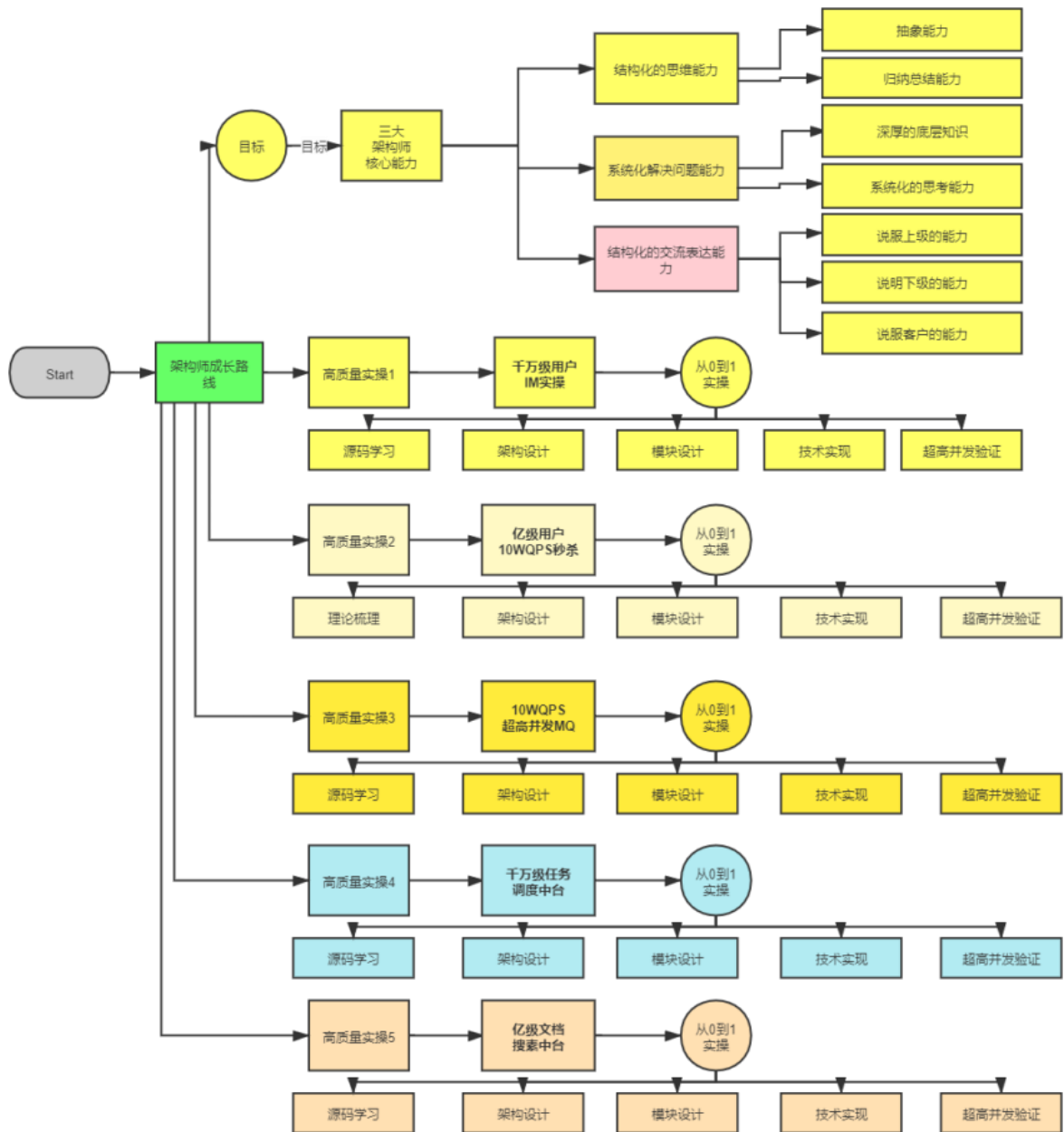
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

[illegible]

成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到 35K

2021年从8K涨到22K

2022年涨到35K

秘诀：
简历指导+ 狠狠卷

9年 小伙伴拿到 年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳定

这个是你微调的

省略号的地方，需要你再补充一点

9月11日 下午17:38

上面你留着这个就行

Java 开发 - 9 年-修改.docx 34.1 KB

主要的工作是啥?

提升了自己的实力，就不用怕

易所 关于数字货币的

po 也有 打盹的时候，该裁员，照样一个不少

年包比 po 多 19w

这么多

po 估计有 70 万

那你不是有 90 个 W?

po 给我 68

就是吗

小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

之前 36*15，现在这个 39*15

今年行情不太好，还有一些 offer 基本都是平薪，或降薪的。

OK

这个马上来

刚在指导简历

哈哈

稍等哈

辛苦啦

这次找工作，您的指导真的起到效果了。

我这次复习基本看的都是咱们课程的

6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

张这多

今年这行情，也算可以了

总包多少呀，让我也了解一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错啦

是的

太牛啦

哈哈哈哈哈，恩师的鼓励指导也很重要

给你前面调整了一下

Java - 6年.docx 24.7 KB

简历电话

恩师，简历我改好了，您再帮我看看一下

9月6日 下午14:47

Java - 6年(1).docx 24.5 KB

简历电话

恩师，看第二个吧

9月6日 晚上19:41

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

恩师老师晚上好，汇报下最近的 offer 情况。最近面试收到了三个 offer。两个是平薪，一个是跳槽电商公司的 offer，涨幅暂时不到 20%。通过这次面试也让我知道自己距离高级开发还有一点距离，还要再多卷才能突破。

最后结合自己的情况，先选择去跨境电商的公司再提升下

之前是 20k*13.5，跨境电商这个说是 23k* (14-15)

恭喜恭喜

毕业两年，能拿到 3 个 offer，已经很不错了。

很多小伙伴，面试电话一个都接不到，简历海投 7000 份，只收到 3 次面试机会，没有一个机会拿到最终 offer

您这个年薪，算下来也有 35W 了吧

在找部分还要确认下，大部分人说只有两个月

这个时间点，拿到这个水平，挺不错的啦

持续加油卷哈

嘿嘿！看看明年自己有没有能力冲击离开

把你视频都吃透

辛苦老师再帮忙指导下哈

秘诀:
简历指导+ 狠狠卷

算法方面，可以找点英语题，刷刷，当作复习

项目的题，再好好改造一下

好的 谢谢老师

5月22日 晚上22:45

恩师老师晚上好，结合老师的视频，我对自己的简历改了多次。看到大家经过老师的修改，有醍醐灌顶的感觉，我也想请老师也帮忙看看给一些建议~ 希望在当前环境有 offer 收获~

看量级 java 开发-本科统招-2022-5年经验.docx 35.9 KB

简历电话

OK

我排好号，回头找您哈

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止那啥... 我才是两...
原先7k, 现在15k

那也很牛...
都翻倍了, 都是牛人...
正常就30%...
慢点来...
下一步可以瞄准25k...
其实我工作一年...
那你就更牛逼拉...
一年经验, 搞定15k offer, 舍你其谁...

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人, 你带领多少人?

你工作几年了?

6年了

改简历脚窝, 20 涨到 25 k, 高级开发, P7 帮的后端团队

任务重, 道路远, 并不是没有方向

7年经验卷王 薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好, 拿 offer 比较容易的

入职了, 最终并没那么涨薪, 还是旧的太... 只涨了30%

恭喜一下

现在不比往年

能涨30%是大牛

拿到 offer 都算大牛

现在很多人投几百发, 连个电话都没有

你已经很... 拉

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历

8月30日晒offer



小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历

7月5日晒offer



卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历

4月15日晒offer

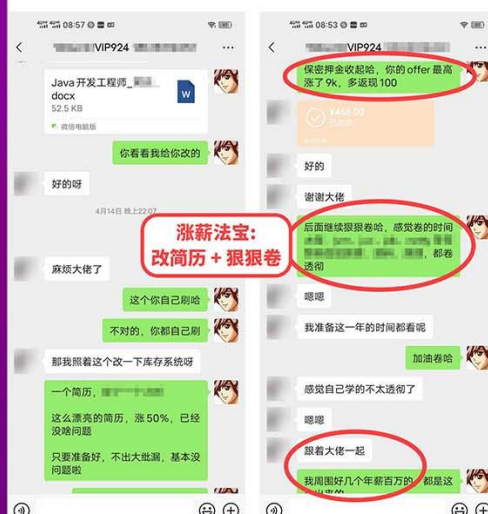


卷王逆袭成功案例

6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历

5月17日晒offer



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有其他很多offer的，还有一个小满的，总体上看boss和小满之间选一个。

了

boss比小满年包多7w以上。

我涨幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发出去确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没了我和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

offer决策图

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台内部系统，新部门，人数少，加班多。

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少呀

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

**涨薪法宝：
改简历 + 狠狠卷**

这个不用加粗

都是一样，加粗了反而没有效果

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空中心

这太牛啦

其实拿到手的也就一个a类国一

尼恩大佬 大三的暑期实习找不到，现在准备秋招应该没关系吧

看身边总是很焦虑，自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习，看您写的书比之前轻松多了

趁着还是学生这段时间，慢慢把知识吃透

嗯哪，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

668.00

模块的开发工作，就这个三个哈

尼恩老师

哈哈，不用了老师，您真的厉害了，光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就说是其他人做的

嗯哪，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

668.00

只收了一笔服务费

咱们得说这话呀

老弟拿着您的那个高并发改造亮点，所向披靡。

ok

从头条刷到面试官讲的明白白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

9大offer 最高年薪30万

公司	部门	岗位	薪资结构	总包
1. 公司	数据数字化产品部	java后端开发	18.5k+14.5k+5k+200k/每月+500k/期权	>30w
2. 公司	交易研发部	java后端开发	16k+14k+15k+100k/每月	22.4w
3. 公司	待定	java游戏开发	15k+15k+100k/每月	>22.5w
4. 公司	待定	java游戏开发	11k+13k+100k/每月	14.2w
5. 公司	待定	java游戏开发	11k+13k+100k/每月	>16.8w
6. 公司	待定	java游戏开发	9k+10k+100k/每月	9k
7. 公司	待定	java游戏开发	9k+10k+100k/每月	9k
8. 公司	待定	java游戏开发	9k+10k+100k/每月	9k
9. 公司	待定	java游戏开发	9k+10k+100k/每月	9k

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>